

Componentes/BonusSales/STBBonusSales.prw

725 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static aBfPrdVal := {}           //Static para acumular o valor da bonificação por produto, utilizado em STBAcPrdBf() e STBDistrBf()
5
6 //-----
7 /*{Protheus.doc} STBBonusSales
8 Verifica se a bonificacao passada por parametro sera contemplada
9 @param      cRegraBon      Codigo da regra de bonificacao
10 @param      cTesBonus      TES para Bonificacao
11 @param      cCodProd       Codigo do produto
12 @return     .T. se a bonificacao se aplica / .F. Se houve falha em algum processo
13 @obs
14 @sample
15 */
16 //-----
17 Function STBBonusSales(cRegraBon,cTesBonus,cCodProd)
18
19 Local aArea      := GetArea()           // Guarda area corrente
20 Local lContinua  := .T.                 // Verifica se continua o processamento
21 Local lBonifica  := .T.                 // Retorno da funcao - Verifica se a regra de bonificao sera aplicada
22 Local cCliente   := STDGPBasket( "SL1" , "L1_CLIENTE" ) // Codigo do cliente na venda
23 Local cLoja      := STDGPBasket( "SL1" , "L1_LOJA" )   // Codigo da loja do cliente na venda
24 Local cCondPag    := STDGPBasket( "SL1" , "L1_CONDPG" ) // Condicao de pagamento da venda
25 Local cFormPg     := STDGPBasket( "SL1" , "L1_FORMPG" ) // Forma de pagamento da venda
26 Local nQuantTotal := 0                  // Quantidade total do item
27 Local nQuantItem  := STDGPBasket( "SL2" , "L2_QUANT", STDPBLength( "SL2" ) ) // Quantidade do item lancado
28 Local nQuantVerf  := 0                  // Quantidade ja existente na venda
29 Local cGrpClient  := ""                 // Grupo de clientes
30 Local lAccBonus    := SuperGetMv("MV_TPONUS",.F.,.T.) // Indica se a regra de bonificacao sera acumulada ou nao
31 Local aUsedRules   := STDBSGetRules( )   // Array contendo as regras de bonificacao ja utilizadas na venda
32 Local nPosRegra    := 0                  // Posicao da regra dentro do array de regras utilizadas
33 Local nTimesUsed   := 0                  // Numero de vezes que a regra foi utilizada
34 Local nLote        := 0                  // numero de produtos que devem ser comprados para validar a regra
35 Local nX           := 0                  // Contador do For
36
37 Default cRegraBon  := "" // Codigo da regra de bonificacao
38 Default cTesBonus  := SuperGetMv("MV_BONUSTS") // Codigo da regra de bonificacao
39 Default cCodProd   := "" // Codigo do produto que esta sendo registrado.
40
41 ParamType 0 Var cRegraBon AS Character Default ""
42 ParamType 1 Var cTesBonus AS Character Default SuperGetMv("MV_BONUSTS")
43 ParamType 2 Var cCodProd AS Character Default ""
44
45 If Len(aUsedRules) > 0
46     nPosRegra := aScan(aUsedRules,{|x| AllTrim(x[1]) == AllTrim(cRegraBon)})
47     If nPosRegra > 0
48         If !lAccBonus // Caso a bonificacao nao seja cumulativa, verifico se a regra ja foi utilizada.
49             lContinua := .F.
50         Else
51             nTimesUsed := aUsedRules[nPosRegra,2]
52     EndIf

```

```

53     EndIf
54 EndIf
55
56 If lContinua
57     //Verifica se a tes eh valida
58     If Empty(cTesBonus)
59         lContinua := .F.
60     Else
61         //Trata se parâmetro for numerico
62         If ValType(cTesBonus) == "N"
63             cTesBonus := AllTrim(cValToChar(cTesBonus))
64         ElseIf ValType(cTesBonus) == "C"
65             cTesBonus := AllTrim(cTesBonus)
66         EndIf
67
68         dbSelectArea("SF4")
69         dbSetOrder(1)
70         If SubStr(cTesBonus,1,3) <= "500" .Or. !DbSeek(xFilial("SF4")+cTesBonus)
71             lContinua := .F.
72         EndIf
73     EndIf
74 EndIf
75
76 If lContinua
77     DbSelectArea("ACQ")
78     ACQ->(DbSetOrder(1))
79     If ACQ->(DbSeek(xFilial("ACQ")+cRegraBon))
80
81         // Verifico se a venda se enquadra nos parametros mais basicos da regra de bonificacao
82         If ((AllTrim(ACQ->ACQ_CODCLI) == AllTrim(cCliente) .Or. Empty(ACQ->ACQ_CODCLI) ).And.;
83             (AllTrim(ACQ->ACQ_LOJA) == AllTrim(cLoja) .Or. Empty(ACQ->ACQ_LOJA) ) .And.;
84             (AllTrim(ACQ->ACQ_CONDPG) == AllTrim(cCondPag) .Or. Empty(ACQ->ACQ_CONDPG) ) .And.;
85             (AllTrim(ACQ->ACQ_FORMPG) == AllTrim(cFormPg) .Or. Empty(ACQ->ACQ_FORMPG)))
86
87             cGrpClient := ACQ->ACQ_GRPVEN
88
89             If !Empty(cGrpClient)
90                 // Avalio se o Grupo de clientes esta preenchido na regra de bonificacao e se o cliente se enquadra
91                 If cGrpClient != Posicione("SA1",1,xFilial("ACQ")+cCliente+cLoja,"A1_GRPVEN")
92                     lBonifica := .F.
93                 EndIf
94             EndIf
95
96             If lBonifica
97                 // Verificacao da data e hora
98                 lBonifica := STBVldDateBonus()
99             EndIf
100
101             If lBonifica
102                 // Verificacao de quantidade
103
104                 If !Empty(ACQ->ACQ_LOTE)

```

```

105         nLote := ACQ->ACQ_LOTE
106     EndIf
107
108     If ACQ->ACQ_TPRGBN == "1"
109         /*
110         Verifica se todos os itens da regra de bonificacao estao na venda e se atende a regra.
111         */
112
113         // Caso ACQ_LOTE esteja preenchido no cabecalho, somente ele sera considerado.
114         // Caso contrario, sera feita a busca pelo lote de cada item.
115         lBonifica := STBItensBonus( cRegraBon, ACQ->ACQ_LOTE, nTimesUsed )
116
117     ElseIf ACQ->ACQ_TPRGBN == "2"
118         // Caso haja a necessidade de comprar Somente Um dos produtos bonificadores.
119         nQuantTotal := STDQuantProd(cCodProd)
120
121         If Empty(nLote) // Caso o lote de quantidade do cabecalho nao tenha sido preenchido
122             nLote := STDBSGtQtd( cRegraBon, cCodProd )
123         EndIf
124
125         If nLote > nQuantTotal
126             lBonifica := .F.
127         ElseIf nQuantTotal > nLote // Caso a quantidade total do produto seja maior que o numero do lote, verifico se a regra foi contemplada novamente.
128
129             /*
130             Abaixo, nQuantVerf recebe o resto (caso exista) da regra de bonificacao, mais os itens recém lançados.
131             Isso é necessario para reavaliar se a regra sera aplicada.
132             */
133             nQuantVerf := ((nQuantTotal - nQuantItem)%nLote)+nQuantItem
134             lBonifica := .F.
135             For nX := 1 To nQuantVerf
136                 If nX % nLote == 0
137                     lBonifica := .T.
138                     Exit
139                 EndIf
140             Next nX
141         EndIf
142     EndIf
143 EndIf
144 Else
145     lBonifica := .F.
146 EndIf
147 EndIf
148 Else
149     lBonifica := .F.
150 EndIf
151
152 If lBonifica
153
154     If nPosRegra > 0 // Caso a regra ja tenha sido utilizada na venda, incrementa o contador do numero de vezes que a regra foi aplicada
155         aUsedRules[nPosRegra,2] := nTimesUsed+1
156     Else

```

```

157      Aadd(aUsedRules,{cRegraBon,1,cCodProd})
158  EndIf
159
160  STDBS7Rules( aUsedRules )
161
162 EndIf
163
164 RestArea(aArea)
165
166 Return lBonifica
167
168
169 //-----
170 /*{Protheus.doc} STBVldDateBonus
171 Avalia se a venda esta dentro da data e hora especificada para contemplar a regra da bonificacao.
172 @return lRet - retorna se Validou a bonificacaoa bonificacao
173 @obs
174 @sample
175 */
176 //-----
177 Static Function STBVldDateBonus()
178
179 Local lRet      := .F.           // Retorno
180 Local cHoraAtual := SUBSTR(Time(), 1, 5) // Pegu apenas as horas e minutos atuais.
181
182 //Caso o tipo de horario seja unico (Regra eh valida da hora inicial do primeiro dia até a hora final do último dia)
183 If ACQ->ACQ_TPHORA == "1"
184
185     // Caso a data atual esteja entre a Data Inicial e a Data Final da Regra de Bonificacao
186     If dDataBase > ACQ->ACQ_DATDE .And. (dDataBase < ACQ->ACQ_DATATE .Or. Empty(ACQ->ACQ_DATATE))
187         lRet := .T.
188
189     //Caso a data atual seja maior que a data inicial e a data final esteja em branco.
190     ElseIf dDataBase > ACQ->ACQ_DATDE .AND. Empty(ACQ->ACQ_DATATE)
191         lRet := .T.
192
193     // Caso a Data atual seja igual a Data Inicial da Regra de Bonificacao
194     ElseIf dDataBase == ACQ->ACQ_DATDE
195
196         //Se a hora atual for maior ou igual que a hora inicial da Regra de Bonificacao
197         If cHoraAtual >= ACQ->ACQ_HORADE
198             lRet := .T.
199         EndIf
200
201     // Caso a Data atual seja igual a Data Final da Regra de Bonificacao
202     ElseIf dDataBase == ACQ->ACQ_DATATE
203
204         //Se a hora atual for menor ou igual que a hora final da Regra de Bonificacao
205         If cHoraAtual <= ACQ->ACQ_HORATE
206             lRet := .T.
207         EndIf
208     EndIf

```

```

209
210 /*
211     Caso o tipo de horario seja recorrente (Regra valida do dia inicial ao final, somente durante os horarios
212     que estiverem entre o horario inicial e o horario final).
213 */
214 ElseIf ACQ->ACQ_TPHORA == "2"
215
216     // Caso a data atual seja maior ou igual a data inicial da regra da bonificacao
217     If dDataBase >= ACQ->ACQ_DATDE
218
219         // Caso a data atual seja menor ou igual a data final da regra da bonificacao, ou caso a data final esteja vazia
220         If dDataBase <= ACQ->ACQ_DATATE .OR. Empty(ACQ->ACQ_DATATE)
221
222             // Caso a hora atual seja maior ou igual que a hora inicial E menor ou igual que a hora final da regra da bonificacao
223             If cHoraAtual >= ACQ->ACQ_HORADE .AND. cHoraAtual <= ACQ->ACQ_HORATE
224                 lRet := .T.
225             EndIf
226         EndIf
227     EndIf
228 EndIf
229 Return lRet
230
231
232
233 //-----
234 /*{Protheus.doc} STBItensBonus
235 Verifica se todos os produtos foram comprados e se a quantidade atende a regra de bonificacao
236
237 @param   cRegraBon - Codigo da regra da bonificacao
238 @param   nLoteCab  - Conteudo do campo lote do cabeçalho. Se vier zerado, sera considerado o lote de cada item.
239 @return  .T. se atende a bonificacao / .F. Se nao atende a bonificacao
240 @obs
241 @sample
242 */
243 //-----
244 Function STBItensBonus( cRegraBon, nLoteCab, nTimesUsed )
245
246 Local aArea           := GetArea()    // Guarda area
247 Local lRet            := .T.          // Retorno
248 Local nQuant          := 0            // Quantidade
249 Local nLote           := 0            // Lote
250 Local lLoteCab        := .F.          // Lote Cabeçalho
251 Local nX              := 0            // Contador do For
252 Local lAchou          := .F.          // Verifica se o item foi encontrado
253
254 DEFAULT cRegraBon     := ""
255 DEFAULT nLoteCab      := 0
256
257 ParamType 0 Var cRegraBon AS Character Default ""
258 ParamType 1 Var nLoteCab  AS Numeric   Default 0
259
260 If !Empty(nLoteCab) //Se algum valor for passado, somente o nLoteCab sera considerado.

```

```

261     lLoteCab := .T.
262 EndIf
263
264 DbSelectArea("ACR")
265 ACR->(DbSetOrder(1))
266 If ACR->(MsSeek(xFilial("ACR")+cRegraBon))
267
268     // Quando o cabeçalho esta preenchido, a quantidade de todos os itens que fazem parte da regra
269     // de bonificacao devem ser maior do que o lote do cabeçalho.
270     If lLoteCab
271
272         While !EOF() .AND. ACR->ACR_FILIAL == xFilial("ACR") .AND. ACR->ACR_CODREG == cRegraBon
273
274             lAchou := .F.
275
276             For nX := 1 To STDPBLength("SL2")
277
278                 If ACR->ACR_CODPRO == STDGPBasket( "SL2" , "L2_PRODUTO" , nX ) // Verifica se o produto existe na cesta
279                     lAchou := .T.
280                     nQuant += STDQuantProd(ACR->ACR_CODPRO)
281                     Exit
282                 EndIf
283
284             Next nX
285
286             If !lAchou
287                 Exit
288             EndIf
289
290             ACR->(DbSkip())
291         EndDo
292
293         If !lAchou
294             lRet := .F.
295         EndIf
296
297         If lRet .AND. nLoteCab > nQuant
298             lRet := .F.
299         EndIf
300
301     Else
302         While !EOF() .AND. ACR->ACR_FILIAL == xFilial("ACR") .AND. ACR->ACR_CODREG == cRegraBon
303
304             nQuant := STDQuantProd(ACR->ACR_CODPRO)
305             nLote := ACR->ACR_LOTE
306
307             If nTimesUsed > 0
308                 nLote := nLote * (nTimesUsed+1)
309             EndIf
310
311             If nLote > nQuant
312                 lRet := .F.

```

```

313             Exit
314         EndIf
315         ACR->(DbSkip())
316
317     EndDo
318 EndIf
319 EndIf
320
321 RestArea(aArea)
322
323 Return lRet
324
325 //-----
326 /*{Protheus.doc} STBPrdSale
327 Essa função tem por objetivo verificar quais os itens lançados
328 na venda e retornar um array contendo o código do produto e a
329 quantidade vendida.
330
331 @return aItens
332 { 1: Produto SL2,
333   2: Quantidade SL2,
334   3: Quantidade MB8 que será lido em STBPrdBonf(),
335   4: Array lido em STDPesqMGB()
336     {1: Produto MGB, 2: Quantidade MGB},
337   5: Categoria (Se Houver)
338 }
339 */
340 //-----
341 Function STBPrdSale()
342
343 Local oModelCesta := STDGPBModel() //Model completo da cesta de vendas
344 Local oItens      := oModelCesta:GetModel('SL2DETAIL') //Model somente da SL2
345 Local aItens      := {} //Array contendo todos os itens da venda agrupados por item
346 Local nIt        := 0 //Posicao do item no array
347 Local nX          := 0 //Variavel de loop
348 Local cCateg      := "" //Código da Categoria
349
350 For nX := 1 To oItens:Length()
351     oItens:GoLine(nX)
352     If !oItens:IsDeleted()
353         nIt := aScan(aItens, {|x| AllTrim(x[1]) == AllTrim(oItens:GetValue('L2_PRODUTO')) })
354         If nIt > 0
355             aItens[nIt][2] += oItens:GetValue('L2_QUANT')
356         Else //Categoria, se houver
357             cCateg := STBCdCt(oItens:GetValue('L2_PRODUTO'))
358             If Empty(cCateg) //Produto
359                 aAdd(aItens,{oItens:GetValue('L2_PRODUTO'), oItens:GetValue('L2_QUANT'), 0, {}, ""})
360             Else //Categoria
361                 nIt := aScan(aItens, {|x| AllTrim(x[5]) == Alltrim(cCateg) }) //Pesquisa por Categoria
362                 If nIt > 0
363                     aItens[nIt][2] += oItens:GetValue('L2_QUANT')
364                 Else

```

```

365             aAdd(aItens,{"", oItens:GetValue('L2_QUANT'), 0, {}, cCateg})
366         EndIf
367     EndIf
368 EndIf
369 EndIf
370 Next nX
371
372 Return aItens
373
374 //-----
375 /*{Protheus.doc} STBPrdBonf
376 Essa função tem por objetivo verificar dos itens que foram vendidos,
377 quais o cliente tem direito de ganhar a bonificacao.
378
379 @return Nil
380 */
381 //-----
382 Function STBPrdBonf()
383
384 Local aItens      := STBPrdSale() //Retorna as qtd dos produtos/categorias da venda aglutinados
385 Local aRegras     := IIF(ExistFunc('STDPesqMei'),STDPesqMei(),{}) //Busca as regras ativas
386 Local lFunc       := ExistFunc('STDPesqMb8') .AND. ExistFunc('STDPesqMgb')
387 Local aProdBonif := {} //Guarda os produtos
388 Local nX          := 0 //Variavel de loop
389 Local nI          := 0 //Variavel de loop
390
391 //Verifica quais as regras que estao ativas
392 If lFunc
393     For nX := 1 To Len(aRegras)
394         For nI := 1 To Len(aItens)
395
396             If (aItens[nI][3] = 0) //Se aItens[nI][3] tiver maior que 0, não entra, pois já preencheu sua quantidade na MB8 e seu produto vinculado
397                 //Verifica se para cada item da venda contém uma regra de bonificacao
398                 If STDPesqMb8(aRegras[nX], aItens[nI][1], aItens[nI][5]) //aItens[nI][1]=Produto lançado e aItens[nI][2]=Qtde. lançada. aItens[nI][5]=Categoria.
399                     aProdBonif := STDPesqMgb(aRegras[nX], PadL(MB8->MB8_REFGRD, TamSX3("MB8_REFGRD")[1], "0") + "MB8")
400                     aItens[nI][3] := MB8->MB8_QTDPRO
401                     aItens[nI][4] := aProdBonif
402                 EndIf
403             EndIf
404
405         Next nI
406     Next nX
407 EndIf
408
409 Return aItens
410
411 //-----
412 /*{Protheus.doc} STBValBonf
413 Essa função tem por objetivo verificar retornar o valor
414 que sera bonificado.
415
416 @return nValBonfTot (Valor total bonificado)

```

```

417 */
418 //-----
419 Function STBValBonf()
420
421 Local aItens      := STBPrdBonf()      //Aglutinado por Produto/Categoria com seu valor bonificado
422 Local nX          := 0                //Variavel de loop
423 Local nI          := 0                //Variavel de loop
424 Local nValBonfTot := 0                //Retorna o valor que sera bonificado
425 Local cProd       := ''              //Recebe o codido do produto
426 Local nQtdBonificar := 0             //Quantidade que sera bonificado
427 Local nQtdBonfIt  := 0               //Quantidade Bonificada por Item
428 Local nValBonfIt   := 0              //Valor Bonificado por Item
429 Local lTemCateg    := .F.            //0 produto bonificado CONSTA no produto que pertence a uma CATEGORIA
430
431 STBACzrBf() //Zera valor por produto bonificado
432
433 //Verifica o valor total da bonificacao
434 For nX := 1 To Len(aItens)
435
436     If Len(aItens[nX]) > 3
437
438         For nI := 1 To Len(aItens[nX][4])
439
440             cProd := AllTrim(aItens[nX][4][nI][1])
441
442             If !Empty(cProd)
443                 nIt := aScan(aItens, {|x| AllTrim(x[1]) == AllTrim(cProd) })
444                 lTemCateg := Iif(nIt == 0 .AND. !Empty(aItens[nX][5]), STBConstCt(aItens[nX][5],cProd), .F.) //0 produto bonificado CONSTA no produto que pertence
a uma CATEGORIA
445
446                 If (nIt > 0) .OR. lTemCateg
447
448                     If (Alltrim(aItens[nX][1]) == cProd .AND. Empty(aItens[nX][5])); //Se for O MESMO PRODUTO tanto bonificador como bonificado, a fórmula abaixo
calculará a qtd. bonificada. Ex: Se 10 e bonificou 1: Eu teria que vender manualmente 11 e não 10 para atingir 1 bonificação.
449                                     .OR. (lTemCateg) //Idem para Categoria
450                     nQtdBonfIt := STBBfSamePrd(aItens[nX][2],aItens[nX][3],aItens[nX][4][nI][2])
451                     Else
452                         nQtdBonificar := Int(aItens[nX][2] / aItens[nX][3]) * aItens[nX][4][nI][2] //Quantidade do item vendido SL2 / Quantidade do item bonificado MB8 *
a Quantidade Máxima da Bonificação por item relacionado MGB
453                         nQtdBonfIt := IIF(nQtdBonificar > aItens[nIt][2], aItens[nIt][2], nQtdBonificar)
454                     EndIf
455                     nValBonfIt := STBSearchPrice(aItens[nX][4][nI][1]) * nQtdBonfIt
456
457                     If nQtdBonfIt > 0
458
459                         nValBonfTot += nValBonfIt
460                         STBACPrdBf(cProd,nQtdBonfIt,nValBonfIt) //Aglutino o valor bonificado por aquele produto
461
462                     EndIf
463
464                 EndIf
465             EndIf

```

```

466
467         Next nI
468
469     EndIf
470
471 Next nX
472
473 STBDistrBf() //Distribuir o valor bonificado na SL2
474
475 Return nValBonfTot
476
477 //-----
478 /*{Protheus.doc} STBFormBf
479 Funcao responsavel em incluir a forma de pagamento bonificacao
480 na tela de pagamento
481 OBS: Entrará somente se foi validado na Regra de Desconto.
482
483 @return lRet (se nVlBonif > 0)
484 */
485 //-----
486 Function STBFormBf()
487
488 Local nVlBonif := 0
489 Local lRet     := .F.
490
491 If GetAPOInfo("STFTOTALUPDATE.PRW")[4] >= Ctod("19/12/2019")
492     nVlBonif := STBValBonf()
493     STFSetTot( "L1_BONIF", nVlBonif )
494     If nVlBonif > 0
495         lRet := .T.
496     EndIf
497 EndIf
498
499 Return lRet
500
501
502 //-----
503 /*{Protheus.doc} STBAcZrBf()
504 Funcao responsavel em zerar a variável statica de valor da bonificação por produto
505 OBS: Entrará somente se foi validado na Regra de Desconto.
506
507 @return nil
508 */
509 //-----
510 Function STBAcZrBf()
511
512 aBfPrdVal := {}
513
514 Return nil
515
516
517 //-----

```

```

518 /*{Protheus.doc} STBacPrdBf()
519 Funcao responsavel em acumular a variável statica de valor da bonificação por produto
520 OBS: Entrará somente se foi validado na Regra de Desconto.
521
522 Composição do elemento em aBfPrdVal:
523 1-Produto
524 2-Quantidade a bonificar em STBacPrdBf() antes de distribuir em STBDistrBF()
525 3-Valor total a bonificar em STBacPrdBf() antes de distribuir em STBDistrBF()
526 4-Quantidade a bonificar após distribuir em STBDistrBF()
527 5-Valor total a bonificar após distribuir em STBDistrBF()
528 Para os itens 4 e 5, é válido para cálculo de SALDO TOTAL.
529
530 @return nil
531 */
532 //-----
533 Function STBacPrdBf(cProd,nQtdBonfIt,nValBonfIt)
534
535 Local nPosProd := 0
536
537
538 Default nQtdBonfIt := 0
539 Default nValBonfIt := 0
540
541 nPosProd := aScan(aBfPrdVal,{|x| AllTrim(x[1]) == AllTrim(cProd)}) //Pesquise o produto na static
542 If nPosProd > 0
543     aBfPrdVal[nPosProd][2] += nQtdBonfIt
544     aBfPrdVal[nPosProd][3] += nValBonfIt
545 Else
546     aAdd(aBfPrdVal, {cProd,nQtdBonfIt,nValBonfIt,0,0})
547 EndIf
548
549 Return nil
550
551
552 //-----
553 /*{Protheus.doc} STBDistrBf()
554 Funcao responsavel em distribuir os valores bonificados por produto na SL2
555 OBS: Entrará somente se foi validado na Regra de Desconto.
556
557 Composição do elemento em aBfPrdVal:
558 1-Produto
559 2-Quantidade a bonificar em STBacPrdBf() antes de distribuir em STBDistrBF()
560 3-Valor total a bonificar em STBacPrdBf() antes de distribuir em STBDistrBF()
561 4-Quantidade a bonificar após distribuir em STBDistrBF()
562 5-Valor total a bonificar após distribuir em STBDistrBF()
563 Para os itens 4 e 5, é válido para cálculo de SALDO TOTAL.
564
565 @return nil
566 */
567 //-----
568 Function STBDistrBf()
569

```

```

570 Local oModelCesta := STDGPBModel() //Model completo da cesta de vendas
571 Local oItens      := oModelCesta:GetModel('SL2DETAIL') //Model somente da SL2
572 Local nItQuant    := 0
573 Local nItValor     := 0
574 Local nSldQuant    := 0
575 Local nSldValor    := 0
576 Local nX           := 0
577 Local lVazio       := .F.
578
579 //Revisão de bonificação por item em SL2
580 For nX := 1 To oItens:Length()
581     lVazio := .F.
582     oItens:GoLine(nX)
583
584     nPos := Ascan(aBfPrdVal, {|x| Alltrim(x[1]) = Alltrim(oItens:GetValue('L2_PRODUTO')) })
585     If nPos > 0 //0 item do produto foi bonificado
586         nItQuant := oItens:GetValue('L2_QUANT')
587         nItValor := oItens:GetValue('L2_VLRITEM')
588         nSldQuant := aBfPrdVal[nPos][2]-aBfPrdVal[nPos][4]
589         nSldValor := aBfPrdVal[nPos][3]-aBfPrdVal[nPos][5]
590
591         If nSldQuant > 0 //Gravo só se tiver saldo
592             STDSPBasket( "SL2" , "L2_PREMIO" , "B" , nX )
593             If nSldQuant < nItQuant
594                 STDSPBasket( "SL2" , "L2_VLDESRE", nSldValor, nX ) //Atribuo o Saldo do valor da bonificação
595                 aBfPrdVal[nPos][4] := aBfPrdVal[nPos][4] + nSldQuant //Assinalo como Produto Bonificado
596                 aBfPrdVal[nPos][5] := aBfPrdVal[nPos][5] + nSldValor //Por enquanto, utilizaremos em L2_VLDESCR a gravação do valor bonificado
597             Else
598                 STDSPBasket( "SL2" , "L2_VLDESRE", nItValor, nX ) //Atribuo o valor do item
599                 aBfPrdVal[nPos][4] := aBfPrdVal[nPos][4] + nItQuant //Assinalo como Produto Bonificado
600                 aBfPrdVal[nPos][5] := aBfPrdVal[nPos][5] + nItValor //Por enquanto, utilizaremos em L2_VLDESCR a gravação do valor bonificado
601             EndIf
602         Else
603             lVazio := .T. //Item sem bonificação
604         EndIf
605     Else
606         lVazio := .T. //Item sem bonificação
607     EndIf
608
609     If lVazio //Se item não foi vendido como bonificado, gravo em branco
610         STDSPBasket( "SL2" , "L2_PREMIO" , "" , nX ) //Assinalo como Produto Bonificado
611         STDSPBasket( "SL2" , "L2_VLDESRE", 0 , nX ) //Por enquanto, utilizaremos em L2_VLDESCR a gravação do valor bonificado
612     EndIf
613
614 Next nX
615
616 Return nil
617
618
619 //-----
620 /*{Protheus.doc} STBBfSamePrd()
621 Utilizado somente se o produto bonificado tem correlação com o MESMO produto

```

```

622 OBS: Entrará somente se foi validado na Regra de Desconto.
623
624 Exemplo:
625 A cada 50 celulares, o 51º será bonificado.
626 Logo, deverá lançar 51 celulares para receber automaticamente 1 produto bonificado.
627
628 Caso a cada 100 unidades, ganha 5 de bonificação, obedeceremos a seguinte escala:
629 * Lembrando que temos que lançar 105 para receber 5 de bonificação *
630 Se lançou 100 (entradas 100,100,5), retorna 0.
631 Se lançou 101 (entradas 101,100,5), retorna 1.
632 Se lançou 102 (entradas 102,100,5), retorna 2.
633 Se lançou 103 (entradas 103,100,5), retorna 3.
634 Se lançou 104 (entradas 104,100,5), retorna 4.
635 Se lançou 105 (entradas 105,100,5), retorna 5.
636 Se lançou 106 até 205, recebe 5.
637 Se lançou 206 (entradas 206,100,5), retorna 6.
638 Se lançou 207 (entradas 207,100,5), retorna 7.
639 Se lançou 208 (entradas 208,100,5), retorna 8.
640 Se lançou 209 (entradas 209,100,5), retorna 9.
641 Se lançou 210 (entradas 210,100,5), retorna 10.
642 Se lançou 211 até 310, recebe 10.
643 Se lançou 311 (entradas 311,100,5), retorna 11.
644 Se lançou 312 (entradas 312,100,5), retorna 12.
645 Se lançou 313 (entradas 313,100,5), retorna 13.
646 Se lançou 314 (entradas 314,100,5), retorna 14.
647 Se lançou 315 (entradas 315,100,5), retorna 15.
648
649 @return nRet (a qtd. bonificada)
650 */
651 //-----
652 Static Function STBBfSamePrd(nQtdVend,nQtdaBonfMB8,nQtdBonfMGB)
653
654 Local nResto      := 0      //0 resto da divisão da quantidade da venda sobre a quantidade bonificada
655 Local nMb8maisMgb := 0      //Soma da quantidade a bonificar MB8 + quantidade bonificada MGB
656
657 Default nQtdVend   := 0      //Quantidade de item vendido SL2
658 Default nQtdaBonfMB8 := 0    //Quantidade A BONIFICAR MB8
659 Default nQtdBonfMGB := 0     //Quantidade BONIFICADA MGB
660
661 nMb8maisMgb := nQtdaBonfMB8+nQtdBonfMGB //Somo as duas quantidades: a bonificar MB8 e a bonificada MGB
662
663 nRet := Int(nQtdVend / nMb8maisMgb) * nQtdBonfMGB
664 nResto := nQtdVend % nMb8maisMgb //Resto de qtd. vendida e nMb8maisMgb
665
666 If nResto > nQtdaBonfMB8
667     nRet := nRet + (nResto-nQtdaBonfMB8)
668 EndIf
669
670 Return nRet
671
672
673 //-----

```

```

674 /*{Protheus.doc} STBCdCt
675 Retorna o primeiro código da categoria daquele produto mencionado
676
677 @params  cCodProd, Carácter, Código do Produto
678 @return  cCodCat, carácter, Código da Categoria
679 */
680 //-----
681 Static Function STBCdCt(cCodProd)
682
683 Local cCodCat := ""
684 Local aArea := GetArea()
685
686 Default cCodProd := ""
687
688 DbSelectArea("ACV")
689 DbSetOrder(5) //ACV_FILIAL+ACV_CODPRO+ACV_CATEG0
690
691 If ACV->(DbSeek(xFilial("ACV")+PadR(Alltrim(cCodProd),TamSx3("ACV_CODPRO")[1])))
692     cCodCat := ACV->ACV_CATEG0
693 EndIf
694
695 RestArea( aArea )
696 Return cCodCat
697
698
699 //-----
700 /*{Protheus.doc} STBConstCt
701 Verifica se o produto faz parte da categoria
702
703 @params  cCodCat, Carácter, Código da Categoria
704 @params  cCodProd, Carácter, Código do Produto
705 @return  lRet, Lógico, Se o produto pertence à categoria
706 */
707 //-----
708 Function STBConstCt(cCodCat, cCodProd)
709
710 Local lRet := .F.
711 Local aArea := GetArea()
712
713 Default cCodCat := ""
714 Default cCodProd := ""
715
716 DbSelectArea("ACV")
717 DbSetOrder(5) //ACV_FILIAL+ACV_CODPRO+ACV_CATEG0
718
719 If ACV->(DbSeek(xFilial("ACV")+PadR(Alltrim(cCodProd),TamSx3("ACV_CODPRO")[1])+PadR(Alltrim(cCodCat),TamSx3("ACV_CATEG0")[1]))) //Verifica se o produto faz parte da categoria
720     lRet := .T.
721 EndIf
722
723 RestArea( aArea )
724
725 Return lRet

```

Componentes/BonusSales/STDBonusSales.prw

394 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDBONUSSALES.CH"
4
5 Static aUsedRules := {}
6
7 //-----
8 /*{Protheus.doc} STDBonusProduto
9 Busca o produto que sera contemplado pela regra de bonificacao e seta a quantidade.
10 @param      cRegraBon      Codigo da regra de bonificacao
11
12 @return  cRet - Codigo do Produto
13 @obs
14 @sample
15 */
16 //-----
17 Function STDBonusProduto(cRegraBon)
18
19 Local aArea      := GetArea()      // Guarda area
20 Local cRet       := ""             // Retorno
21 Local nQuant     := 0              // quantidade
22
23 Default cRegraBon := ""           // Codigo da regra de bonificacao
24
25 ParamType 0 Var cRegraBon AS Character Default ""
26
27 DbSelectArea("ACQ")
28 ACQ->(DbSetOrder(1))
29 If ACQ->(MSeek(xFilial("ACQ")+cRegraBon))
30     cRet := ACQ->ACQ_CODPRO
31     nQuant := ACQ->ACQ_QUANT
32
33     STBSetQuant(nQuant)
34 EndIf
35
36 RestArea(aArea)
37
38 Return cRet
39
40
41 //-----
42 /*{Protheus.doc} STDQuantProd
43 Retorna a quantidade total do produto que esta contido na cesta.
44 @param  cProd - Codigo do produto que esta sendo registrado
45 @return nRet Quantidade do produto
46 @obs
47 @sample
48 */
49 //-----
50 Function STDQuantProd(cProd)
51
52 Local nX      := 0

```

```

53 Local nRet      := 0
54 Local oCesta := STDGPBModel()
55
56 Default cProd   := ""    // Codigo da regra de bonificacao
57
58 ParamType 0 Var cProd AS Character Default ""
59
60 oCesta := oCesta:GetModel("SL2DETAIL")
61
62 For nX := 1 To STDPBLength("SL2")
63
64     oCesta:GoLine(nX)
65     If cProd == STDGPBasket( "SL2" , "L2_PRODUTO" , nX ) .AND. !oCesta:IsDeleted()
66         nRet += STDGPBasket( "SL2" , "L2_QUANT" , nX )
67     EndIf
68
69 Next nX
70
71 Return nRet
72
73 //-----
74 /*{Protheus.doc} STDBSFilter
75 Filtra as regras de bonificacao que contem o item que esta sendo registrado
76
77 @param  nItemLine - Numero do item da venda
78 @return Nil
79 @obs
80 @sample
81 */
82 //-----
83 Function STDBSFilter( nItemLine )
84 Local aArea      := GetArea()
85 Local cCodigo    := STDGPBasket( "SL2" , "L2_PRODUTO" , nItemLine )
86 Local cGrupoProd := Posicione("SB1",1,xFilial("SB1")+cCodigo,"B1_GRUPO")
87 Local cFilter    := ""
88 Local aRet       := {}
89 Local cKey       := ""
90
91 If !Empty(cGrupoProd)
92     cFilter := "" + AllTrim(cCodigo) + "" == AllTrim(ACR->ACR_CODPRO) .OR. "" + AllTrim(cGrupoProd) + "" == AllTrim(ACR->ACR_GRUPO)
93 Else
94     cFilter := "" + AllTrim(cCodigo) + "" == AllTrim(ACR->ACR_CODPRO)
95 EndIf
96
97 DbSelectArea("ACR")
98 ACR->(DbSetOrder(1)) // ACR_FILIAL+ACR_CODREG+ACR_ITEM
99 If ACR->(!EOF())
100     ACR->(DbGoTo(ACR->(LastRec())+1))
101 EndIf
102 ACR->(DbSetFilter({ || &cFilter }, cFilter))
103 ACR->(DbSkip(-1))
104

```

```

105 While ACR->(!BOF())
106
107     cKey := ACR->ACR_FILIAL+ACR->ACR_CODREG
108     If aScan(aRet,cKey) == 0
109         Add(aRet,cKey)
110     EndIf
111     ACR->(DbSkip(-1))
112 End
113 ACR->(DbClearFilter())
114
115 RestArea(aArea)
116
117 Return aRet
118
119
120 //-----
121 /*{Protheus.doc} STDSetBonusQuantidade
122 Seta a quantidade do produto que sera bonificado. Ao chamar esta rotina, todas as validacoes ja devem ter sido realizadas.
123
124 @param  cRegraBon - Codigo da regra de bonificacao
125 @return Nil
126 @obs
127 @sample
128 */
129 //-----
130 Function STDSetBonusQuantidade(cRegraBon)
131
132 Local aArea      := GetArea()    // Guarda area
133 Local nQuant     := 0            // Quantidade a ser bonificada
134 Local nLote      := 0            // Numero de itens a ser comprados para bonificar
135 Local nComprados := 0            // Numero de itens comprados
136 Local nMultiplo  := 0            // Multiplicador
137 Local lLoteCab   := .F.          // Indica se utiliza o lote do cabecalho
138 Local cTipoBon   := ""           // Tipo da bonificacao
139 Local lAccBonus  := SuperGetMv("MV_TPBONUS",.F.,.T.) // Indica se a regra de bonificacao sera acumulada ou nao
140 Local nTimesUsed := 0
141 Local nPosRegra  := 0
142
143 If Len(aUsedRules) > 0
144     nPosRegra := aScan(aUsedRules,{|x| AllTrim(x[1]) == AllTrim(cRegraBon)})
145     If nPosRegra > 0
146         nTimesUsed := aUsedRules[nPosRegra,2]
147     EndIf
148 EndIf
149
150 Default cRegraBon := ""           // Codigo da regra de bonificacao
151
152 ParamType 0 Var cRegraBon AS Character Default ""
153
154 DbSelectArea("ACQ")
155 ACQ->(DbSetOrder(1))
156 If ACQ->(MsSeek(xFilial("ACQ")+cRegraBon))

```

```

157     nQuant      := ACQ->ACQ_QUANT
158     cTipoBon    := ACQ->ACQ_TPRGBN
159     nLote       := ACQ->ACQ_LOTE
160 EndIf
161
162 If nLote > 0
163     lLoteCab    := .T.
164 EndIf
165
166 DbSelectArea("ACR")
167 ACR->(DbSetOrder(1)) // ACR_FILIAL+ACR_CODREG+ACR_ITEM
168 If ACR->(MsSeek(xFilial("ACR")+cRegraBon))
169     While !EOF() .AND. ACR->ACR_FILIAL == xFilial("ACR") .AND. ACR->ACR_CODREG == cRegraBon
170
171         If cTipoBon == "1" // Caso todos os produtos da regra tenham de ser vendidos.
172
173             If lLoteCab
174                 nComprados += STDQuantProd(ACR->ACR_CODPRO)
175             Else
176                 nComprados := STDQuantProd(ACR->ACR_CODPRO)
177
178                 If nMultiplo > 0
179                     /*
180                     Como todos os produtos devem ser comprados, apenas o menor multiplicador sera considerado
181                     Ex: Lote do produto A = 10 e Lote do produto B = 10 bonificam 1 unidade do produto C
182                     Caso 50 produtos A sejam vendidos e apenas 10 produtos B sejam vendidos, apenas 1 produto C deve ser bonificado
183                     */
184                     If nMultiplo > Int(nComprados/ACR->ACR_LOTE)
185                         nMultiplo := Int(nComprados/ACR->ACR_LOTE)
186                     EndIf
187                 Else
188                     nMultiplo := Int(nComprados/ACR->ACR_LOTE)
189                 EndIf
190             EndIf
191
192         ElseIf cTipoBon == "2"
193
194             If lLoteCab
195                 nComprados += STDQuantProd(ACR->ACR_CODPRO)
196             Else
197                 nComprados := STDQuantProd(ACR->ACR_CODPRO) // quantidade de determinado produto
198
199                 If lAccBonus // Indica se a bonificacao sera acumulada
200
201                     // Se acumula, apenas incremento o multiplicador
202                     nMultiplo += Int(nComprados/ACR->ACR_LOTE)
203                 Else
204                     // Se nao acumular, considero apenas o maior multiplicador
205                     If Int(nComprados/ACR->ACR_LOTE) > nMultiplo
206                         nMultiplo := Int(nComprados/ACR->ACR_LOTE)
207                     EndIf
208

```

```

209         EndIf
210     EndIf
211 EndIf
212     ACR->(DbSkip())
213 EndDo
214
215 If lLoteCab // Se utilizar o lote do cabecalho, o nMultiplo nao pode ser calculado durante o laco
216     nMultiplo := Int(nComprados/nLote)
217 EndIf
218
219 If lAccBonus
220     If nTimesUsed == 1 .AND. nMultiplo > 1
221         nQuant := nQuant * nMultiplo
222         aUsedRules[nPosRegra,2] := nMultiplo
223     ElseIf nMultiplo > nTimesUsed
224         nTimesUsed-= 1
225         nMultiplo -= nTimesUsed
226         If nMultiplo > 1
227             nQuant := nQuant * nMultiplo
228             aUsedRules[nPosRegra,2] += (nMultiplo-1)
229         EndIf
230     EndIf
231 EndIf
232
233 EndIf
234
235 STBSetQuant(nQuant)
236
237 RestArea(aArea)
238
239 Return nQuant
240
241 //-----
242 /*{Protheus.doc} STDBSGtQtd
243 Retorna o lote de quantidade do produto passado via parametro
244
245 @return Nil
246 */
247 //-----
248 Function STDBSGtQtd( cRegraBon, cCodProd )
249 Local aArea      := GetArea()
250 Local nRet       := 0
251
252 Default cRegraBon := "" // Codigo da regra de bonificacao que esta sendo avaliada
253 Default cCodProd  := "" // Codigo do produto a ser buscado
254
255 If !Empty(cRegraBon) .AND. !Empty(cCodProd)
256
257     DbSelectArea("ACR")
258     ACR->(DbSetOrder(1)) // ACR_FILIAL+ACR_CODREG+ACR_ITEM
259     If DbSeek(xFilial("ACR")+cRegraBon)
260         While (xFilial("ACR")+cRegraBon) == ACR->ACR_FILIAL+ACR->ACR_CODREG .AND. ACR->(!EOF())

```

```

261         If ACR->ACR_CODPRO == cCodProd
262             nRet := ACR->ACR_LOTE
263             Exit
264         EndIf
265         ACR->(DbSkip())
266     End
267 EndIf
268
269 EndIf
270
271 RestArea(aArea)
272
273 Return nRet
274
275 //-----
276 /*{Protheus.doc} STDBS7Rules
277 Seta a variavel static que guarda as regras de bonificacao utilizadas, alem do numero de vezes que ela foi utilizada.
278
279 @return Nil
280 */
281 //-----
282 Function STDBS7Rules( aRules )
283
284 DEFAULT aRules := {}
285
286 aUsedRules := aRules
287
288 Return
289
290 //-----
291 /*{Protheus.doc} STDBSGetRules
292 Limpa a variavel static que guarda as regras de bonificacao utilizadas
293
294 @return Nil
295 */
296 //-----
297 Function STDBSGetRules( )
298
299 Return aUsedRules
300
301 //-----
302 /*{Protheus.doc} STDPesqMei
303 Pesquisa as regras de desconto ativas e que sejam
304 regras de desconto por item
305
306 @return aMei -> Retorno das regras ativas
307 */
308 //-----
309 Function STDPesqMei()
310
311 Local aMei := {} //Variavel de retorno
312 Local cCustomer := STDGPBasket("SL1","L1_CLIENTE")

```

```

313 Local cLoja      := STDGPBasket("SL1","L1_LOJA")
314 Local lSTDValRule := ExistFunc("STDVALRULE") //Ver se está em Function antes de executar
315 Local aArea      := GetArea()
316
317 If lSTDValRule
318     dbSelectArea('MEI')
319     MEI->(DbSetOrder(1)) //MEI_FILIAL+MEI_CODREG
320     MEI->(DbSeek(xFilial('MEI')))
321
322     While MEI->(!EOF()) .AND. MEI->MEI_FILIAL == xFilial('MEI')
323         If MEI->MEI_ATIVA = "1" .AND. MEI->MEI_TPIMPD == "I" .AND. STDValRule( cCustomer , cLoja ) // Validacao generica por Data, Ativa, Cliente, Grupo de cliente, Filial e
            Prioridade
324             aAdd(aMei,MEI->MEI_CODREG)
325         EndIf
326         MEI->(DbSkip())
327     End
328 Else
329     MsgAlert(STR0001) //"Necessário que o RPO esteja com o fonte STDRuleDiscount.prw atualizado a partir de 01/11/2019!"
330 EndIf
331
332 RestArea(aArea)
333 Return aMei
334
335 //-----
336 /*{Protheus.doc} STDPesqMb8
337 Pesquisa em determina regra de desconto se determinado
338 produto esta contido naquela regra e se eh de bonificacao
339
340 @return lRet -> Retorna .T. se encontrou
341 */
342 //-----
343 Function STDPesqMb8(cCodReg, cCodProd, cCodCat)
344
345 Local lRet := .F. //Variavel de retorno
346 Local aArea := GetArea()
347
348 Default cCodReg := ""
349 Default cCodProd := ""
350 Default cCodCat := "" //Retorno Código de Categoria
351
352 dbSelectArea('MB8')
353 If !Empty(cCodCat) //Pesquisa Categoria
354     MB8->(DbSetOrder(3)) //MB8_FILIAL+MB8_CATEGO
355     If MB8->(DbSeek(xFilial('MB8')+cCodCat)) .AND. MB8->MB8_TPREGR = '2'
356         lRet := .T.
357     EndIf
358 Else
359     MB8->(DbSetOrder(4)) //MB8_FILIAL+MB8_CODREG+MB8_CODPRO
360     If MB8->(DbSeek(xFilial('MB8')+cCodReg+cCodProd)) .AND. MB8->MB8_TPREGR = '2'
361         lRet := .T.
362     EndIf
363 EndIf

```

```
364
365 RestArea(aArea)
366 Return lRet
367
368 //-----
369 /*{Protheus.doc} STDPesqMgb
370 Retorna qual sera o produto que sera bonificado para o
371 cliente
372
373 @return aRetMgb -> Contem o produto que sera bonificado
374 */
375 //-----
376 Function STDPesqMgb(cCodReg, cIdProd)
377
378 Local aRetMgb := {}
379 Local aArea   := GetArea()
380
381 Default cCodReg := ""
382 Default cIdProd := ""
383
384 dbSelectArea('MGB')
385 MGB->(DbSetOrder(1)) //MGB_FILIAL+MGB_CODREG+MGB_IDPROD
386 MGB->(DbSeek(xFilial('MGB')+cCodReg+cIdProd))
387
388 While MGB->(!EOF()) .AND. AllTrim(MGB->MGB_FILIAL+MGB_CODREG+MGB_IDPROD) == AllTrim(xFilial('MGB')+cCodReg+cIdProd)
389     aAdd(aRetMgb, {MGB->MGB_CODPROD, MGB->MGB_QTDPRO})
390     MGB->(DbSkip())
391 End
392
393 RestArea(aArea)
394 Return aRetMgb
```

Componentes/BonusSales/STWBonusSales.prw

84 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWBONUSSALES.CH"
4 //-----
5 /*{Protheus.doc} STWBonusSales
6 Executa as rotinas de Cenário de Vendas
7 @param oProdModel Objeto Model
8 @return lBonifica - .T. se foi executada sem erros / .F. Se houve falha em algum processo
9 @obs
10 @sample
11 */
12 //-----
13 Function STWBonusSales( nItemLine )
14
15 Local aArea      := GetArea()           // Guarda a area corrente
16 Local oCliModel  := STDGCLiModel()      // Model com as informacoes do cliente
17 Local lBonifica  := .F.                // Indica se a regra de bonificacao sera aplicada
18 Local cCodigo    := STDGPBasket( "SL2" , "L2_PRODUTO" , nItemLine ) // Codigo do produto que esta sendo lancado
19 Local cTesBonus  := ""                 // TES da bonificacao
20 Local cRegraBon  := ""                 // Codigo da regra de bonificacao
21 Local cBonusTd   := ""                 // Indica se sera utilizada mais de uma regra de bonificacao.
22 Local cProdBon   := ""                 // Codigo do produto bonificado.
23 Local cTypeDesc  := "V"               // Tipo de desconto (percentual)
24 Local lAddItem   := .T.                // Passado para o ItemReg. Indica que o item a ser registrado eh um item adicional.
25 Local aRegras    := STDBSFilter( nItemLine ) // Array com as regras de bonificacao que o item lancado faz parte
26 Local nX         := 0                  // Contador do For
27 Local nItens     := 0                  // Numero de itens que serao bonificados
28 Local nValDesc   := 0                  // Valor do desconto ( deixará o produto bonificado com o valor de R$0,01 )
29 Local nPrice     := 0                  // Preço do produto bonificado
30 Local nQuant     := 0                  // Quantidade que sera lancada do produto bonificado
31 Local nResto     := 0.01               // Resto que sobrara como valor do produto bonificado
32
33 For nX := 1 To Len(aRegras)
34
35     DbSeek(aRegras[nX])
36
37     cProdBon   := ""
38
39     cBonusTd   := SuperGetMv( "MV_BONUSTD",.F., "2" )
40     cTesBonus  := SuperGetMv( "MV_BONUSTS" )
41
42     cRegraBon  := ACR->ACR_CODREG
43
44     // verifica se a regra sera contemplada.
45     lBonifica  := STWBonusSales( cRegraBon, cTesBonus, cCodigo )
46
47     If lBonifica
48
49         nItens++
50         cProdBon := STDBonusProduto( cRegraBon ) // Funcao que retorna o codigo do produto que sera bonificado
51
52     If !Empty(cProdBon)

```

```

53
54     nQuant := STDSetBonusQuantidade( cRegraBon ) // Funcao que seta a quantidade que sera bonificada.
55
56     nPrice := STWFormPr(   cProdBon      , oCliModel:GetValue("SA1MASTER","A1_COD")  , Nil      , oCliModel:GetValue("SA1MASTER","A1_LOJA") , ;
57                          STBGetCurrency()
58                          )
59
60     nValDesc := (nPrice*nQuant) - nResto      // Calcula o valor do item bonificado para sobrar o valor de 1 centavo
61
62     STBSetItDiscount( nValDesc , cTypeDesc )
63     If !STWItemReg( nItemLine+nItens, cProdBon,Nil,Nil,STBGetCurrency(),nValDesc,cTypeDesc,lAddItem,cTesBonus,,,"BON" )
64         // Havendo falha no registro do item, gera nova mensagem e aborta a emissão dos demais
65         STFMessage("STBKitSales", "STOP", STR0001) // "Não foi possível registrar os itens que compoem o kit."
66         Exit
67     Else
68         STWTotDisc( nResto , "V" , "" , .T. )
69         STDSPBasket( "SL2" , "L2_BONIFICADOR" , .T. , nItemLine )
70     EndIf
71 EndIf
72
73 // Caso o conteudo do parametro MV_BONUSTD seja 2 nao sera processada mais nenhuma regra
74 If cBonusTd == "2"
75     Exit
76 EndIf
77
78 EndIf
79
80 Next nX
81
82 RestArea(aArea)
83
84 Return lBonifica

```

Componentes/CancelSale/STBCancelSale.prw

1339 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBCANCELSALE.CH"
4
5 Static lLjcFid      := SuperGetMv("MV_LJCFID",,.F.) .AND. CrdxInt()      //Indica se a recarga de cartao fidelidade esta ativa
6 Static cSiglaSat    := IIF( ExistFunc("LjSiglaSat"),LjSiglaSat(), "SAT" ) //Retorna sigla do equipamento que esta sendo utilizado
7 Static lPdvOn       := ExistFunc("STFPdvOn") .AND. STFPdvOn()          // Verifica se é PDV-Online
8
9 //-----
10 /*{Protheus.doc} STBCSIsProgressSale
11 Verifica se a venda está em andamento
12
13 @param  nenhum
14 @return lRet          Retorna se a venda está em andamento ou não
15 @obs
16 @sample
17 */
18 //-----
19 Function STBCSIsProgressSale()
20
21 Local lRet          := .F.          // Retorna se a venda está em andamento ou não
22 Local aRet          := {}           // Armazena retorno impressora
23 Local aIsOpenReceipt := { "5" , "" } // Armazena retorno se o cupom está aberto
24
25 /*
26     Se usa impressora Fiscal, verifica cupom aberto
27 */
28 If STFUseFiscalPrinter()
29
30     /*
31         Verifica cupom aberto
32     */
33     aRet := STFFireEvent(  ProcName(0)          , ; // Nome do processo
34                           "STPrinterStatus"    , ; // Nome do evento
35                           aIsOpenReceipt       )
36
37     If Len(aRet) > 0 .AND. ValType(aRet[1]) == "N"
38         If aRet[1] == 7
39             lRet := .T. // Aberto
40         Else
41             lRet := .F. // Fechado
42         EndIf
43     EndIf
44
45     /*
46         Verifica se existe venda com apenas itens não-fiscais
47     */
48     If !lRet
49
50         If STDPBLength("SL2") > 0
51
52             If !STBExistItemFiscal()

```

```

53         lRet := .T.
54     EndIf
55
56     EndIf
57
58     EndIf
59
60 Else
61
62     /*/
63     Verifica se existe item registrado
64     /*/
65     If STDPBLength("SL2") > 0
66         lRet := .T.
67     Else
68         lRet := .F.
69     EndIf
70
71 EndIf
72
73 Return lRet
74
75 //-----
76 /*/{Protheus.doc} STBCSNumPrinter
77 Busca o Ultimo Cupom da Impressora
78
79 @param
80 @return  cNumDoc          Retorna o ultimo cupom da impressora
81 @obs     Se der erro na impressora retorna "ERROR", se nao usa impressora retorna vazio
82 @sample
83 /*/
84 //-----
85 Function STBCSNumPrinter()
86
87 Local cNumDoc      := ""          // Retorno funcao
88 Local aPrinter     := {}         // Armazena retorno da impressora
89 Local aDados       := { "" , Nil } // Dados do evento
90
91 aPrinter := STFFireEvent( ProcName(0) , "STGetReceipt" , aDaDos )
92
93 If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aDados) < 1
94
95     STFMessage("STCancelSale","STOP", STR0003) //Falha na obtenção do cupom
96     STFShowMessage("STCancelSale")
97
98     cNumDoc := "ERROR"
99
100 Else
101     cNumDoc := aDados[1]
102 EndIf
103
104 Return cNumDoc

```

```

105
106
107 //-----
108 /*{Protheus.doc} STBCSGetPDV
109 Busca o PDV atual
110
111 @param
112 @return   aRet           Retorna se obteve e a numeração do PDV
113 @obs
114 @sample
115 */
116 //-----
117 Function STBCSGetPDV()
118
119 Local aPrinter := {}           // Armazena retorno da impressora
120 Local aRet     := {}           // Retorno funcao
121 Local aPDV     := {}           // Retorno evento get PDV
122
123 If STFUseFiscalPrinter()
124     /*
125     Get PDV Impressora
126     */
127     aPrinter := STFFireEvent( ProcName(0) , "STGetPDV" , aPDV )
128
129     If ValType(aPrinter) <> "A" .OR. Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aPDV) < 1
130
131         STFMessage("STCancelSale","STOP", STR0003) //Falha na obtenção do cupom
132         STFSHOWMessage("STCancelSale")
133         aRet := { .F. , "" }
134
135     Else
136         aRet := { .T. , PadR( AllTrim(aPDV[1] ) , TamSx3("L1_PDV")[1] , " " ) }
137     EndIf
138
139 Else
140     aRet := { .T. , STFGetStat("PDV") }
141 EndIf
142
143 Return aRet
144
145
146 //-----
147 /*{Protheus.doc} STBCanChoose
148 Verifica se pode escolher a venda a ser cancelada
149
150 @param
151 @return   lRet           Retorna se pode escolher a venda a ser cancelada
152 @obs
153 @sample
154 */
155 //-----
156 Function STBCanChoose()

```

```

157
158 Local lRet := .F.      // Retorna se pode escolher a venda a ser cancelada
159
160 If STFGetCfg("lCanChoose")
161     lRet := .T.
162 EndIf
163
164 Return lRet
165
166
167 //-----
168 /*{Protheus.doc} STBCSFiCanCancel
169 Verifica se é permitido cancelar vendas Finalizadas
170
171 @param
172 @return lRet      Retorna se é permitido cancelar vendas finalizadas
173 @obs
174 @sample
175 */
176 //-----
177 Function STBCSFiCanCancel()
178
179 Local lRet := .T.      // Retorna se é permitido cancelar vendas finalizadas
180
181 If STFGetCfg("lFiCanCancel")
182     lRet := .F.
183     STFMessagem("STCancelSale","STOP", STR0004) //Comprovantes de vendas anteriores nao podem ser cancelados
184 EndIf
185
186 Return lRet
187
188
189 //-----
190 /*{Protheus.doc} STBCSCanCancel
191 Verifica se pode Cancelar a Venda em andamento
192
193 @param
194 @return aRet Retorna array com informacoes do cancelamento
195 @obs
196 @sample
197 */
198 //-----
199 Function STBCSCanCancel(cGetDoc, lProfile, cGetSerie, cL1Num)
200
201 Local aRet      := {}
202 Local lCanCancel := .F.
203 Local lIsProgressSale := .F.
204 Local aProfile   := {}
205 Local cNumLastSale := ""
206 Local cLastDoc   := ""
207 Local cDocPrinter := ""
208 Local cPDV       := STFGetStation("PDV")

```

```

// Retorno funcao
// Retorna se pode cancelar a venda
// Define se a venda está em andamento
// Armazena retorno permissao do usuario
// Numeração da ultima venda
// Numeração do ultimo doc do banco
// Numeração do Doc atual da impressora
// Numero do PDV

```

```

209 Local cSuperior      := ""
210 Local cDocPed         := ""                                //Doc da serie nao fiscal
211 Local lTemCCCD        := .F.
212 Local lTemPD          := .F.
213 Local cDocRPS         := ""                                //Cancelamento RPS
214 Local cSerRPS         := ""                                //Cancelamento RPS
215 Local lPdvOn          := ExistFunc("STFPdvOn") .AND. STFPdvOn() // Verifica se é PDV-Online
216
217 Default cGetDoc        := ""
218 Default lProfile       := .T.
219 Default cGetSerie      := STFGetStation("SERIE")
220 Default cL1Num        := ""
221
222 If STB0penCash()
223
224   If lProfile
225     aProfile := STFProfile( 8 )
226     lCanCancel := aProfile[1]
227     cSuperior := aProfile[2]
228   Else
229     lCanCancel := .T.
230   EndIf
231
232   If lCanCancel
233
234     lIsProgressSale := STBCSIsProgressSale() // Venda em Andamento?
235
236     /*/
237     Venda em andamento
238     /*/
239     If lIsProgressSale
240
241       If STFGetCfg("lCanActSale")
242
243         If STDPBLength("SL2") == 0 // Permitido apenas o cancelamento da venda corrente com mais de um item lançado
244           lCanCancel := .F.
245           STFMessage("STCancelSale","STOP", STR0005) //Permitido apenas o cancelamento da venda corrente com mais de um item lançado
246         EndIf
247
248       EndIf
249
250     If lCanCancel
251
252       If STFUseFiscalPrinter()
253
254         cDocPrinter := STBCSNumPrinter()
255
256         If cDocPrinter == "ERROR"
257           lCanCancel := .F.
258         Else
259           lCanCancel := .T.
260           cLastDoc := cDocPrinter

```

```

261         EndIf
262
263     EndIf
264
265 EndIf
266
267 Else
268     /*/
269     Venda Finalizada
270     /*/
271
272     If !Empty(cGetDoc)
273         cLastDoc      := cGetDoc
274         cNumLastSale:= Iif((lPdvOn .And.!Empty(cL1Num)), cL1Num, STDCSNum(cGetDoc,cPDV,,cGetSerie))
275     Else
276         cNumLastSale  := STDCSLastSale()
277         cLastDoc      := STDCSDoc( cNumLastSale )
278     EndIf
279
280     If Empty(cLastDoc) .And. ExistFunc("STDGetDocP")
281         //Recupera o numedo do documento nao fiscal
282         cDocPed := STDGetDocP()
283         cDocRPS := STDGtDocRPS()
284         cSerRPS := STDGtSerRPS()
285     ElseIf ExistFunc("STDGtDocRPS") .And. (cLastDoc == STDGtDocRPS() .And. ExistFunc("STDGetDocP"))
286         //Recupera o numedo do documento nao fiscal após processamento do LJGRVBATCH pois alimenta o campo L1_DOC.
287         cDocPed := STDGetDocP()
288         cDocRPS := STDGtDocRPS()
289         cSerRPS := STDGtSerRPS()
290     EndIf
291
292     If STFUseFiscalPrinter()
293
294         cDocPrinter := STBCSNumPrinter()
295
296         If cDocPrinter == "ERROR"
297             lCanCancel := .F.
298         EndIf
299
300         If lCanCancel
301
302             If !STBCanChoose()
303
304                 If AllTrim(cLastDoc) == AllTrim(cDocPrinter)
305                     lCanCancel := .T.
306                     cLastDoc := cDocPrinter
307                 ElseIf FindFunction("STDCancTef") .And. STDCancTef(cNumLastSale, @lTemCCCD,@lTemPD)
308                     lCanCancel := .T.
309
310             Else
311                 lCanCancel := .F.
312                 STFMessage("STCancelSale","STOP", STR0006) //A numeração do último cupom do ECF não corresponde com a última venda. Não será feito o cancelamento

```

```

do cupom
313                                     EndIf
314
315                                     EndIf
316
317                                     EndIf
318
319                                     EndIf
320
321                                     EndIf
322
323     Else
324         STFS showMessage("STFProfile") // "Acesso Negado. O caixa XX não tem permissão para executar a seguinte operação: Cancelamento Cupom"
325     EndIf
326
327 Else
328
329     STFS message("STCancelSale","STOP", STR0007) // Caixa Fechado
330
331 EndIf
332
333 STFS showMessage("STCancelSale")
334
335 If lCanCancel
336
337     AADD( aRet , lCanCancel          )
338     AADD( aRet , lIsProgressSale      )
339     AADD( aRet , cSuperior            )
340     AADD( aRet , cLastDoc             )
341     AADD( aRet , cNumLastSale         )
342     AADD( aRet , cDocPed              )
343     AADD( aRet , cGetSerie            )
344     AADD( aRet , lTemCCCD             )
345     AADD( aRet , lTemPD               )
346     AADD( aRet , cDocRPS              )//10º
347     AADD( aRet , cSerRPS              )//11º
348
349 Else
350
351     AADD( aRet , .F.                  )
352     AADD( aRet , .F.                  )
353     AADD( aRet , ""                   )
354     AADD( aRet , ""                   )
355     AADD( aRet , ""                   )
356     AADD( aRet , ""                   )
357     AADD( aRet , ""                   )
358     AADD( aRet , lTemCCCD             )
359     AADD( aRet , lTemPD               )
360
361 EndIf
362
363 Return aRet

```

```

364
365
366 //-----
367 /*{Protheus.doc} STBCSCanLastCancel
368 Valida se pode cancelar Ultimo Cupom
369
370 @param  cNumDocPrint      Numero do documento da Impressora
371 @param  cNumSale          Numero da Venda a ser cancelada
372 @param  cNumDoc           Numero do Documento da venda a ser cancelada
373 @return  lRet             Retorna se pode cancelar
374 @obs
375 @sample
376 */
377 //-----
378 Function STBCSCanLastCancel( cNumDocPrint , cNumSale , cNumDoc )
379
380 Local lRet                := .T.           // Retorna se Pode Cancelar
381
382 Default cNumDocPrint      := ""
383 Default cNumSale          := ""
384 Default cNumDoc           := ""
385
386 ParamType 0 Var  cNumDocPrint  As Character  Default ""
387 ParamType 1 Var  cNumSale      As Character  Default ""
388 ParamType 2 Var  cNumDoc       As Character  Default ""
389
390 If STGetCfg("lVldCanLastSale") // Configuração para Bloquear se a ultima venda nao eh o Ultimo Cupom
391     // Num da Impressora Diferente do Num da última venda Bloqueia
392     If cNumDocPrint <> cNumDoc
393         lRet := .F.
394         STFMessage("STCancelSale","STOP", STR0008) //"0 último cupom do ECF não corresponde com a última venda. Não será feito o cancelamento do cupom.")
395     Endif
396 EndIf
397
398 Return  lRet
399
400
401 //-----
402 /*{Protheus.doc} STBCSUseDocChoose
403 Verifica se escolhe venda a ser Cancelada a Partir do L1_DOC
404
405 @param
406 @return  lRet             Retorna se escolhe por DOC
407 @obs
408 @sample
409 */
410 //-----
411 Static Function STBCSUseDocChoose()
412
413 Local lRet := .F.         // Retorna se escolhe por DOC
414
415 // Verifica se escolhe a venda a partir do Ultimo L1_DOC

```

```

416 If STFGetCfg("lseDocChoose")
417     lRet := .T.
418 EndIf
419
420 Return lRet
421
422
423 //-----
424 /*{Protheus.doc} STBCSFactura
425 Valida se Cancela a Venda verificando se pertence a uma Factura Global
426
427 @param   cNumSale      Numero da Venda a ser cancelada
428 @param   cNumDoc       Numero do documento
429 @param   cPDVPrint     Numero do PDV
430 @return  lRet          Retorna se Pode Cancelar a Venda
431 @obs
432 @sample
433 */
434 //-----
435 Function STBCSFactura( cNumSale , cNumDoc , cPDVPrint )
436
437 Local lRet          := .T.          // Retorna se Pode Cancelar a Venda
438 Local aSale         := {}          // Armazena dados da venda para analise de factura global
439 Local oRemoteCall   := NIL         // Retorna da chamada da funcao na Retaguarda
440
441 Default cNumSale    := ""
442 Default cNumDoc     := ""
443 Default cPDVPrint   := ""
444
445 ParamType 0 Var cNumSale As Character Default ""
446 ParamType 1 Var cNumDoc As Character Default ""
447 ParamType 2 Var cPDVPrint As Character Default ""
448
449 If STFGetCfg("lGlobFact") // Verifica Factura Global
450
451     // Busca informações da Venda para analise de Factura Global
452     aSale := STDCSFactura( cNumDoc , cPDVPrint )
453
454     oRemoteCall := FWCallFunctionality("FR271CPGGlobal", {aSale[1], aSale[2], aSale[3], aSale[4]})
455     If oRemoteCall:nStatusCode < 0
456         lRet := STFMessage("STCancelSale", "YESNO", STR0009) //"A conexao com o servidor de BackOffice por alguma razao se encontra interrompida. Esse cupom pode pertencer a
uma Nota Fiscal. Assim mesmo deseja cancelar o cupom de numero " + cNumDoc)
457     Else
458         aRet := oRemoteCall:uResult
459         lRet := aRet[2]
460         Do Case
461             Case AllTrim(aRet) == "GLOBAL"
462                 STFMessage("STCancelSale", "STOP", STR0010) //0 cupom nao podera ser cancelado porque pertence a uma Nota Fiscal Global
463             Case AllTrim(aRet) == "NFCUPOM"
464                 STFMessage("STCancelSale", "STOP", STR0011) //0 cupom nao podera ser cancelado porque foi gerada uma Nota Fiscal sobre cupom
465             Case AllTrim(aRet) == "QTDEDEV"
466                 STFMessage("STCancelSale", "STOP", STR0012) //Cupom não poderá ser cancelado, porque foi realizado devolução de um ou mais itens desta venda

```

```

467         Case AllTrim(aRet) == "BAIXADO"
468             STFMessage("STCancelSale","STOP", STR0013) //0 cupom nao podera ser cancelado porque ja foi gerada a baixa
469         EndCase
470     EndIf
471 EndIf
472
473 Return lRet
474
475
476 //-----
477 /*{Protheus.doc} STBCSCancCupPrint
478 Manda o cancelamento para a impressora
479
480 @param  cSupervisor      ID do Usuario Supervisor
481 @param  cNumDoc          Numero do documento
482 @param  cNumSale         Numero da Venda
483 @param  lNFCETSS        NFce Transmitida pelo TSS?
484 @param  lForceCancel     Força cancelamento da Venda?
485 @param  cTipoCanc        Tipo de Cancelamento
486 @param  lInutiliza       Inutiliza NFce?
487 @return  lRet            Retorna se cancelou na impressora
488 */
489 //-----
490 Function STBCSCancCupPrint( cSupervisor , cNumDoc, cNumSale, lNFCETSS,;
491                             lForceCancel, cTipoCanc, lInutiliza, cSerie, lTemCCCD,lTemPD)
492 Local lRet          := .T.           // Retorna se cancelou na impressora
493 Local aPrinter      := {}           // Armazena Retorno da impressora
494 Local cCOOCCD       := ""           // Armazena o Comprovante vinculado (CCD) a ser estornado
495 Local cNumCup       := ""           // Armazena o numero do Comprovante vinculado (CCD) menos 1
496 Local cCpfCnpj      := ""           // Cnpj/CPF do cliente da venda para o estorno do CCD
497 Local cMensagem     := ""           // Mensagem a ser impressa no estorno do CCD
498 Local nCCDs         := 1
499 local nX
500
501 Default cNumSale    := ""
502 Default cSupervisor := ""
503 Default cNumDoc     := ""
504 Default lNFCETSS    := .T.
505 Default lForceCancel:= .F.
506 Default cTipoCanc   := ""
507 Default lInutiliza  := .F.
508 Default cSerie      := STFGetStation("SERIE")
509 Default lTemCCCD    := .F.
510 Default lTemPD      := .F.
511
512 LjGrvLog( "L1_NUM: "+cNumSale+"L1_DOC: "+cNumDoc, "Cancelamento de venda" ) //Gera LOG
513
514 If STFUseFiscalPrinter()
515
516     If lTemCCCD .And. lTemPD
517         nCCDs := 2
518     EndIf

```

```

519
520 //Solicitar o cancelamento do CCD antes do cancelamento do cupom se tiver.
521 cC00CCD := STBCSNumPrinter()
522 cNumCup := PADR(StrZero(Val(AllTrim(cC00CCD))-nCCDs,Len(cC00CCD)),TamSX3("L1_DOC")[1])
523
524 If !Empty(cNumDoc) .And. AllTrim(cNumDoc) == AllTrim(cNumCup)
525
526     cCpfCnpj := AllTrim(STDCSLastSale("L1_CGCCLI"))
527     cMensagem := "Cancelamento de Comprovante de Credito e Debito"
528
529     For nX := 1 To nCCDs
530         aPrinter := STFFireEvent( ProcName(0) , "STCancelBound" , {cCpfCnpj, "" , "", cMensagem, StrZero(Val(cNumCup) + nX,Len(cC00CCD)) } )
531
532         If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0
533             lRet := .F.
534             STFMessage("STCancelSale","STOP", STR0022) //Erro com a Impressora Fiscal. Não foi efetuado o cancelamento do Comprovante de Credito e Debito (CCD).
535         EndIf
536     Next
537 EndIf
538
539 If lRet
540     aPrinter := STFFireEvent( ProcName(0) , "STCancelReceipt" , {cSupervisor} )
541     If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0
542         lRet := .F.
543         STFMessage("STCancelSale","STOP", STR0014) //Erro com a Impressora Fiscal. Não foi efetuado Cancelamento do Cupom
544     Else
545         lRet := .T.
546     EndIf
547 EndIf
548 Else
549     /*
550     Se nao utiliza impressora fiscal, retornar .T.
551     */
552     lRet := .T.
553     If !lNFCETSS .AND. ExistFunc("LjDNFCeCanc")
554         lRet := LjDNFCeCanc( @cNumDoc, @cNumSale, lNFCETSS, @cTipoCanc,@lInutiliza, cSerie )
555         If lForceCancel .AND. !lRet
556             lRet := .T.
557             LjGrvLog( "L1_NUM: "+cNumSale+"L1_DOC"+cNumDoc, "Ocorreu erro no cancelamento da NFce mas a venda será excluída" ) //Gera LOG
558         EndIf
559     EndIf
560 EndIf
561
562 Return lRet
563
564
565 //-----
566 /*{Protheus.doc} STBCSCancGP
567 Cancelar Vale Presente
568
569 @param   cNumSale      Numero da Venda
570 @return  lRet          Retorna se cancelou VPs

```

```

571 @obs
572 @sample
573 /*/
574 //-----
575 Function STBCSCancGP( cNumSale )
576
577 Local lRet           := .T.           // Retorna se cancelou VPs
578 Local uResult        := NIL           // Retorno da chamada da funcao na Retaguarda
579 Local aGiftVoucher   := {}           // Armazena VP a estornar
580
581 Default cNumSale     := ""
582
583 ParamType 0 Var  cNumSale  As Character  Default ""
584
585 aGiftVoucher := STDCSGiftVoucher( cNumSale ) // Busca VPs na Venda
586
587 If Len(aGiftVoucher) > 0
588     If STBRemoteExecute("LjVpAtiva" ,{aGiftVoucher}, NIL,.T.      ,@uResult)
589         lRet := uResult
590     EndIf
591 EndIf
592
593 EndIf
594
595 Return lRet
596
597
598 //-----
599 /*/{Protheus.doc} STBCSChkStatusPrint
600 Verifica STATUS Impressora
601
602 @param
603 @return  lRet           Retorna se STATUS da impressora está ok
604 @obs
605 @sample
606 /*/
607 //-----
608 Function STBCSChkStatusPrint()
609
610 Local lRet           := .T.           // Retorna se STATUS da impressora está ok
611 Local aRet           := {}           // Armazena retorno impressora
612 Local aStatusPrint   := { "9" , "" } // Armazena retorno Impressora
613
614 /*/
615     Se usa impressora Fiscal, verifica cupom aberto
616 /*/
617 If STFUseFiscalPrinter()
618
619     /*/
620     Verifica Status
621     /*/
622     aRet := STFFireEvent( ProcName(0) , ; // Nome do processo

```

```

623             "STPrinterStatus"                , ; // Nome do evento
624             aStatusPrint                      )
625
626     If Len(aRet) > 0 .AND. ValType(aRet[1]) == "N"
627
628         If aRet[1] == 0
629             lRet := .T.
630         Else
631             lRet := .F.
632             STFMessage("STCancelSale","STOP", STR0015) //"Erro com a Impressora Fiscal. Operação não efetuada
633         EndIf
634     EndIf
635
636 Else
637     lRet := .T.
638 EndIf
639
640 Return lRet
641
642 -----
643 /*{Protheus.doc} STBCSQuestion
644 Formula a perunta de confirmação de cancelamento
645
646 @param lIsProgressSale Define se a venda está em andamento
647 @param lAllNotFiscal   Todos os itens não fiscais
648 @param cNumber         Numeração a ser cancelada
649 @return cQuestion      Retorna Pergunta de confirmação
650 @obs
651 @sample
652 */
653 -----
654 Function STBCSQuestion( lIsProgressSale , lAllNotFiscal , cNumber )
655
656 Local cQuestion          := ""          // Mensagem
657
658 Default lIsProgressSale  := .F.
659 Default lAllNotFiscal    := .F.
660 Default cNumber          := ""
661
662 ParamType 0 Var lIsProgressSale As Logical Default .F.
663 ParamType 1 Var lAllNotFiscal   As Logical Default .F.
664 ParamType 2 Var cNumber         As Character Default ""
665
666 If lIsProgressSale
667     /*
668     Venda em andamento
669     */
670     If STFUseFiscalPrinter()
671         If STBExistItemFiscal()

```

```

675         cQuestion := STR0016 //Venda em andamento. Realiza o CANCELAMENTO deste Cupom Fiscal?
676     Else
677         cQuestion := STR0017 //Venda em andamento. Realiza o CANCELAMENTO deste Cupom Não Fiscal?
678     EndIf
679
680 Else
681
682     cQuestion := STR0018 //Venda em andamento. Realiza o CANCELAMENTO desta venda?
683
684 EndIf
685
686 Else
687
688     /*/
689     Venda Finalizada
690     /*/
691     If STBCanChoose()
692         cQuestion := STR0019 + cNumber + ")" //"Deseja cancelar o último cupom emitido (Numero: "
693     Else
694
695         If STFUseFiscalPrinter()
696
697             If lAllNotFiscal
698                 cQuestion := STR0020 //"Realiza o CANCELAMENTO do Cupom não Fiscal. "
699             Else
700                 cQuestion := STR0021 + cNumber + " ?" //"Realiza o CANCELAMENTO do Cupom Fiscal nº "
701             EndIf
702
703         EndIf
704
705     EndIf
706
707 EndIf
708
709 Return cQuestion
710
711
712 //-----
713 /*/{Protheus.doc} STBCSTotalSale
714 Formula a perunta de confirmação de cancelamento
715
716 @param  lIsProgressSale  Define se a venda está em andamento
717 @param  cNumberSale      Numeração a ser cancelada (L1_NUM)
718 @return  nTotalSale      Retorna o total da venda a ser cancelada
719 @obs
720 @sample
721 /*/
722 //-----
723 Function STBCSTotalSale( lIsProgressSale , cNumberSale )
724
725 Local nTotalSale          := 0                // Retorna o total da venda a ser cancelada
726 Local oTotal              := STFGetTot()      // Totalizador

```

```

727
728 Default lIsProgressSale      := .F.
729 Default cNumberSale         := ""
730
731 ParamType 0 Var lIsProgressSale As Logical Default .F.
732 ParamType 1 Var cNumberSale     As Character Default ""
733
734
735 If lIsProgressSale
736     nTotalSale := oTotal:GetValue("L1_VLRTOT")
737
738 Else
739     If !Empty(cNumberSale)
740         nTotalSale := STDCSTotalSale(cNumberSale)
741     EndIf
742 EndIf
743
744 Return nTotalSale
745
746
747
748
749
750
751
752 //-----
753 /*{Protheus.doc} STBCSNumberSale
754 Formula a perunta de confirmação de cancelamento
755
756 @param lIsProgressSale Define se a venda está em andamento
757 @param cDocSale         Numeração do Documento(L1_DOC)
758 @return cDocSale        Retorna o documento
759 @obs
760 @sample
761 */
762 //-----
763 Function STBCSNumberSale( lIsProgressSale , cDocSale )
764
765 Default lIsProgressSale      := .F.
766 Default cDocSale            := ""
767
768 ParamType 0 Var lIsProgressSale As Logical Default .F.
769 ParamType 1 Var cDocSale        As Character Default ""
770
771 If lIsProgressSale
772     cDocSale := STDGPBasket("SL1","L1_DOC")
773
774 Else
775     If Empty(cDocSale)
776

```

```

779      /*/
780      Busca o DOC a partir da ultima venda
781      /*/
782      cDocSale := STDCSDoc( STDCSLastSale() )
783
784      EndIf
785
786 EndIf
787
788 Return cDocSale
789
790 //-----
791 /*/{Protheus.doc} STBActionCancel
792 Ação botão de cancelar venda
793
794 @param      cGetCanc      Código da venda
795 @return
796 @obs
797 @sample
798 /*/
799 //-----
800 Function STBActionCancel(cGetCanc, cGetSerie)
801
802 Local lEmitNFce := STBGetNFCE()           //Valida se é NFC-e ou não
803 Local cDoc      := ""                    //Numero do documento
804 Local lUseSat   := STFGetCfg("lUseSAT", .F.) //Utiliza SAT
805 Local cEspeLast := STDCSLastSale("L1_ESPECIE") //Especie ultima venda
806 Local lRet      := .T.
807 Local cNFisCanc := ""                    //Doc de cancelamento
808 Local cRetsx5   := ""                    //Tamanho da serie no SX5
809 Local cMsg      := ""                    //Mensagem referente ao cancelamento do SAT
810 Local aInfCanSAT := {}
811 Local lRet      := .T.
812 Local aCancel   := STIGetCancel()         //Retornar o array com as informacoes da venda a ser cancelada
813 Local lDocNf    := .F.                    //Indica se a venda que esta sendo cancelada eh nao fiscal
814 Local aProfile  := {}                    //Recebe o retorno do STFProfile
815 Local lNfe      := .F.                    //Indica se é uma NFe
816 Local aAreaSL1 := SL1->(GetArea())
817 Local oRaas     := Nil
818 Local oFidelityC := Nil
819 Local lL1Tel    := SL1->(ColumnPos("L1_TEL")) > 0 //Variavel para verificar a existencia do campo L1_TEL
820 Local lDocRPS   := .F. //venda de serviço RPS
821 Local lExistReg  := .F.
822 Local lChgX5FIL  := ExistBlock("CHGX5FIL") //Ponto de Entrada CHGX5FIL
823 Local cFilSx5    := ""
824 Local lRetNfe    := .F.                    // Retorno do DBSeek para buscar L1_IMPNTF
825
826 Default cGetCanc := ""
827 Default cGetSerie := STFGetStation("SERIE")
828
829
830 If aCancel[4]<>cGetCanc

```

```

831     STISetCancel(STBCSCancel(cGetCanc,,cGetSerie)) //Atualiza a informacoes da venda a ser cancelada no PDV ONLINE
832     aCancel := STIGetCancel()
833 Endif
834
835 If ValType(aCancel) == "A" .AND. Len(aCancel) > 5 .AND. !Empty(aCancel[6])
836     //Se entrar no IF, significa que eh uma venda nao fiscal vale credito ou vale presente
837     lDocNf := .T.
838 Endif
839
840 lDocRPS := ValType(aCancel) == "A" .AND. Len(aCancel) >= 11 .AND. !Empty(aCancel[10]) .AND. !Empty(aCancel[11])
841
842 If Valtype(lUseSat) = "U"
843     lUseSat := .F.
844     If lEmitNFCe
845         lUseSat := LjUseSat()
846     Endif
847 Endif
848
849 //Ponto de Entrada ao checar se esse cupom pode cancelar ou não
850 If ExistBlock("STCANACT")
851     lRet := ExecBlock( "STCANACT",.F.,.F.,{PadL(AllTrim(cGetCanc),TamSX3("L1_DOC")[1], "0")})
852     If !lRet
853         STIGridCupRefresh() // Sincroniza a Cesta com a interface
854         STIRegItemInterface()
855         STFMessage(ProcName(), "STOP", "Nao será permitido o cancelamento dessa venda.") //"Nao permitido o cancelamento desta venda."
856         STFSHOWMessage(ProcName())
857     Endif
858 Endif
859
860 If !STBCSIsProgressSale()
861     DbSelectArea("SL1")
862     if lPdvOn .AND. Len(aCancel) >= 5 .AND. !Empty(aCancel[5]) .AND. !lDocRPS
863         SL1->(DbSetOrder(1))//L1_FILIAL+L1_NUM
864         lRetNfe := SL1->( DbSeek(xFilial("SL1") + aCancel[5]))
865     Else
866         SL1->(DbSetOrder(2))//L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
867         lRetNfe := SL1->(DbSeek(xFilial("SL1")+cGetSerie+cGetCanc))
868     Endif
869     If lRetNfe
870         lNfe := SL1->L1_IMPNT
871     Endif
872 Endif
873
874 If !STBCSIsProgressSale() .AND. !STBValCanc()
875     lRet:=.F.
876 Endif
877
878 //Somente poderá prosseguir com cancelamento de cupom finalizado caso não exista item com reserva
879 If lRet .AND. !STBCSIsProgressSale() .AND. (lEmitNFCe .OR. lUseSAT)
880     lRet:= STBVldItRes(SL1->L1_NUM, SL1->L1_SERIE, SL1->L1_NUMORIG)
881 Endif
882

```

```

883 //Valida se o motivo da Perda de Venda foi informado
884 If lRet .AND. (lEmitNFCe .OR. lUseSAT) .AND. SuperGetMV ("MV_LJMVPE",,.F.) .AND. ExistFunc("STIVldRsnInf")
885     lRet:= STIVldRsnInf()
886 EndIf
887
888 If lRet .And. lEmitNFCe .And. !STBCSIsProgressSale() .And. (!lUseSat .Or. lNfe)
889
890     If !lDocNf .AND. !lDocRPS
891         //Ajusta tamanho caso o usuario ano informe os zeros.
892         If lChgX5FIL
893             LjGrvLog(NIL, "Antes da execução do PE CHGX5FIL pela rotina LjFilSX5()")
894             cFilSx5 := LjFilSX5()
895             LjGrvLog(NIL, "Depois da execução do PE CHGX5FIL pela rotina LjFilSX5()",cFilSx5 )
896             dbSelectArea("SX5")
897             SX5->(dbSetOrder(1))//X5_FILIAL+X5_TABELA+X5_CHAVE
898             SX5->(dbSeek(cFilSx5 +'01'+ PADR(cGetSerie, TamSX3("X5_CHAVE")[1])))
899             If SX5->(Found())
900                 cRetsx5 := SX5->X5_DESCRI
901             Endif
902         Else
903             cRetsx5 := Tabela("01", AllTrim(cGetSerie) )
904         Endif
905
906         If !Empty(cRetsx5)
907             cGetCanc := PadL(AllTrim(cGetCanc),Len(AllTrim(cRetsx5)), "0")
908         Else
909             cGetCanc := PadL(AllTrim(cGetCanc),TamSX3("L1_DOC")[1], "0")
910         EndIf
911     EndIf
912
913     //verifica o SITUA do documento antes de prosseguir com a exclusao
914     If STBSitCanc(cGetCanc, cGetSerie)
915         If !lDocNf .AND. !lDocRPS
916             if lPdvOn .AND. Len(aCancel) >= 5 .AND. !Empty(aCancel[5])
917                 STICancelSale(cGetCanc, cGetSerie, aCancel[5])
918             Else
919                 STICancelSale(cGetCanc, cGetSerie)
920             Endif
921         Else
922             If ExistFunc('STWCancNF')
923                 aProfile := STFProfile( 8 )
924                 If ValType(aProfile) == 'A' .AND. aProfile[1]
925                     STWCancNF()
926                 Else
927                     LJGrvLog(Nil, "Venda nao foi cancelada porque nao foi informado o superior - L1_DOCPED: ", cGetCanc)
928                 EndIf
929             Else
930                 LJGrvLog(Nil, "Venda nao foi cancelada porque nao existe a funcao STWCancNF - L1_DOCPED: ", cGetCanc)
931             EndIf
932         EndIf
933     EndIf
934

```

```

935 ElseIf lRet .And. lEmitNFCe .And. STBCSIsProgressSale()
936     /*Cancela venda quando configurado com nfc-e e esta em andamento*/
937     cDoc := STDGPBasket("SL1","L1_DOC")
938     STWCancelSale(.T.,,cDoc, "L1_NUM",,)
939     STIGridCupRefresh()
940     STIRegItemInterface()
941
942 Else
943
944     If lRet .And. lUseSat .And. !STBCSIsProgressSale()
945
946         DbSelectArea("SL1")
947
948         If lDocRPS .And. cGetCanc == aCancel[10]
949             SL1->( DbSetOrder(1) ) //L1_FILIAL+L1_NUM
950             aCancel[5]:= Padr(aCancel[5], TamSx3("L1_NUM")[1])
951             lExistReg := SL1->(DbSeek(xFilial("SL1") + aCancel[5]))
952         Else
953             If lPdvOn .AND. Len(aCancel) >= 5 .AND. !Empty(aCancel[5])
954                 SL1->(DbSetOrder(1)) //L1_FILIAL+L1_NUM
955                 lExistReg:= SL1->( DbSeek(xFilial("SL1") + aCancel[5]))
956             Else
957                 SL1->( DbSetOrder(2) ) //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
958                 lExistReg:= SL1->( DbSeek(xFilial("SL1") + IIf(!Empty(cGetSerie),cGetSerie, STFGetStation("SERIE")) + cGetCanc + STFGetStation("PDV"))) )
959             Endif
960         Endif
961
962         If lExistReg
963
964             If lDocRPS .And. cGetCanc == aCancel[10]
965                 aInfCanSAT := STBCSCanCancel()
966             Else
967                 If lPdvOn .AND. Len(aCancel) > 5 .AND. !Empty(aCancel[5])
968                     aInfCanSAT := STBCSCanCancel(cGetCanc,,cGetSerie,aCancel[5])
969                 else
970                     aInfCanSAT := STBCSCanCancel(cGetCanc,,cGetSerie)
971                 Endif
972             Endif
973
974             STISetCancel( aInfCanSAT )
975
976             lRet := aInfCanSAT[1]
977
978         If lRet
979             // Gera cancelamento SAT
980             If !Empty(SL1->L1_SERSAT) .And. !Empty(SL1->L1_KEYNFCE)
981                 lRet := IIF(ExistFunc("LJSatxCanc"),LJSatxCanc(.F.,@cNFisCanc),.F.)
982             Endif
983             If !lRet
984                 cMsg := StrTran(STR0027,"SAT",cSiglaSat) //"SAT - Venda já cancelada ou excedeu o período de 30 minutos."
985             Endif
986

```

```

987         Else
988             cMsg := StrTran(STR0031,"SAT",cSiglaSat) //"SAT - Usuário sem permissão para cancelar venda"
989         EndIf
990
991     Else
992         lRet := .F.
993         cMsg := StrTran(STR0028,"SAT",cSiglaSat) //"SAT - Venda não encontrada."
994     EndIf
995
996     If !lRet
997         STFMessage(ProcName(), "STOP", cMsg) //"SAT - Venda já cancelada." ou "SAT - Venda não encontrada."
998         STFShowMessage(ProcName())
999     EndIf
1000
1001 EndIf
1002
1003 /*Cancela venda ecf */
1004 If lRet
1005     If lUseSat
1006         STICancel(cGetCanc,cNFisCanc)
1007     Else
1008         STICancel()
1009     EndIf
1010 EndIf
1011
1012 EndIf
1013
1014 // -- Cancelamento FidelityCore
1015 If lRet .And. ExistFunc("LjxRaasInt") .And. LjxRaasInt()
1016     oRaas := STBGetRaas()
1017     If Valtype(oRaas) == "0" .AND. SL1->L1_FIDCORE
1018         If oRaas:ServiceIsActive("TFC")
1019             oFidelityC := oRaas:GetFidelityCore()
1020             If !oFidelityC:CancelBonus(SL1->L1_FILIAL + SL1->L1_NUM, STFGetStat("CODIGO"), IIF(!!lTel,AllTrim(SL1->L1_TEL),""))[1]
1021                 MsgAlert( I18n(STR0035, {CRLF + CRLF}), STR0034) //"Não foi possível cancelar o bônus aplicado nesta venda.#1Entre em contato com seu parceiro de bônus."
1022             // "TOTVS Bonificação"
1023             EndIf
1024         Else
1025             MsgAlert(STR0036, STR0034) //"A venda possui uso de programa de bonificação, porém o programa de bonificação esta desativado no cadastro de estação." // "TOTVS
1026             Bonificação"
1027             Endif
1028         EndIf
1029     EndIf
1030 EndIf
1031 // Limpa variavel de verificação de regra de desconto por item
1032 STBLimpRegra(.F.)
1033
1034 If lRet .And. ExistFunc("STDRestCanL1") .And. !Empty(STDGPBasket("SL1","L1_CGCLI"))
1035     STDRestCanL1()
1036 EndIf
1037
1038 RestArea(aAreaSL1)
1039 Return Nil

```

```

1037
1038
1039 //-----
1040 /*{Protheus.doc} STBCSitDoc
1041 Verifica o L1_SITUA do documento antes de permitir o cancelamento
1042
1043 @param      cDoc      Número do documento fiscal
1044 @return      lRet      Indica se pode continuar com o cancelamento
1045 @obs
1046 @sample
1047 */
1048 //-----
1049 Static Function STBSitCanc(cDoc, cSerie)
1050
1051 Local lRet      := .T. //indica se o cancelamento pode continuar
1052 Local cPDV      := "" //numero do PDV ( LG_PDV )
1053 Local lDSitDoc  := FindFunction("STDSitDoc") //retorna o L1_SITUA do documento fiscal
1054 Local aSitua    := {}
1055 Local nSpedExc  := SuperGetMV("MV_SPEDEXC",, 72) // Indica a quantidade de horas q a NFe pode ser cancelada
1056 Local nNfceExc  := SuperGetMV("MV_NFCEEXC",, 0) // Indica a quantidade de horas q a NFCe pode ser cancelada
1057 Local aCancel   := STIGetCancel() //Retornar o array com as informacoes da venda a ser cancelada
1058 Local lDocNf    := .F. //Indica se a venda que esta sendo cancelada eh nao fiscal
1059 Local lDocRPS   := .F. //Cancelamento RPS
1060 Local cL1Num    := ""
1061 Local cLojaNf   := AllTrim(SuperGetMV("MV_LOJANF", .F. , "")) // Serie da NF-e para validar o prazo de cancelamento.
1062
1063 Default cSerie  := STFGetStation("SERIE")
1064
1065 cPDV := STFGetStation("PDV")
1066
1067 //Tratamento para manter o legado do parametro MV_SPEDEXC e em caso de NF-e pegar o prazo correto.
1068 If nNfceExc <= 0 .Or. cSerie == cLojaNf
1069     nNfceExc := nSpedExc
1070 EndIf
1071
1072 //se nao estiver compilado, nao validamos o documento, assim como era antes
1073 If lDSitDoc
1074
1075     If ValType(aCancel) == "A" .AND. Len(aCancel) > 5 .AND. !Empty(aCancel[6])
1076         //Se entrar no IF, significa que eh uma venda nao fiscal vale credito ou vale presente
1077         lDocNf := .T.
1078     ElseIf ValType(aCancel) == "A" .AND. Len(aCancel) >= 11 .AND. !Empty(aCancel[10]) .AND. !Empty(aCancel[11])
1079         lDocRPS := .T.
1080         cL1Num := aCancel[5]
1081     ElseIf ValType(aCancel) == "A" .AND. lPdvOn .AND. Len(aCancel) >= 11 .AND. !Empty(aCancel[5]) .AND. Empty(aCancel[10])
1082         cL1Num := aCancel[5]
1083     EndIf
1084
1085     //retorna os campos L1_SITUA e L1_STORC
1086     aSitua := STDSitDoc(cSerie, cDoc, cPDV, lDocNf, lDocRPS, cL1Num )
1087
1088     If aSitua[1] == "404"

```

```

1089     lRet := .F.
1090     STFMessage("STCancelSale", "STOP", STR0023) //"Documento fiscal não encontrado"
1091     STFShowMessage("STCancelSale")
1092
1093     //A - caso de nota cancelada automaticamente | C - cancelamento manual
1094     ElseIf aSituat[2] $ "A|C"
1095         lRet := .F.
1096         If !lDocNf .AND. !lDocRPS
1097             STFMessage("STCancelSale", "STOP", STR0024) //"Documento fiscal já enviado para cancelamento"
1098         Else
1099             STFMessage("STCancelSale", "STOP", STR0033) //"Documento não fiscal já enviado para cancelamento"
1100         EndIf
1101         STFShowMessage("STCancelSale")
1102     ElseIf !lDocNf .AND. !lDocRPS .AND. !STBCancTime(nNfceExc, cDoc, cSerie) //verifica o prazo do documento antes de prosseguir com a exclusao
1103         lRet := .F.
1104         STFMessage("STCancelSale", "STOP", STR0029 + " " + AllTrim(STR(nNfceExc)) + STR0030) //"Prazo para o cancelamento de venda é de"#"horas. Verifique o parâmetro
MV_NFCEEXC"
1105         STFShowMessage("STCancelSale")
1106     ElseIf aSituat[1] $ "00|TX"
1107         If !lDocNf .AND. !lDocRPS
1108             STFMessage("STCancelSale", "YESNO", STR0025 + cDoc + "/" + cSerie + " ?" ) //Deseja cancelar o documento fiscal: "
1109         Else
1110             STFMessage("STCancelSale", "YESNO", STR0032 + cDoc + "/" + cSerie + " ?" ) //"Deseja cancelar o documento não fiscal: "
1111         EndIf
1112         lRet := STFShowMessage("STCancelSale")
1113     Else
1114         STFMessage( "STCancelSale", "STOP", STR0026 ) //"Venda não finalizada/cancelada. Por favor, verifique a venda."
1115         STFShowMessage("STCancelSale")
1116     EndIf
1117 EndIf
1118
1119 EndIf
1120
1121 Return lRet
1122
1123
1124 /*/{Protheus.doc} STBChkInut
1125 Verifica se todos os requisitos foram atendidos
1126 @type      function
1127 @return     logico, se todos os requisitos foram atendidos
1128 @obs       Melhoria: Pode ser usada como funcao generica, recebendo o array
1129 */
1130 Function STBChkInut()
1131
1132 Local cAliasName      := ""
1133 Local nI              := 0
1134 Local nPos            := 0
1135 Local lRet            := .T.
1136 Local aAux            := {}
1137 Local aArea           := GetArea()
1138
1139 //Campos e funcoes necessarias para a Inutilizacao

```

```

1140 Add( aAux, {"FIELD", "LX_MODALDOC" , .F.} )
1141 Add( aAux, {"FUNC", "STWDELPAY" , .F.} )      //STWCANCELSALE.PRW
1142 Add( aAux, {"FUNC", "STDDELPAY" , .F.} )      //STDCANCELSALE.PRW
1143
1144 For nI := 1 to Len(aAux)
1145     If aAux[nI][1] == "FIELD"
1146         nPos := At( "_", aAux[nI][2] )
1147         //ZZ_FieldName
1148         If nPos == 3
1149             cAliasName := 'S' + SubStr(aAux[nI][2], 1, 2)
1150             //ZZZ_FieldName
1151         Else
1152             cAliasName := SubStr(aAux[nI][2], 1, 3)
1153         EndIf
1154         DbSelectArea( cAliasName )
1155         If &(cAliasName)->( ColumnPos(aAux[nI][2]) ) > 0
1156             aAux[nI][3] := .T.
1157         Else
1158             LJGrvLog( Nil, "CAMPO não encontrada no dicionario de dados", aAux[nI][2])
1159         EndIf
1160     ElseIf aAux[nI][1] == "FUNC"
1161         If ExistFunc(aAux[nI][2])
1162             aAux[nI][3] := .T.
1163         Else
1164             LJGrvLog( Nil, "FUNCAO não encontrada no repositorio", aAux[nI][2])
1165         EndIf
1166     EndIf
1167 Next
1168 RestArea(aArea)
1169 nPos := aScan( aAux, {|x| !x[3]} )
1170 If nPos > 0
1171     lRet := .F.
1172 EndIf
1173 Return lRet
1174
1175 //-----
1176 /*{Protheus.doc} STBCancTime
1177 Verifica se eh venda NFCE transmitida, pois neste caso deve respeitar o
1178 parametro MV_SPEDEXC que indica o numero de horas que pode ser excluida
1179
1180 @type      function

```

```

1192 @return      logico, se venda esta no prazo de cancelamento
1193 /*/
1194 //-----
1195 Static Function STBCancTime(nNfceExc, cDoc, cSerie)
1196 Local aArea      := GetArea()
1197 Local aAreaSL1    := SL1->(GetArea())
1198 Local lRet        := .T.
1199 Local dDtDigit    := dDataBase
1200 Local nHoras      := 0
1201 Local cEstacao    := Padr(STFGetStation("PDV"), TamSx3("L1_PDV")[1])
1202 Local lEndFis     := SuperGetMv("MV_SPEDEND",, .F.)           // Se estiver como F refere-se ao endereço de Cobrança se estiver T ao endereço de Entrega.
1203 Local cEstSM0     := IIf(!lEndFis, SM0->M0_ESTCOB, SM0->M0_ESTENT)
1204 Local cHoraUF     := FwTimeUF(cEstSM0)[2]
1205
1206 DEFAULT cSerie   := Padr(STFGetStation("SERIE"), TamSx3("L1_SERIE")[1])
1207
1208 If Empty(cHoraUF)
1209     cHoraUF := Time()
1210 EndIf
1211
1212 cDoc := Padr(cDoc, TamSx3("L1_DOC")[1])
1213
1214 SL1->(dbSetOrder(2)) //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
1215
1216 If SL1->(dbSeek(xFilial("SL1") + cSerie + cDoc + cEstacao)) .And. ExistFunc("LjSubtHora")
1217     //Verifica se a NFCE foi transmitida pela chave
1218     If !Empty(SL1->L1_KEYNFCE)
1219         dDtDigit := SL1->L1_EMITNF
1220         nHoras   := LjSubtHora(dDtDigit, SL1->L1_HORA, dDataBase, SubStr(cHoraUF, 1, 2) + ":" + SubStr(cHoraUF, 4, 2))
1221
1222         If nHoras > nNfceExc
1223             lRet := .F.
1224         EndIf
1225     EndIf
1226 EndIf
1227
1228 RestArea(aAreaSL1)
1229 RestArea(aArea)
1230
1231 Return lRet
1232
1233 //-----
1234 /*/{Protheus.doc} STBValCanc()
1235 Verifica SEI da Retaguarda se o titulo diferente de R$ foi baixada,
1236 caso sim, não permite o cancelamento do cupom.
1237
1238 @type      function
1239 @return    lógico, se o titulo não estiver em aberto retorna .T. para prosseguir o processo
1240 /*/
1241 //-----
1242 Static Function STBValCanc()
1243 Local lRet      := .T.

```

```

1244 Local uDados          := Nil
1245 Local aParam           := {}
1246 Local uResult          := Nil
1247 Local aFields           := {}
1248 Local aTables           := {}
1249 Local cWhere            := ""
1250 Local cMvLjBxAut        := SuperGetMv("MV_LJBXTIT ",",",",")
1251 Local nX                 := 0
1252
1253 If !Empty(cMvLjBxAut) .AND. Substr(SL1->L1_CONFVEN,6,1) <> "N" .AND. !(SL1->L1_CREDITO > 0)
1254     aFields := {"E1_TIPO",; //1
1255                 "E1_ORIGEM"} //2
1256
1257     aTables := {"SE1"}
1258
1259     cWhere := " E1_FILIAL = '" + SL1->L1_FILIAL + "' AND"
1260     cWhere += " E1_PREFIXO = '" + SL1->L1_SERIE + "' AND"
1261     cWhere += " E1_NUM = '" + SL1->L1_DOC + "' AND"
1262     cWhere += " E1_VALOR <> E1_SALDO AND"
1263     cWhere += " E1_TIPO <> 'R$' AND E1_TIPO <> 'CR' AND E1_TIPO <> 'NCC' AND E1_TIPO <> 'VP'AND"
1264     cWhere += " D_E_L_E_T_ = ' '"
1265
1266     aParam := { aFields;;
1267                 aTables;;
1268                 cWhere;;
1269                 ',,;
1270                 10;
1271             }
1272
1273     If STBRemoteExecute("STDQueryDB", aParam,,, @uResult)
1274         uDados := uResult
1275     EndIf
1276
1277     If ValType(uDados) == 'A'
1278         For nX := 1 To Len(uDados)
1279             If Alltrim(Upper(uDados[nX][1])) $ cMvLjBxAut .AND. AllTrim(Upper(uDados[nX][2])) $ "LOJA010/LOJA220/LOJA701/RPC/LOJA720/FATA701"
1280                 lRet := .F.
1281                 STFMessage("STBValCanc", "POPUP", STR0037 + ". "+ STR0038 + uDados[nX][1]+".")
1282                 STFShowMessage("STBValCanc")
1283                 Exit
1284             EndIf
1285         Next
1286     EndIf
1287
1288 Endif
1289
1290 Return lRet
1291
1292 /*/{Protheus.doc} STBDelItAS() Deleta Itens Auto Serviço
1293 Rotina criada para cancelamento de venda em andamento onde existe item auto serviço e importação de orçamento
1294 e com Totvs PDV ONLINE. Neste cenário, é preciso excluir os itens que não fazem parte do orçamento, pois caso
1295 contrário, os itens vendidos no auto serviço vai fazer parte do orçamento quando for importar de novo.

```

```

1296 @type Function
1297 /*/
1298 Function STBDeItAS()
1299 Local oModel := STDGPBModel()
1300 Local nX := 0
1301 Local nQtRegSL2 := STDPBLength("SL2")
1302 Local lCanceled := .F.
1303
1304 For nX:= 1 to nQtRegSL2
1305 oModel:GetModel("SL2DETAIL"):GoLine(nX)
1306 If Empty(oModel:GetValue("SL2DETAIL", "L2_NUMORIG")) .AND. !oModel:GetModel("SL2DETAIL"):IsDeleted()
1307 LjGrvLog(NIL, "exclusão de itens auto serviço", oModel:GetValue("SL2DETAIL", "L2_PRODUTO") )
1308 STWItemCancel (oModel:GetValue("SL2DETAIL", "L2_PRODUTO"), , ,oModel:GetValue("SL2DETAIL", "L2_ITEM"))
1309 lCanceled:= .T.
1310 EndIf
1311 Next nX
1312
1313 If lCanceled
1314 iif (Existfunc("STBISL4Refresh"),STBISL4Refresh(), nil)
1315 STDSaveSale()
1316 Endif
1317
1318 Return
1319
1320
1321 /*/{Protheus.doc} STBVldItRes
1322 Verifica se existe item com reserva.
1323 Caso exista, não pode prosseguir com cancelamento
1324 uso somente para cancelamento e venda finalizada
1325 @type Function
1326 @param cSL1NUM , caracter , L1_NUM
1327 @param cL1SERIE , caracter , L1_SERIE
1328 @param cSL1NUMORI , caracter , L1_NUMORIG
1329 @return lRet , lógico , se .T. pode prosseguir com cancelamento
1330 /*/
1331 Static Function STBVldItRes(cSL1NUM, cL1SERIE, cSL1NUMORI )
1332 Local lRet := .T.
1333 If STDCSChkRes(cSL1NUM, cSL1NUMORI )
1334 lRet := .F.
1335 MsgAlert(STR0039)//Existe Pedido e Reserva de itens vinculados à essa Venda. Favor realizar a Troca/Devolução pela Retaguarda
1336 STFMessage("STBVldItRes","STOP", STR0039)
1337 LjGrvLog(cSL1NUM,STR0039)
1338 Endif
1339 Return lRet

```

Componentes/CancelSale/STDCancelSale.prw

1580 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cDocPed := "" //Armazena o L1_DOCPED da venda
5 Static cDocRPS := "" //Cancelamento RPS
6 Static cSerRPS := "" //Cancelamento RPS
7 Static lPdvOn := ExistFunc("STFPdvOn") .AND. STFPdvOn() // Verifica se é PDV-Online
8
9 //-----
10 /*{Protheus.doc} STDCSChkRes
11 Verifica se existe reserva para todos itens da Venda
12
13 @param cNumSale      Numero da Venda
14 @param cNumOrig      Numero da Original da SL1
15 @return lRet         Retorna se a venda está em andamento ou não
16 */
17 //-----
18 Function STDCSChkRes( cNumSale, sNumOrig )
19 Local lRet := .F. // Retorna se existe item com reserva
20 Local aArea := GetArea() // Guarda Area
21 Local aAreaSL2 := {}
22 Local cFilSL2 := ""
23
24 Default cNumSale:= ""
25 Default sNumOrig:= ""
26
27 LjGrvLog(cNumSale,"Validação se venda possui reserva para permitir cancelamento")
28
29 DbSelectArea("SL2")
30 aAreaSL2 := SL2->(GetArea()) // Guarda area do SL2
31 SL2->( DbSetOrder(1) )//L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
32 cFilSL2 := xFilial("SL2")
33 LjGrvLog(cNumSale," Itens a serem procurados - Dados: L2_FILIAL [" + cFilSL2 + "] || L2_NUM [" + cNumSale + "]")
34
35 If SL2->( DbSeek( cFilSL2 + cNumSale ) )
36   If lPdvOn .AND. !Empty(sNumOrig)
37     lRet := STDValIRes(sNumOrig)
38   Else
39     LjGrvLog(cNumSale,"Achou os itens da venda",cFilSL2 + cNumSale)
40     While SL2->( !EOF() ) .AND. (SL2->L2_FILIAL + SL2->L2_NUM == cFilSL2 + cNumSale)
41       LjGrvLog(cNumSale,"SL2->L2_ENTREGA",SL2->L2_ENTREGA)
42       LjGrvLog(cNumSale,"SL2->L2_RESERVA",SL2->L2_RESERVA)
43       If !Empty(AllTrim(SL2->L2_ENTREGA)) .AND. !(SL2->L2_ENTREGA == "2") .AND. !Empty(AllTrim(SL2->L2_RESERVA))
44         lRet := .T.
45       Exit
46     EndIf
47     SL2->( DbSkip() )
48   EndDo
49 Endif
50
51 If lRet
52   LjGrvLog(cNumSale,"Existe item com reserva, não permite cancelar a venda pelo Totvs Pdv - lRet = .T.",lRet)

```

```

53     Endif
54
55 Else
56     LjGrvLog(cNumSale,"Numero de orçamento não encontrado - L1_NUM [" + cNumSale + "]")
57     lRet := .F.
58 EndIf
59
60 RestArea(aAreaSL2)
61 RestArea(aArea)
62
63 Iif(lRet,LjGrvLog(cNumSale,"Existe(m) produto(s) de reserva - venda não pode ser cancelada"),NIL)
64
65 Return lRet
66
67 //-----
68 /*{Protheus.doc} STDCSLastSale
69 Busca Numero do Ultimo Orçamento
70
71 @param   cRetField - Nome do campo no dicionario de dados da tabela SL1
72 @return  cNumSale - Retorna o Numero ou outro campo da tabela SL1 da Ultima Venda
73 @obs
74 @sample
75 */
76 //-----
77 Function STDCSLastSale(cRetField, cGetSerie)
78
79 Local cNumSale := ""                // Retorna o Numero da Ultima Venda ou o campo do parametro
80 Local nRecnoL1 := 0                // Armazena Recno do ultimo registro físico da SL1
81 Local aArea := {}                 // Armazena Area corrente
82 Local lReImp := IsInCallStack("LjNFCeReImp") // É Reimpressão de Cupom?
83
84 Default cRetField := "L1_NUM"
85 Default cGetSerie := STFFGetStation("SERIE")
86
87 aArea := SL1->(GetArea())
88
89 // Posiciona o SL1 na ultima venda
90 DbSelectArea("SL1")
91
92 If lPdvOn .And. !lReImp
93     cNumSale := STDLastPdvOn(cRetField, @cGetSerie)
94 Else
95     SL1->(DbSetOrder(1))
96     nRecnoL1 := SL1->(LASTREC())
97     SL1->(DbGoto(nRecnoL1))
98     While !BOF() .And. SL1->( Deleted() )
99         SL1->(DbSkip(-1))
100     EndDo
101     cNumSale := &("SL1->("+cRetField+")")
102
103     If !Empty(SL1->L1_SERIE)
104         cGetSerie :=SL1->L1_SERIE

```

```

105     Endif
106
107     If !Empty(STDGtDocRPS()) .And. FWSIXUtil():ExistIndex( "SL1" , "0" )
108         //Este indice está cadastrado somente para o Brasil.
109         SL1->(DbSetOrder(24)) // L1_FILIAL+L1_DOCRPS+L1_SERRPS+L1_PDV
110         If DbSeek(xFilial("SL1")+STDGtDocRPS()+STDGtSERRPS())
111             cNumSale := &("SL1->("+cRetField+"")
112         EndIf
113     EndIf
114 EndIf
115
116
117 LjGrvLog( "STDCSLastSale", "Ultimo registro da SL1 posicionado:"+cRetField,cNumSale)
118
119 RestArea(aArea)
120
121 Return cNumSale
122
123
124 //-----
125 /*/{Protheus.doc} STDCSDoc
126 Busca o DOC da venda
127
128 @param   cNumSale      Numero da Venda
129 @return  cDoc           Retorna o L1_DOC da Venda
130 @obs
131 @sample
132 */
133 //-----
134 //
135 Function STDCSDoc( cNumSale )
136
137 Local cDoc      := ""           // Retorna o L1_DOC da Venda
138 Local aArea     := {}          // Armazena Area corrente
139 Local aAreaSL1  := {}          // Armazena Area SL1
140
141 Default cNumSale:= ""
142
143 aArea      := GetArea()
144 aAreaSL1   := SL1->(GetArea())
145
146 DbSelectArea("SL1")
147 DbSetOrder(1)//L1_FILIAL+L1_NUM
148 If !PdvOn == .T.
149     SL1->(DbSeek(xFilial("SL1")+cNumSale)) // Posiciona na SL1 retornada da query
150 EndIf
151
152 If (xFilial("SL1") == SL1->L1_FILIAL) .AND. (SL1->L1_NUM == cNumSale) // Ja posicionado
153
154     cDocPed := SL1->L1_DOCPED
155     cDocRPS := SL1->L1_DOCRPS
156     cSerRPS := SL1->L1_SERRPS

```

```

157
158   If !Empty(SL1->L1_DOC)
159       cDoc := SL1->L1_DOC
160   Endif
161
162   STDSPBasket("SL1", "L1_CLIENTE" ,SL1->L1_CLIENTE)
163   STDSPBasket("SL1", "L1_LOJA" ,SL1->L1_LOJA )
164   STDSPBasket("SL1", "L1_CGCCLI" ,SL1->L1_CGCCLI )
165
166 Else
167
168   If DbSeek( xFilial("SL1") + cNumSale )
169
170       cDocPed := SL1->L1_DOCPED
171
172       If !Empty(SL1->L1_DOC)
173           cDoc := SL1->L1_DOC
174       Endif
175       //Alimenta o codigo e o nome do cliente na cesta
176       STDSPBasket("SL1", "L1_CLIENTE" ,SL1->L1_CLIENTE)
177       STDSPBasket("SL1", "L1_LOJA" ,SL1->L1_LOJA)
178       STDSPBasket("SL1", "L1_CGCCLI" ,SL1->L1_CGCCLI)
179   EndIf
180 EndIf
181
182 RestArea(aAreaSL1)
183 RestArea(aArea)
184
185 Return cDoc
186
187
188 //-----
189 /*{Protheus.doc} STDCSNum
190 Busca L1_NUM a partir do DOC/PDV
191
192 @param  cNumDoc      Numero do documento (L1_DOC)
193 @param  cPDV         Numero do PDV
194 @return  cNum        Retorna o numero da venda L1_NUM
195 @obs
196 @sample
197 */
198 //-----
199 Function STDCSNum( cNumDoc , cPDV, lDocNf, cSerie, cL1Num )
200
201 Local cNum      := ""           // Retorna L1_NUM
202 Local aArea     := GetArea()    // Armazena Area corrente
203 Local aAreaSL1  := SL1->(GetArea()) // Armazena Area SL1
204 Local cSerNf    := "" //Serie nao fiscal
205 Local cDocNf    := "" //Numero do documento nao fiscal
206
207 Default cNumDoc := ""
208 Default cPDV    := ""

```

```

209 Default lDocNf      := .F.
210 Default cSerie       := STFGetStation("SERIE") // Serie da estacao, acrescentado para cancelamento NF-e
211 Default cL1Num       := "" // L1_NUM, utilizar somente quando é cancelamento de venda TOTVS ONLINE
212
213 cSerie := PadR( cSerie, SL1->(TamSx3("L1_SERIE"))[1] )
214 cNumDoc := PadR( cNumDoc, SL1->(TamSx3("L1_DOC"))[1] )
215 cPDV := PadR( cPDV, Len(CriaVar("L1_PDV",.F.)), ' ' )
216
217 DbSelectArea("SL1")
218
219 If lPdvOn .AND. !Empty(cL1Num)
220     cNum = cL1Num
221 ElseIf !lDocNf
222     SL1->( DbSetOrder(2) ) //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
223     If SL1->( DbSeek(xFilial("SL1") + cSerie + cNumDoc + cPDV) )
224         cNum := SL1->L1_NUM
225     Else
226         SL1->(DbSetOrder(11)) //L1_FILIAL+L1_SERPED+L1_DOCPED
227         If SL1->( DbSeek(xFilial("SL1")+cSerie+cNumDoc)) .AND. AllTrim(SL1->L1_PDV) == Alltrim(cPDV)
228             cNum := SL1->L1_NUM
229             ElseIf FWSIXUtil():ExistIndex( "SL1" , "0" )
230                 //Este indice está cadastrado somente para o Brasil.
231                 SL1->(DbSetOrder(24)) //L1_FILIAL+L1_DOCRPS+L1_SERRPS+L1_PDV
232                 If SL1->(DbSeek(xFilial("SL1")+cNumDoc+cSerie+AllTrim(cPDV)))
233                     cNum := SL1->L1_NUM
234                     cDocRPS := SL1->L1_DOCRPS
235                     cSerRPS := SL1->L1_SERRPS
236                 EndIf
237             EndIf
238         EndIf
239 Else
240     If !Empty(cDocRPS) .AND. !Empty(cSerRPS) .AND. SL1->L1_DOCRPS == cDocRPS .AND. SL1->L1_SERRPS == cSerRPS //Cancelamento RPS
241         cNum := SL1->L1_NUM
242     Else
243         cSerNf := PadR( STFGetStation("SERNFIS"), SL1->(TamSx3("L1_SERPED"))[1] )
244         cDocNf := PadR( cNumDoc, SL1->(TamSx3("L1_DOCPED"))[1] )
245
246         SL1->(DbSetOrder(11)) //L1_FILIAL+L1_SERPED+L1_DOCPED
247         If SL1->( DbSeek(xFilial("SL1")+cSerNf+cDocNf)) .AND. AllTrim(SL1->L1_PDV) == Alltrim(cPDV)
248             cNum := SL1->L1_NUM
249         EndIf
250     EndIf
251 EndIf
252
253
254 RestArea(aAreaSL1)
255 RestArea(aArea)
256
257 Return cNum
258
259
260 //-----

```

```

261 /*/{Protheus.doc} STDCSFactura
262 Busca Dados da venda para análise se Factura Global
263
264 @param   cNumDoc           Numero do documento (L1_DOC)
265 @param   cPDV              Numero do PDV
266 @return  aRet[1]           L1_DOC
267 @return  aRet[2]           L1_SERIE
268 @return  aRet[3]           L1_CLIENTE
269 @return  aRet[4]           L1_LOJA
270 @obs
271 @sample
272 */
273 //-----
274 Function STDCSFactura( cNumDoc , cPDV )
275
276 Local aRet      := {}           // Array de retorno
277 Local aArea     := GetArea()    // Armazena Area corrente
278 Local aAreaSL1  := SL1->(GetArea()) // Armazena Area SL1
279
280 Default cNumDoc := ""
281 Default cPDV    := ""
282
283 SL1->(DbSetOrder(8))//L1_FILIAL+L1_PDV+L1_DOC
284 If SL1->(DbSeek(xFilial("SL1") + cPDV + cNumDoc))
285     aRet := { SL1->L1_DOC, SL1->L1_SERIE, SL1->L1_CLIENTE, SL1->L1_LOJA }
286 EndIf
287
288 RestArea(aAreaSL1)
289 RestArea(aArea)
290
291 Return aRet
292
293
294 //-----
295 /*/{Protheus.doc} STDCSRefsh
296 Desposiciona e reposiciona no mesmo registro. Utilizado em caso de registro alterado por outra thread
297
298 @param   cTab              Tabela do registro a ser atualizado
299 @return  Nil
300 @obs
301 @sample
302 */
303 //-----
304 Function STDCSRefsh(cTab)
305
306 Local nRecno := 0           // Numero do recno
307
308 Default cTab := ""
309
310 If !Empty(cTab)
311     nRecno := (cTab)->(RecNo())
312     (cTab)->(DbSkip())

```

```

313      (cTab)->(DbGoto(nRecno))
314 EndIf
315
316 Return Nil
317
318
319 //-----
320 /*{Protheus.doc} STDCSDeleteSale
321 Realiza a deleção do SL1, SL2 e SL4 e SE5
322
323 @param  cNumSale      Numero da Venda
324 @param  lDelChange    Define se deleta SE5 (Troco para localizações)
325 @return  lRet         Retorna se deletou venda
326 @obs
327 @sample
328 */
329 //-----
330 Function STDCSDeleteSale( cNumSale , lDelChange, cNumOrig )
331
332 Local lRet          := .T.           // Retorna se deletou venda
333 Local aArea         := GetArea()     // Armazena Area corrente
334 Local aAreaSL1      := SL1->(GetArea()) // Armazena Area SL1
335 Local lCancVenda    := .T.
336 Local lUseSat       := STFGetCfg("lUseSAT", .F.)// Utiliza SAT
337 Local lMobile       := STFGetCfg("lMobile", .F.)// Utiliza SAT
338 Local lCancPdvOn    := .F.
339
340 Default cNumSale    := ""
341 Default lDelChange  := .F.
342 Default cNumOrig    := ""
343
344 If ValType(lMobile) <> "L"
345     lMobile := .F.
346 EndIf
347
348 /*
349 Indica que a venda deverá ser cancelada, porém no caso de uma NFC-e rejeitada,
350 ela deve ser subir para a retaguarda como uma venda normal, por isso não a cancelamos.
351 O flag (_STORC = "A") indica que ela deverá ser inutilizada pelo LjGrvBatch,
352 por esse motivo ela não gera SLI e os campos ficarão dessa forma: L1_SITUA = 00 e L1_STORC = "A"
353 */
354 If !lMobile
355     If lPdvOn .And. (!Empty(cNumOrig) .Or. (STBIsImpOrc() .And. STBCSIsProgressSale()))
356         lCancVenda := .F.
357         lCancPdvOn := .T.
358         If ExistFunc("STBDeItAS")
359             STBDeItAS()
360         Endif
361     Else
362         lCancVenda := !( STBGetNFCE() .AND. STBCSIsProgressSale() .AND. !lUseSAT .AND. STDGPBasket("SL1","L1_SITUA") == '65' )
363     EndIf
364 ElseIf STBGetNFCE() .AND. !STBCSIsProgressSale() .And. !lUseSat

```

```

365      //nao cancela para a venda subir e cancelar no gravabatch (piloto PDV Mobile)
366      lCancVenda := .F.
367 EndIf
368
369 LjGrvLog( "STDCSDeleteSale", "lCancVenda ",lCancVenda)
370
371 DbSelectArea("SL1")
372 DbSetOrder(1) //L1_FILIAL+L1_NUM
373 If DbSeek( xFilial("SL1") + cNumSale )
374     If !RecLock( "SL1", .F. )
375         lRet := .F.
376     EndIf
377 Else
378     lRet := .F.
379 EndIf
380
381 LjGrvLog( "STDCSDeleteSale", " Registro da SL1 lRet = ",lRet)
382
383 If lRet
384     DbSelectArea("SL4")
385     DbSetOrder(1)//L4_FILIAL+L4_NUM+L4_ORIGEM
386     DbSeek(xFilial("SL4")+cNumSale)
387     While ( ( SL4->L4_FILIAL + SL4->L4_NUM ) == ( xFilial("SL1") + cNumSale ) ) .AND. !EOF()
388         If lCancVenda
389             RecLock("SL4",.F.)
390             SL4->( DbDelete() )
391             MsUnlock()
392         EndIf
393         SL4->( DbSkip() )
394     EndDo
395
396     DbSelectArea("SL2")
397     DbSetOrder(1)//L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
398     DbSeek(xFilial("SL2") + cNumSale)
399     While ( (SL2->L2_FILIAL + SL2->L2_NUM) == (xFilial("SL1") + cNumSale) ) .AND. !EOF()
400         If lCancVenda
401             RecLock("SL2", .F.)
402             SL2->( DbDelete() )
403             MsUnlock()
404             LjGrvLog( "STDCSDeleteSale", "SL2 deletado",cNumSale)
405         EndIf
406         SL2->( DbSkip() )
407     EndDo
408
409     If lDelChange
410         DbSelectArea("SE5")
411         DbSetOrder(2)//E5_FILIAL+E5_TIPODOC+E5_PREFIXO+E5_NUMERO+E5_PARCELA+E5_TIPO+Dtos(E5_DATA)+E5_CLIFOR+E5_LOJA+E5_SEQ
412         If DbSeek(xFilial("SE5") + "VL" + SL1->L1_SERIE + SL1->L1_DOC)
413             While !EOF() .AND. ( xFilial("SE5") + "VL" + SE5->E5_PREFIXO + SE5->E5_NUMERO ) == ( xFilial("SE5") + "VL" + SL1->L1_SERIE + SL1->L1_DOC )
414                 If ( Empty(SE5->E5_TIPO) .AND. ;
415                     SE5->E5_MOEDA == "TC" .AND. ;
416                     SE5->E5_RECPEG == "P" .AND. ;

```

```

417             (SE5->E5_CLIFOR == SL1->L1_CLIENTE) .AND. ;
418             (SE5->E5_LOJA == SL1->L1_LOJA) )
419
420             RecLock("SE5",.F.)
421             DbDelete()
422             MsUnLock()
423         EndIf
424         DbSkip()
425     EndDo
426 EndIf
427 EndIf
428
429 DbSelectArea("SL1")
430 RecLock("SL1",.F.)
431 If lCancPdvOn .AND. !lCancVenda
432     Replace SL1->L1_SITUA with ""
433     Replace SL1->L1_STORC with ""
434     Replace SL1->L1_DOC with ""
435     Replace SL1->L1_SERIE with ""
436     Replace SL1->L1_PDV with ""
437     Replace SL1->L1_EMISNF with STOD("")
438     Replace SL1->L1_OPERADO with ""
439     Replace SL1->L1_NUMORIG with ""
440     Replace SL1->L1_NUMMOV with ""
441     Replace SL1->L1_TPORC with ""
442 Else
443     If lCancVenda
444         Replace SL1->L1_SITUA with "07"
445         SL1->( DbDelete() )
446     Else
447         Replace SL1->L1_SITUA with "00"
448         Replace SL1->L1_STORC with "A"
449     EndIf
450 EndIf
451 SL1->( MsUnLock() )
452
453 EndIf
454
455 RestArea(aAreaSL1)
456 RestArea(aArea)
457
458 Return lRet
459
460
461 //-----
462 /*{Protheus.doc} STDCSGiftVoucher
463 Busca Vale Presentes a serem estornados
464
465 @param cNumSale          Numero da Venda
466 @return aRet             Retorna info do vale presente utilizado
467
468 @obs

```

```

469 @sample
470 /*/
471 //-----
472 Function STDCSGiftVoucher( cNumsale )
473
474 Local aRet          := {}           // Retorna VPs a serem estornados
475 Local aArea         := GetArea()    // Armazena Area corrente
476
477 Default cNumSale    := ""
478
479 ParamType 0 Var  cNumSale  As Character  Default ""
480
481 // Busca se foi utilizado VP na venda como forma de pagamento
482 DbSelectArea("SL4")
483 DbSetOrder(1)//L4_FILIAL+L4_NUM+L4_ORIGEM
484 If DbSeek(xFilial("SL4") + cNumsale)
485     While (!SL4->(EOF())) .AND. (SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + cNumsale)
486         If AllTrim(SL4->L4_FORMA) == "VP"
487             AADD(aRet,{AllTrim(SL4->L4_CODVP),"2"})
488         EndIf
489         DbSkip()
490     EndDo
491 EndIf
492
493 // Busca se foi vendido algum VP na venda
494 DbSelectArea("SL2")
495 DbSetOrder(1)//L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
496 If DbSeek(xFilial("SL2") + cNumsale)
497     While !SL2->(Eof()) .AND. (SL2->L2_FILIAL + SL2->L2_NUM == xFilial("SL2") + cNumsale)
498         If SL2->(FieldPos("L2_VALEPRE")) > 0 .AND. !Empty(SL2->L2_VALEPRE)
499             AADD(aRet,{AllTrim(SL2->L2_VALEPRE),"1"})
500         EndIf
501         DbSkip()
502     EndDo
503 EndIf
504
505 RestArea(aArea)
506
507 Return aRet
508
509
510 //-----
511 /*/{Protheus.doc} STDCSTotalSale
512 Busca o total da venda a ser cancelada
513
514 @param  cNumSale      Numero da Venda(L1_NUM)
515 @return nTotalSale    Retorna o valor total da venda
516
517 @obs
518 @sample
519 /*/
520 //-----

```

```

521 Function STDCSTotalSale(cNumSale)
522
523 Local nTotalSale      := 0                // Retorna o valor total da venda
524 Local aArea           := GetArea()        // Armazena Area corrente
525 Local aAreaSL1        := SL1->(GetArea())  // Armazena Area SL1
526
527 Default cNumSale      := ""
528
529 ParamType 0 Var  cNumSale      As Character      Default ""
530
531 DbSelectArea("SL1")
532 DbSetOrder(1) //L1_FILIAL+L1_NUM
533 If SL1->(DbSeek(xFilial("SL1") + cNumSale))
534     nTotalSale := SL1->L1_VLRTOT
535 EndIf
536
537 RestArea(aAreaSL1)
538 RestArea(aArea)
539
540 Return nTotalSale
541
542
543 //-----
544 /*{Protheus.doc} STDCSRequestCancel
545 Busca informações para solicitação de cancelamento
546
547 @param  cNumSale      Numero da Venda(L1_NUM)
548 @param  lCancVenc     Cancela vendas vencidas
549 @return aRequest      Retorna se realizou a solicitação de cancelamento
550 @obs
551 @sample
552 */
553 //-----
554 Function STDCSRequestCancel( cNumSale , lCancVenc, cNFisCanc )
555
556 Local lRet           := .T.                // Retorna se realizou a solicitação de cancelamento
557 Local aArea          := GetArea()          // Armazena area corrente
558 Local aAreaSL1       := SL1->(GetArea())    // Armazena Area SL
559 Local aSL1           := {}                // Array Gravação SL1
560 Local cLiMsg         := ""                // Msg SLI para cancelamento
561 Local cStation       := ""                // SLI_Estacao
562 Local cPAFMD5        := ""
563 Local lFirst         := .F.                // Integra com First?
564 Local cIntegration    := "DEFAULT"         // Tipo de Integração
565 Local lConfStatus    := .F.                // Valida se populou SL1 atualizando STORC
566 Local cAliasAtu      := ""
567 Local lUseSat         := STFGetCfg("lUseSAT", .F.)//Utiliza SAT
568 Local lCentPDV       := LjGetCPDV()[1]     //Server é Central de Pdv -
569
570 Default cNumSale      := ""
571 Default lCancVenc     := .F.
572 Default cNFisCanc     := ""

```

```

573
574 If Findfunction("STFCfgIntegration")
575     cIntegration := STFCfgIntegration()
576 ElseIf SuperGetMv("MV_LJ RMS",,.F.)
577     cIntegration := "RMS"
578 EndIf
579
580 lFirst := cIntegration == "FIRST"
581
582 //Por seguranca, valida se SL1 está posicionado na venda que sera cancelada
583 If xFilial("SL1") + cNumSale <> SL1->L1_FILIAL + SL1->L1_NUM
584     LjGrvLog( cNumSale,"Venda nao estava posicionada, sera realizado Seek para posicionar. L1_NUM atual:",SL1->L1_NUM )
585
586     //armazena o Alias corrente
587     cAliasAtu := Alias()
588
589     DbSelectArea("SL1")
590     SL1->( DbSetOrder(1) )
591     lRet := SL1->( DbSeek(xFilial("SL1") + cNumSale) )
592     aAreaSL1 := SL1->( GetArea() )
593
594     //se o Alias for SL1, entao pegamos a Area novamente, pois a restauracao posicionaria ele no registro errado
595     If cAliasAtu == "SL1"
596         aArea := aClone(aAreaSL1)
597     EndIf
598
599     LjGrvLog( cNumSale,"Apos Seek para posicionar. L1_NUM atual:",SL1->L1_NUM )
600 EndIf
601
602 If lRet
603
604     LjGrvLog( cNumSale,"Situacao atual da venda, no momento do cancelamento :",SL1->L1_SITUA ) //Gera LOG
605
606     If !lFirst
607         If !Empty(cNFisCanc) .AND. !(lPdvOn .AND. lCentPDV .AND. lUseSat) //Acrescentado a serie para que o cancelamento ocorra na nota certa
608             cLiMsg := SL1->L1_NUMORIG + "|" + SL1->L1_DOC + "|" + SL1->L1_PDV + "|" + cNFisCanc + Iif (lPdvOn, "|" + SL1->L1_NUM, "")
609         Else
610             If!Empty(SL1->L1_SERRPS) .AND. !Empty(SL1->L1_DOCRPS) //Cancelamento de Venda com RPS
611                 cLiMsg := SL1->L1_NUMORIG + "|" + SL1->L1_DOC + "|" + SL1->L1_PDV + "|" + SL1->L1_SERRPS + "|" + SL1->L1_NUM
612             Else
613                 cLiMsg := SL1->L1_NUMORIG + "|" + SL1->L1_DOC + "|" + SL1->L1_PDV + "|" + SL1->L1_SERIE + Iif (lPdvOn, "|" + SL1->L1_NUM, "")
614             Endif
615         EndIf
616
617         LjGrvLog( cNumSale,"Dados armazenados para cancelamento via SLI: ",cLiMsg ) //Gera LOG
618
619         cStation := Space(TamSX3("LI_ESTACA0")[1])
620         lRet := STFSLICreate( cStation , "CAN" , cLiMsg , "NOVO" )
621
622         //Se registro em uso, rotina STFSaveTab tem controle padrao para aguardar liberacao. Nao compromete o processo, quando cancelamento sobe, tambem atualiza o STORC
623         //Cenario pouco provavel: Registro em uso nesse momento + retaguarda fora + fechamento de caixa. Impacto: Venda será considerada no movimento do caixa.
624         If !lPdvOn .OR. (lPdvOn .AND. lCentPDV .AND. lUseSat)

```

[illegible]

```

677     SL1->(DbSetOrder())
678     SL1->(DbSeek(xFilial("SL1")+cNumSale))
679
680     DbSelectArea("SL2")
681     SL2->(DbSetOrder(1))
682     If DbSeek(xFilial("SL2")+cNumSale)
683         While !SL2->(Eof()) .AND. (SL2->L2_FILIAL+SL2->L2_NUM == xFilial("SL2")+cNumSale)
684             If STBAvalShopCard(SL2->L2_PRODUTO) .AND. !Empty(SL2->L2_NUMCFID) .AND. !Empty(SL2->L2_DTSDFID) .AND. (SL2->L2_VLRFCID > 0)
685                 aParam := {SL2->L2_NUMCFID,SL1->L1_DOC,SL1->L1_SERIE,SL1->L1_LOJA,"4"}
686                 //Cancelamento de venda de recarga de cartao fidelidade
687                 If STBRemoteExecute("CA280ESLD",aParam,NIL,.T.,@uResult)
688                     If ValType(uResult) == "L" .AND. !uResult
689                         aSL1 := {"L1_SITUA", "00"} // Retira a solicitação de cancelamento
690                         STFSaveTab( "SL1" , aSL1 ) // Salva na tabela a alteracao
691                     EndIf
692                 Else
693                     // TO DO: Gerar contingencia
694                 EndIf
695                 Exit
696             EndIf
697             SL2->(DbSkip())
698         End
699     EndIf
700
701     //Cancelamento de venda com forma de pagamento FID - Cartao fidelidade
702     DbSelectArea("SL4")
703     DbSetOrder (1)
704     If DbSeek (xFilial("SL4")+cNumSale)
705         While SL4->(Eof()) .AND. SL4->L4_NUM == cNumSale
706             If Alltrim(SL4->L4_FORMA) == "FID"
707                 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA ¿
708                 //³Se a venda foi paga com cartao fidelidade, entao o ³
709                 //³movimento de saida sera estornado da tabela MBN ³
710                 //³e o valor devolvido ao respectivo saldo. ³
711                 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
712                 aParam := {SL2->L2_NUMCFID,SL1->L1_DOC,SL1->L1_SERIE,SL1->L1_LOJA,"3"}
713                 If STBRemoteExecute("CA280ESLD",aParam,NIL,.T.,@uResult)
714                     If ValType(uResult) == "L" .AND. !uResult
715                         aSL1 := {"L1_SITUA", "00"} // Retira a solicitação de cancelamento
716                         STFSaveTab( "SL1" , aSL1 ) // Salva na tabela a alteracao
717                     EndIf
718                 Else
719                     // TO DO: Gerar contingencia
720                 EndIf
721                 Exit
722             EndIf
723             SL4->(DbSkip())
724         End
725     EndIf
726 EndIf
727 EndIf
728

```

```

729 RestArea(aArea)
730
731 Return
732
733 //-----
734 /*{Protheus.doc} STDLogCanc
735 Apos cancelamento do cupom grava SLX
736 A gravacao da SLX no Totvs Pdv foi feita devido a integracao com a RMS
737
738 @param
739 @return
740 @obs
741 @sample
742 */
743 //-----
744 Function STDLogCanc(lIsItem , nItem , cSuper , lImportSale,;
745                    lDocNf , lCancCF )
746 Local lRet          := .T.                //Variavel de retorno
747 Local lLX_SITTRI    := SLX->(ColumnPos("LX_SITTRIB")) > 0
748 Local lSTBIsPAF     := STBIsPAF()
749 Local lVendaRecup    := ExistFunc("STWRSVerRe") .And. STWRSVerRe()
750 Local cPdv          := STFGetStat( "PDV" )    //Codigo da estacao
751 Local cCaixa        := xNumCaixa()           //Codigo do caixa
752 Local cSerie        := STFGetStat( "Serie" )  //Serie
753 Local aInfCan       := STIGetCancel()        //Informacoes do cancelamento
754 Local aArea         := {}
755 Local oModelCesta   := STDGPBModel()         //Model da cesta
756 Local cSupervisor   := ""                   // Supervisor
757 Local nI            := 0
758 Local cSerNf        := ""                   //Armazena a serie nao fiscal
759 Local cAux          := ""
760 Local cItem         := ""
761 Local cDoc          := ""
762 Local cFilSLX       := ""
763 Local cLXItem       := ""
764 Local lFoundSLX     := .F.
765 Local cSerNFe       := PadR( GetNewPar("MV_LOJANF",""), len(cSerie) ) //Ajuste para NFe
766 Local lEmitNfce     := Iif(FindFunction("LjEmitNFCe"), LjEmitNFCe(), .F.) // Sinaliza se utiliza NFC-e
767 Local lCancRPS      := .F.                 // Cancelamento de venda somente RPS
768
769 Default lIsItem      := .F.
770 Default nItem        := 0
771 Default cSuper       := ""
772 Default lImportSale  := .F.                 //Controla se houve importação de orçamento.
773 Default lDocNf       := .F.
774 Default lCancCF      := .F.
775
776 If !(lImportSale .AND. lSTBIsPAF)           //Se for orçamento importado e ambiente PAF, não pede a autorização novamente.
777     If Empty(cSuper)
778         cSupervisor := STFProFile(7)[2]      // Supervisor, aqui chamara a tela de supervisor
779     Else
780         cSupervisor := cSuper                // Supervisor vindo da rotina STWItemCancel.Protecao para nao chamar a tela supervisor duas vezes

```

```

781     EndIf
782 Endif
783
784 Aadd(aArea,GetArea())
785 DbSelectArea("SLX")
786 Aadd(aArea,SLX->(GetArea()))
787 Aadd(aArea,SL2->(GetArea()))
788 Aadd(aArea,SL1->(GetArea()))
789
790 //Cancelamento Item
791 If lIsItem
792
793     oModelCesta := oModelCesta:GetModel("SL2DETAIL")
794     oModelCesta:GoLine(nItem)
795
796     cSerie := IIF(!lDocNf,Iif( cSerie <> cSerNFe,cSerie, cSerNFe ),cSerNf)
797
798     If lSTBIsPAF
799         cDoc := Right(SL1->L1_DOC, TamSX3("LX_CUPOM")[1])
800     Else
801         cDoc := PadL( Iif( lEmitNFCe .AND. Empty(AllTrim(SL1->L1_DOC)), oModelCesta:GetValue("L2_DOC"), AllTrim(SL1->L1_DOC)), TamSX3("LX_CUPOM")[1] , "0" )
802         cDoc := STDTrataDoc( PADR(cPDV,TamSX3("LX_PDV")[1]) + PADR(cDoc,TamSX3("LX_CUPOM")[1]) + PADR(cSerie,TamSX3("LX_SERIE")[1]) +
PADR(oModelCesta:GetValue("L2_ITEM"),TamSX3("LX_ITEM")[1]) + SubStr(Time(),1,5) ,cDoc)
803     Endif
804
805     SLX->(RecLock("SLX", .T.))
806     REPLACE SLX->LX_FILIAL WITH xFilial("SLX")
807     REPLACE SLX->LX_PDV WITH cPDV
808     REPLACE SLX->LX_OPERADO WITH cCaixa
809     REPLACE SLX->LX_DTMOVTO WITH dDatabase
810     REPLACE SLX->LX_HORA WITH Time()
811
812     //Tratamento para Totvs PDV com NFe
813     REPLACE SLX->LX_SERIE WITH cSerie
814
815     REPLACE SLX->LX_CUPOM WITH cDoc
816     REPLACE SLX->LX_ITEM WITH oModelCesta:GetValue("L2_ITEM")
817     REPLACE SLX->LX_TPCANC WITH "I"
818
819     If SLX->(ColumnPos("LX_VRUNIT")) > 0
820         REPLACE SLX->LX_VRUNIT WITH oModelCesta:GetValue("L2_VRUNIT")
821     Endif
822
823     REPLACE SLX->LX_VALOR WITH oModelCesta:GetValue("L2_VLRITEM") + oModelCesta:GetValue("L2_VALDESC")
824     REPLACE SLX->LX_PRODUTO WITH oModelCesta:GetValue("L2_PRODUTO")
825     REPLACE SLX->LX_QTDE WITH oModelCesta:GetValue("L2_QUANT")
826     REPLACE SLX->LX_SUPERVI WITH cSupervisor
827     REPLACE SLX->LX_SITUA WITH "00"
828
829     If SLX->(ColumnPos("LX_PRCTAB")) > 0
830         REPLACE SLX->LX_PRCTAB WITH oModelCesta:GetValue("L2_PRCTAB")
831     Endif

```

```

832
833   If lSTBIsPAF
834       REPLACE SLX->LX_CONTD0C WITH STBRetCoD()
835   EndIf
836
837   cAux := AllTrim(oModelCesta:GetValue("L2_SITTRIB"))
838   If !Empty(cAux) .And. Substr(cAux,1,1) == "T" //somente grava se for aliquota de ICMS
839       REPLACE SLX->LX_ALIQICM WITH Val(Substr(cAux,2,Len(cAux)))/100
840   EndIf
841
842   If lLX_SITTRI
843       REPLACE SLX->LX_SITTRIB WITH cAux
844   EndIf
845
846   REPLACE SLX->LX_DESCON WITH oModelCesta:GetValue("L2_VALDESC")
847
848   SLX->(MsUnlock())
849
850   // Seto o numero do registro da SLX no item da SL2
851   If oModelCesta:HasField("L2_REGSLX")
852       STDSPBasket( "SL2" , "L2_REGSLX" , SLX->(Recno()))
853   Endif
854 Else //Cancelamento Cupom
855
856   oModelCesta := oModelCesta:GetModel("SL2DETAIL")
857   //Se houverem itens na cesta, cancelamento de cupom aberto
858   If oModelCesta:Length() > 0 .And. !Empty(oModelCesta:GetValue("L2_PRODUTO"))
859
860       //Quando PAF-ECF e cancelamento de venda no meio da venda
861       //esta duplicando caso o registro ja tenha sido cancelado
862       If lCancCF
863           //Como abaixo serão criados todos os itens de novo, eu apago caso já exista algum
864           SLX->(DbSetOrder( 1 )) //LX_FILIAL+LX_PDV+LX_CUPOM+LX_SERIE+LX_ITEM
865
866           cAux := xFilial("SLX") + PadR(cPDV, TamSX3("LX_PDV")[1]) + ;
867                   PadR(AllTrim(SL1->L1_D0C), TamSX3("LX_CUPOM")[1]) + PadR(cSerie, TamSX3("LX_SERIE")[1])
868
869           If SLX->(DbSeek(cAux))
870               While !SLX->(Eof()) .And. SLX->(LX_FILIAL+LX_PDV+LX_CUPOM+LX_SERIE) == cAux
871                   IF !(AllTrim(SLX->LX_ITEM) == "00")
872                       RecLock("SLX",.F.)
873                       SLX->(DbDelete())
874                       SLX->(MsUnlock())
875                   EndIf
876                   SLX->(DbSkip())
877               End
878           EndIf
879       EndIf
880
881       For nI := 1 To oModelCesta:Length()
882           oModelCesta:GoLine(nI)
883

```

```

884      cItem := oModelCesta:GetValue("L2_ITEM")
885      cDoc  := Right(SL1->L1_DOC, TamSX3("LX_CUPOM")[1])
886
887      SLX->(DbSetOrder(1)) //LX_FILIAL, LX_PDV, LX_CUPOM, LX_SERIE, LX_ITEM, LX_HORA
888
889      /*
890      Valido se é uma venda recuperada pois se efetuo um cancelamento do ultimo
891      cupom e houve itens cancelados eu devo considerar todos os itens novos pois
892      há um reordenamento do numero do item
893      */
894      If lVendaRecup .And. SLX->(DbSeek(xFilial("SLX") + cPDV + cDoc + cSerie + cItem))
895          //-- Essa situação é um item cancelado de um cupom cancelado que existe na SL2 e SLX
896          If SLX->LX_TPCANC == "I"
897              LjGrvLog( NIL, "Item [" + AllTrim(cItem) + "] já foi gravado na SLX [Num. Doc:" + AllTrim(cDoc) + ;
898                  "]" com cancelamento de item - não sera gravado no cancelamento total")
899          Else
900              SLX->(RecLock("SLX", .F.))
901              REPLACE SLX->LX_TPCANC WITH "A" //Insiro "A" para que não seja acumulado duas vezes no XML do PAF
902              SLX->(MsUnlock())
903          EndIf
904      Else
905          SLX->(RecLock("SLX", .T.))
906          REPLACE SLX->LX_FILIAL WITH xFilial("SLX")
907          REPLACE SLX->LX_PDV WITH cPDV
908          REPLACE SLX->LX_OPERADO WITH cCaixa
909          REPLACE SLX->LX_DTMOVTO WITH dDatabase
910          REPLACE SLX->LX_HORA WITH Time()
911
912          //Tratamento para Totvs PDV com NFe
913          REPLACE SLX->LX_SERIE WITH IIF(!lDocNf,;
914              iif( cSerie <> cSerNFe,cSerie, GetMV("MV_LOJANF") ),;
915              cSerNf)
916
917          REPLACE SLX->LX_CUPOM WITH cDoc
918          REPLACE SLX->LX_ITEM WITH cItem
919          REPLACE SLX->LX_TPCANC WITH "A"
920          REPLACE SLX->LX_VRUNIT WITH oModelCesta:GetValue("L2_VRUNIT")
921          REPLACE SLX->LX_VALOR WITH oModelCesta:GetValue("L2_VLRITEM") + oModelCesta:GetValue("L2_VALDESC")
922          REPLACE SLX->LX_PRODUTO WITH oModelCesta:GetValue("L2_PRODUTO")
923          REPLACE SLX->LX_QTDE WITH oModelCesta:GetValue("L2_QUANT")
924          REPLACE SLX->LX_SUPERVI WITH cSupervisor
925          REPLACE SLX->LX_SITUA WITH "00"
926          If SLX->(ColumnPos("LX_PRCTAB")) > 0
927              REPLACE SLX->LX_PRCTAB WITH oModelCesta:GetValue("L2_PRCTAB")
928          EndIf
929
930          If lSTBIsPAF
931              REPLACE SLX->LX_CONTDLOC WITH STBRetCoD()
932          EndIf
933
934          cAux := AllTrim(oModelCesta:GetValue("L2_SITTRIB"))
935          If !Empty(cAux) .And. Substr(cAux,1,1) == "T" //somente grava se for aliquota de ICMS

```

```

936         REPLACE SLX->LX_ALIQUICM WITH Val(Substr(cAux,2,Len(cAux)))/100
937     EndIf
938
939     If lLX_SITTRI
940         REPLACE SLX->LX_SITTRIB WITH cAux
941     EndIf
942
943     REPLACE SLX->LX_DESCON WITH oModelCesta:GetValue("L2_VALDESC")
944
945     SLX->(MsUnlock())
946 EndIf
947 Next nI
948 Else
949     //Cancelamento de cupom encerrado
950     SL2->(dbSetOrder(1)) //L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
951     SL2->(dbSeek(SL1->L1_FILIAL + SL1->L1_NUM))
952
953     If lDocNf
954         If !Empty(SL1->L1_DOCRPS) .AND. !Empty(SL1->L1_SERRPS) //Cancelamento RPS
955             cSerNf := SL1->L1_SERRPS
956             lCancRPS:= .T.
957         Else
958             cSerNf := PadR( STFGetStation("SERNFIS"), SL1->(TamSx3("L1_SERPED"))[1] )
959         Endif
960     ElseIf SL1->L1_IMPNF .AND. !Empty(cSerNFe)
961         cSerie := PadR(cSerNFe, len(SL1->L1_SERIE))
962     EndIf
963
964     cFilSLX := xFilial("SLX")
965     While SL2->(EOF()) .And. SL2->L2_FILIAL == SL1->L1_FILIAL .And. SL2->L2_NUM == SL1->L1_NUM
966
967         SLX->(DbSetOrder(1)) //LX_FILIAL, LX_PDV, LX_CUPOM, LX_SERIE, LX_ITEM, LX_HORA
968         /*
969         Valido se é uma venda recuperada pois se efetuo um cancelamento do ultimo
970         cupom e houve itens cancelados eu devo considerar todos os itens novos pois
971         há um reordenamento do numero do item
972         */
973         If lVendaRecup .And. SLX->(DbSeek(cFilSLX + SL2->L2_PDV + SL2->L2_DOC + SL2->L2_SERIE + SL2->L2_ITEM))
974             //-- Essa situação é um item cancelado de um cupom cancelado que existe na SL2 e SLX
975             If SLX->LX_TPCANC == "I"
976                 LjGrvLog( NIL, "Item [" + AllTrim(SL2->L2_ITEM) + "] já foi gravado na SLX [Num. Doc:" + AllTrim(SL2->L2_DOC) + ;
977                     "]" com cancelamento de item - não sera gravado no cancelamento total")
978             Else
979                 SLX->(RecLock("SLX", .F.))
980                 REPLACE SLX->LX_TPCANC WITH "A" //Insiro "A" para que não seja acumulado duas vezes no XML do PAF
981                 SLX->(MsUnlock())
982             EndIf
983         Else
984
985             /*Ajuste para o SQLite:
986             - quando se tem um deletado e um item vendido o L2_ITEM muda e
987             com isso ao cancelar a venda em cupom finalizado deve-se alterar

```

```

988      o LX_ITEM para evitar erro de chave duplicada aqui na SLX*/
989      nI := 0
990      cLXItem := ""
991      lFoundSLX := SLX->(DbSeek(cFilSLX + SL2->L2_PDV + SL2->L2_DOC + SL2->L2_SERIE + SL2->L2_ITEM))
992      While lFoundSLX
993          nI += 1
994
995          If Len(AllTrim(Str(nI))) == 1
996              cLXItem := '0' + AllTrim(Str(nI))
997          Else
998              cLXItem := AllTrim(Str(nI))
999          EndIf
1000
1001          If Val(cLXItem) > 99
1002              cLXItem := STBPegaIT(Val(cLXItem))
1003          EndIf
1004
1005          cLXItem := AllTrim(cLXItem)
1006          lFoundSLX := SLX->(DbSeek(cFilSLX + SL2->L2_PDV + SL2->L2_DOC + SL2->L2_SERIE + PadR(cLXItem,TamSX3("L2_ITEM")[1])))
1007      End
1008
1009      SLX->(RecLock("SLX", .T.))
1010      REPLACE SLX->LX_FILIAL WITH xFilial("SLX")
1011      REPLACE SLX->LX_PDV WITH cPDV
1012      REPLACE SLX->LX_OPERADO WITH cCaixa
1013      REPLACE SLX->LX_DTMOVTO WITH dDatabase
1014      REPLACE SLX->LX_HORA WITH Time()
1015      REPLACE SLX->LX_SERIE WITH IIF(!lDocNf, cSerie, cSerNf)
1016      REPLACE SLX->LX_CUPOM WITH IIF(!lDocNf, Right(SL1->L1_DOC, TamSX3("LX_CUPOM")[1]), Iif(lCancRPS, SL1->L1_DOCRPS, Right(STDGetDocP(), TamSX3("LX_CUPOM")[1])))
1017
1018      REPLACE SLX->LX_ITEM WITH IIF(Empty(cLXItem),SL2->L2_ITEM,cLXItem)
1019      REPLACE SLX->LX_TPCANC WITH "A"
1020      REPLACE SLX->LX_VRUNIT WITH SL2->L2_VRUNIT
1021      IF lSTBIsPAF
1022          REPLACE SLX->LX_VALOR WITH SL2->L2_VLRITEM + SL2->L2_VALDESC + SL2->L2_DESCPRO
1023      Else
1024          REPLACE SLX->LX_VALOR WITH SL2->L2_VLRITEM + SL2->L2_VALDESC
1025      EndIf
1026      REPLACE SLX->LX_PRODUTO WITH SL2->L2_PRODUTO
1027      REPLACE SLX->LX_QTDE WITH SL2->L2_QUANT
1028
1029      If ValType(aInfCan) == "A" .AND. Len(aInfCan) > 0
1030          REPLACE SLX->LX_SUPERVI WITH aInfCan[3]
1031      EndIf
1032
1033      REPLACE SLX->LX_SITUA WITH "00"
1034
1035      If SLX->(ColumnPos("LX_PRCTAB")) > 0
1036          REPLACE SLX->LX_PRCTAB WITH SL2->L2_PRCTAB
1037      EndIf
1038
1039      If lSTBIsPAF

```

```

1039         REPLACE SLX->LX_CONTD0C WITH STBRetCoD()
1040     EndIf
1041
1042     cAux := AllTrim(SL2->L2_SITTRIB)
1043     If !Empty(cAux) .And. Substr(cAux,1,1) == "T" //somente grava se for aliquota de ICMS
1044         REPLACE SLX->LX_ALIQICM WITH Val(Substr(cAux,2,Len(cAux)))/100
1045     EndIf
1046
1047     If lLX_SITTRI
1048         REPLACE SLX->LX_SITTRIB WITH cAux
1049     EndIf
1050
1051     If lSTBIsPAF
1052         REPLACE SLX->LX_DESCON WITH SL2->L2_VALDESC + SL2->L2_DESCPRO
1053     Else
1054         REPLACE SLX->LX_DESCON WITH SL2->L2_VALDESC
1055     Endif
1056
1057     SLX->(MsUnlock())
1058 EndIf
1059
1060     SL2->(dbSkip())
1061 EndDo
1062 EndIf
1063 EndIf
1064
1065 For nI:=1 to Len(aArea)
1066     RestArea(aArea[nI])
1067 Next nI
1068
1069 Return lRet
1070
1071 //-----
1072 /*{Protheus.doc} STDSitDoc
1073 Retorna o L1_SITUA a partir do L1_SERIE + L1_DOC + L1_PDV
1074
1075 @param cSerie      Numero de Série
1076 @param cNumDoc     Numero do Documento Fiscal
1077 @param cPDV        Numero do PDV
1078 @return cSitua     Campo L1_SITUA da venda
1079 @obs
1080 @sample
1081 */
1082 //-----
1083 Function STDSitDoc( cNumSerie, cNumDoc, cPDV, lDocNf, lDocRPS, cL1Num )
1084
1085 Local aArea      := GetArea()           // Armazena Area corrente
1086 Local aAreaSL1   := SL1->(GetArea())    // Armazena Area SL1
1087 Local aRet       := {}
1088 Local cDocNf     := "" //Documento nao fiscal
1089 Local cSerNf     := "" //Serie nao fiscal
1090

```

```

1091 Default cNumSerie      := ""
1092 Default cNumDoc        := ""
1093 Default cPDV            := ""
1094 Default lDocNf          := .F.
1095 Default lDocRPS         := .F.
1096 Default cL1Num          := ""
1097
1098
1099 If !lDocNf
1100     If lDocRPS //Cancelamento RPS
1101         cL1Num := PadR( cL1Num, SL1->(TamSx3("L1_NUM"))[1] )
1102         aRet := GetAdvFVal( "SL1", {"L1_SITUA", "L1_STORC"}, (xFilial("SL1")+cL1Num), 1, {"404", ""}, .T. )
1103     Else
1104         IF lPdvOn .AND. !Empty(cL1Num) //Cancelamento NFC-e do PDV Online
1105             cL1Num := PadR( cL1Num, SL1->(TamSx3("L1_NUM"))[1] )
1106             aRet := GetAdvFVal( "SL1", {"L1_SITUA", "L1_STORC"}, (xFilial("SL1")+cL1Num), 1, {"404", ""}, .T. )
1107         Else
1108             cNumSerie := PadR( cNumSerie , SL1->(TamSx3("L1_SERIE"))[1] )
1109             cNumDoc := PadR( cNumDoc , SL1->(TamSx3("L1_DOC"))[1] )
1110             cPDV := PadR( cPDV , SL1->(TamSx3("L1_PDV"))[1] )
1111             //o ultimo parametro indica se deve usar o valor default (penultimo parametro) caso nao encontre o registro (nao documento no TDN)
1112             aRet := GetAdvFVal( "SL1", {"L1_SITUA", "L1_STORC"}, (xFilial("SL1")+cNumSerie+cNumDoc+cPDV), 2, {"404", ""}, .T. )
1113         Endif
1114     Endif
1115 Else
1116     cDocNf := PadR( cNumDoc, SL1->(TamSx3("L1_DOCPED"))[1] )
1117     cSerNf := PadR( STFGetStation("SERNFIS"), SL1->(TamSx3("L1_SERPED"))[1] )
1118     aRet := GetAdvFVal( "SL1", {"L1_SITUA", "L1_STORC"}, (xFilial("SL1")+cSerNf+cDocNf), 11, {"404", ""}, .T. )
1119 Endif
1120
1121 RestArea(aAreaSL1)
1122 RestArea(aArea)
1123
1124 Return aRet
1125
1126 //-----
1127 /*/{Protheus.doc} STDCancTef
1128 Verifica se existe transação TEF ou em POS para cancelar a venda
1129
1130 @param cLastNum      Numero do orçamento
1131 @return lRet          .T.->Venda com TEF/POS/ .F.->Venda sem TEF/POS
1132 @obs
1133 @sample
1134 */
1135 //-----
1136 Function STDCancTef(cLastNum,lTemCCCD,lTemPD)
1137 Local lRet          := .F.
1138 Local aSL40rd       := {}
1139 Local lCmpsIdTRN     := SL4->(ColumnPos("L4_TRNID")) > 0 .And. SL4->(ColumnPos("L4_TRNPCID")) > 0 .And. SL4->(ColumnPos("L4_TRNEXID")) > 0 //Verifica se existem os campos que
    guardam os IDs das transações TEF
1140
1141 Default lTemCCCD := .F.

```

```

1142 Default lTempD := .F.
1143
1144 If AllTrim(STDCSLastSale("L1_SITUA")) <> "07"
1145
1146     aSl40rd := SL4->(GetArea())
1147
1148     SL4->(DbSetOrder(1))
1149     If SL4->(DbSeek(xFilial("SL4") + cLastNum))
1150         While !SL4->(Eof()) .AND. AllTrim(SL4->L4_FILIAL)+AllTrim(SL4->L4_NUM) == AllTrim(xFilial("SL4"))+AllTrim(cLastNum)
1151
1152             If (Alltrim(SL4->L4_FORMA) == "CC" .OR. Alltrim(SL4->L4_FORMA) == "CD") .And. ( (!Empty(SL4->L4_DOCTEF) .And. !Empty(SL4->L4_HORATEF) ) .Or. (lCmpsIdTRN .AND.
!Empty(SL4->L4_TRNPCID)))
1153                 lTemCCCD := .T.
1154             Endif
1155
1156             If Alltrim(SL4->L4_FORMA) $ "PD/PIX"
1157                 lTempD := .T.
1158             Endif
1159
1160             SL4->(DbSkip())
1161         Enddo
1162     EndIf
1163
1164     RestArea(aSl40rd)
1165
1166 EndIf
1167
1168 lRet := lTemCCCD .Or. lTempD
1169
1170 Return lRet
1171
1172 //-----
1173 /*{Protheus.doc} STDLastSat
1174 Busca Numero do Ultimo Orçamento SAT
1175
1176 @param cRetField - Nome do campo no dicionario de dados da tabela SL1
1177 @return cNumSale - Retorna o Numero ou outro campo da tabela SL1 da Ultima Venda
1178 @obs
1179 @sample
1180 */
1181 //-----
1182 Function STDLastSat(cRetField, cRetSerie)
1183
1184 Local cNumSale := "" // Retorna o Numero da Ultima Venda ou o campo do parametro
1185
1186 Default cRetField := "L1_NUM"
1187 Default cRetSerie := Nil //Tratamento por conta da NFe
1188
1189 // Posiciona o SL1 na ultima venda
1190 DbSelectArea("SL1")
1191
1192 If lPdvOn

```

```

1193     cNumSale := STDLastPdvOn(cRetField, @cRetSerie)
1194 Else
1195     SL1->( DbSetOrder(1) ) //L1_FILIAL+L1_NUM
1196     If SL1->( DbSeek(xFilial("SL1")) )
1197         SL1->(DbGoBottom())
1198
1199         //Macrosubstituicao consome muito recurso
1200         cNumSale := SL1->( FieldGet( ColumnPos(cRetField) ) )
1201
1202         //Tratamento por conta da NFe
1203         if cRetSerie <> Nil
1204             cRetSerie := SL1->L1_SERIE
1205         endif
1206     EndIf
1207 EndIf
1208
1209 Return cNumSale
1210
1211 /*{Protheus.doc} STDDelPay
1212 Reseta e/ou apaga os pagamentos ja realizados no BD
1213 @type      function
1214 @param      cNumSale, caracter, numero do orcamento
1215 @return     logico, os pagamentos foram resetados/excluidos da BD
1216 */
1217 Function STDDelPay(cNumSale)
1218
1219 Local cL4origem := Space( TamSX3("L4_ORIGEM")[1] )
1220 Local lRet      := .F.           //indica se excluiu algum pagamento
1221 Local lExcNCC   := .F.           //indica se zerou o campo L1_CREDITO (NCC)
1222 Local lExcSL4   := .F.           //indica se excluiu algum pagamento presente na SL4
1223 Local aSL1Area  := SL1->(GetArea()) //armazena a area da SL1
1224 Local aSL4Area  := SL4->(GetArea()) //armazena a area da SL4
1225 Local aArea     := GetArea()      //armazena a area corrente
1226
1227 Default cNumSale := ""
1228
1229 DbSelectArea("SL1")
1230 SL1->( DbSetOrder(1) ) //L4_FILIAL + L4_NUM
1231 If SL1->( DbSeek(xFilial("SL1") + cNumSale) )
1232     If SL1->L1_CREDITO > 0
1233         RecLock("SL1",.F.)
1234         Replace SL1->L1_CREDITO with 0
1235         SL1->( MsUnlock() )
1236
1237         lExcNCC := .T.
1238     EndIf
1239 EndIf
1240
1241 DbSelectArea("SL4")
1242 SL4->( DbSetOrder(1) ) //L4_FILIAL + L4_NUM + L4_ORIGEM
1243 lExcSL4 := SL4->( DbSeek(xFilial("SL4") + cNumSale + cL4origem) )
1244 If lExcSL4

```

```

1245 While SL4->( !EoF() ) .AND. xFilial("SL4") + cNumSale + cL4origem == SL4->L4_FILIAL + SL4->L4_NUM + SL4->L4_ORIGEM
1246   RecLock("SL4",.F.)
1247   SL4->( DbDelete() )
1248   SL4->( MsUnlock() )
1249
1250   SL4->( DbSkip() )
1251 End
1252 EndIf
1253
1254 RestArea(aSL1Area)
1255 RestArea(aSL4Area)
1256 RestArea(aArea)
1257
1258 //se excluiu a NCC ou os pagamentos da SL4
1259 If lExcNCC .OR. lExcSL4
1260   lRet := .T.
1261 Else
1262   LJGrvLog(cNumSale, "Nenhum pagamento (L1_CREDITO ou SL4) foi resetado")
1263 EndIf
1264
1265 Return lRet
1266
1267 //-----
1268 /*{Protheus.doc} STDAjstSLX
1269 Função responsável por ajustar o campo LX_CUPOM para os casos de
1270 item cancelado ou cupom cancelado em ambiente que utilize impressora
1271 nao-fiscal, no caso de NFC-e.
1272
1273 @param
1274 @return
1275 @obs
1276 @sample
1277 */
1278 //-----
1279 Function STDAjstSLX(cNumDOC)
1280 Local cPdv      := STFGetStat( "PDV" )      //Codigo da estacao
1281 Local cSerie    := STFGetStat( "Serie" )    //Serie
1282 Local nX        := 0
1283 Local nTamCupom := TamSX3("LX_CUPOM")[1]
1284 Local nTamPdv   := TamSX3("LX_PDV")[1]
1285 Local aSlxArea  := SLX->(GetArea()) //armazena a area da tabela SLX
1286 Local aArea     := GetArea()        //armazena a area corrente
1287 Local oModelCesta := STDGPBModel()   //Model da cesta
1288
1289 Default cNumDOC := ""
1290
1291 If !Empty(cNumDOC) .And. AllTrim(Upper(cNumDOC)) != "NFCE"
1292
1293   oModelCesta := oModelCesta:GetModel("SL2DETAIL")
1294
1295   For nX := 1 To oModelCesta:Length()
1296     oModelCesta:GoLine(nX)

```

```

1297
1298     If oModelCesta:HasField("L2_REGSLX")
1299         If oModelCesta:GetValue('L2_REGSLX') > 0
1300             SLX->(DbGoTo(oModelCesta:GetValue('L2_REGSLX'))) //Posiciona no Registro a ser atualizado
1301
1302             If AllTrim(Upper(SLX->LX_CUPOM)) == "NFCE"
1303                 RecLock("SLX", .F.)
1304                 SLX->LX_CUPOM := cNumDOC
1305                 SLX->(MsUnLock())
1306             EndIf
1307         Endif
1308     Endif
1309     Next nX
1310
1311 Else
1312     //Se nao foi passado o numero do DOC, entao os registros temporarios na SLX devem ser excluidos (Cujo DOC = "NFCe")
1313     DbSelectArea("SLX")
1314     SLX->( DbSetOrder(1) ) //LX_FILIAL+LX_PDV+LX_CUPOM+LX_SERIE+LX_ITEM+LX_HORA
1315     While SLX->( DbSeek( xFilial("SLX")+PadR(cPdv,nTamPdv)+PadR("NFCE",nTamCupom)+cSerie ) )
1316         RecLock("SLX", .F.)
1317         SLX->(DbDelete())
1318         SLX->(MsUnLock())
1319     End
1320 EndIf
1321
1322 RestArea(aSLxArea)
1323 RestArea(aArea)
1324
1325 Return
1326
1327 //-----
1328 /*{Protheus.doc} STDGetDocP()
1329 Retorna o numero do Doc Ped caso seja uma venda de cupom nao fiscal (venda entrega/vale presente)
1330
1331 @param
1332 @return  cDocPed -> Retorna o Doc Ped de uma venda nao fiscal
1333 */
1334 //-----
1335 Function STDGetDocP()
1336 Return cDocPed
1337
1338 //-----
1339 /*{Protheus.doc} STDUpdDocC()
1340 Realiza a gravação dos campos (DOCCANC, DATCANC, HORCANC) referentes ao cancelamento do TEF.
1341
1342 @param
1343 @param cChave, Caracter, Chave para busca na tabela SL4
1344 @param aDados, Array, Dados para gravação na SL4, array com a seguinte estrutura
1345     {
1346     {
1347         "Nome do campo",
1348         "Conteudo do campo"

```

```

1349     }
1350 }
1351
1352 @param aRecnos, Array, Recnos da SL4 ou SLV que devem ser atualizados com as informações passadas no array aDados.
1353 @param cAliasTab, Caractere, Alias da tabela a ser atualizada (SL4 ou SLV).
1354
1355 @return Null, Nulo
1356 */
1357 //-----
1358 Function STDUpdDocC(aDados,aRecnos,cAliasTab)
1359     Local nX      := 0
1360     Local nY      := 0
1361
1362     Default aDados      := {}
1363     Default aRecnos     := {}
1364     Default cAliasTab   := "SL4"
1365
1366     If Len(aDados) > 0 .And. Len(aRecnos) > 0
1367         DbSelectArea(cAliasTab)
1368
1369         For nX:=1 To Len(aRecnos)
1370             (cAliasTab)->(DbGoTo(aRecnos[nX]))
1371             RecLock(cAliasTab,.F.)
1372             For nY:=1 To Len(aDados)
1373                 &(aDados[nY][1]) := aDados[nY][2]
1374             Next nY
1375             (cAliasTab)->(MsUnLock())
1376         Next nX
1377     EndIf
1378
1379 Return
1380
1381 //-----
1382 /*{Protheus.doc} STDTrataDoc()
1383 Realiza o tratamento para o campo LX_CUPOM verificando o se ja existe a chave e caso exista inclua um sequencial no LX_CUPOM.
1384
1385 @param  cChave, Caracter, Chave para busca
1386 @param  cDocEnt, Caracter, LX_CUPOM Sem tratamento
1387 @return cDocSai, Caracter, LX_CUPOM Tratado
1388 */
1389 //-----
1390
1391 Static Function STDTrataDoc(cChave,cDocEnt)
1392     Local aSlxArea := {}
1393     Local aArea    := {}
1394     Local nX       := 1
1395     Local cDocSai   := cDocEnt
1396     Local cDocAux   := ""
1397
1398     If "NFCE" $ cDocEnt
1399         aSlxArea := SLX->(GetArea()) //armazena a area da tabela SLX
1400         aArea    := GetArea()

```

```

1401
1402     DbSelectArea("SLX")
1403     SLX->( DbSetOrder(1) ) //LX_FILIAL+LX_PDV+LX_CUPOM+LX_SERIE+LX_ITEM+LX_HORA
1404
1405     While SLX->(DbSeek(xFilial("SLX") + cChave)) .AND. nX <= 99999 // Maximo 99999
1406
1407         nLenDoc := TamSX3("L1_DOC")[1] - 4
1408
1409         cDocAux := STRZero(nX,nLenDoc) + "NFCF"
1410         cChave := StrTran(cChave,cDocEnt,cDocAux)
1411         cDocEnt := cDocAux
1412
1413         If SLX->(DbSeek(xFilial("SLX") + cChave))
1414             nX++
1415             Loop
1416         Else
1417             cDocSai := cDocEnt
1418             Exit
1419         EndIf
1420
1421     End
1422
1423     RestArea(aSLxArea)
1424     RestArea(aArea)
1425
1426 EndIf
1427
1428 Return cDocSai
1429
1430
1431 /*{Protheus.doc} STDGtDocRPS
1432     Retorna o numero do DOCRPS
1433     @return cDocRPS, caractere, número do documento do RPS
1434 */
1435 Function STDGtDocRPS()
1436 Return cDocRPS
1437
1438 /*{Protheus.doc} STDGtSerRPS
1439     Retorna o numero do SERRPS
1440     @return cSerRPS, caractere, série do RPS
1441 */
1442 Function STDGtSerRPS()
1443 Return cSerRPS
1444
1445 /*{Protheus.doc} STDLastPdvOn
1446     Retorna a ultima numeração para cancelamento no TOTVS PDV on-line.
1447     @param cRetField, caracter, nome do campo
1448     @param cRetSerie, caracter, numero da serie
1449     @return return_cNumSale, caracter , Retorna a numeração da ultima venda
1450 */
1451 Function STDLastPdvOn(cRetField, cRetSerie)
1452 Local cNumSale      := ""
1453
1454 // Numeração que será retornado

```

```

1453 Local cPdv      := PadR(STFGetStation("PDV"), TamSX3("L1_PDV")[1])
1454 Local cAliasSL1  := GetNextAlias()
1455 Local aDados     := {}
1456 Local cRotina    := ProcName(1)
1457 Local cQry      := ""
1458 Local cSgbd     := Upper(AllTrim(TcGetDB()))
1459
1460 Default cRetField := "L1_NUM"
1461 Default cRetSerie := STFGetStation("SERIE")
1462
1463 //Cria a query
1464 cQry := "SELECT "
1465
1466 If cSgbd $ "MSSQL"
1467     cQry += "TOP 1 L1_FILIAL,L1_NUM,L1_SERIE,L1_DOC "
1468 Else
1469     cQry += "L1_FILIAL,L1_NUM,L1_SERIE,L1_DOC "
1470 EndIf
1471
1472 cQry += "FROM " + RetSqlName("SL1")
1473 cQry += " WHERE "
1474 cQry += " L1_FILIAL = '" + xFilial("SL1") + "' "
1475 cQry += " AND L1_PDV = '" + cPdv + "' "
1476 cQry += " AND D_E_L_E_T_ = ' ' "
1477 cQry += " AND L1_SITUA NOT IN ('X0','X1','X2','X3', 'PR', 'FR') "
1478 cQry += " AND (L1_DOC <> ' ' OR L1_DOCRPS<> ' ') "
1479
1480
1481 Do Case
1482     Case cSgbd $ "MSSQL"
1483         cQry += "ORDER BY R_E_C_N_O_ DESC"
1484     Case cSgbd $ "POSTGRES"
1485         cQry += "ORDER BY R_E_C_N_O_ DESC LIMIT 1"
1486     Case cSgbd $ "ORACLE"
1487         cQry += "ORDER BY R_E_C_N_O_ DESC "
1488         cQry += "FETCH FIRST ROW ONLY"
1489 EndCase
1490
1491 cQry := ChangeQuery(cQry)
1492
1493 DbUseArea(.T., "TOPCONN", TcGenQry(,cQry),cAliasSL1,.T.,.F.)
1494
1495 If !(cAliasSL1)->(EOF())
1496
1497     aAdd(aDados,{L1_NUM,L1_DOC,L1_SERIE})
1498
1499     DbSelectArea("SL1")
1500     SL1->( DbSetOrder(1)) // L1_FILIAL+L1_NUM
1501     SL1->(DbSeek(xFilial("SL1")+aDados[1][1])) // Ultima numeração na SL1, posiciona na SL1 retornada da query
1502
1503     If cRetField == "L1_NUM" .And. (cRotina == Upper("STDCSLastSale") .Or. cRotina == Upper("STDLastSat"))
1504         cNumSale := aDados[1][1] // Ultima numeração na SL1

```

```

// Numero do PDV
// Guarda a workarea corrente
// Guarda os dados da venda
// Informa a rotina que chamou
// Query para realizar a busca do ultimo orçamento na SL1
// Retorna o banco que está sendo utilizado

```

```

// Serie padrão da SLG

```

```

1505         cRetSerie := aDados[1][3] // Numero da ultima Serie na SL1
1506     Else
1507         cNumSale := aDados[1][2] // Numero do ultimo Documento na SL1
1508         cRetSerie := aDados[1][3] // Numero da ultima Serie na SL1
1509     Endif
1510 EndIf
1511
1512 (cAliasSL1)->(DBCcloseArea())
1513
1514 Return cNumSale
1515
1516 /*{Protheus.doc} STDValIRes
1517     Rotina para verificar se existe item com Reserva quando PDVonline
1518     @type Static Function
1519     @param cNumOrig, caracteer, numero do Numorig
1520     @return lRet, lógico, se existe item com reserva retorna .T.
1521 */
1522 Static Function STDValIRes(cNumOrig)
1523 Local cPdv      := PadR(STFGetStation("PDV"), TamSX3("L1_PDV")[1])
1524 Local cQry      := ""
1525 Local lRet      := .F.
1526 Local oPrepared := Nil
1527
1528 If !Empty(cNumOrig)
1529
1530     cQry := "SELECT COUNT(L2_NUM) RESERVA "
1531     cQry += " FROM " + RetSqlName("SL2") + " WHERE "
1532     cQry += " L2_FILIAL = ?"
1533     cQry += " AND L2_PDV = ?"
1534     cQry += " AND L2_NUMORIG = ?"
1535     cQry += " AND L2_RESERVA <> ' ' "
1536     cQry += " AND L2_ENTREGA <> '2' "
1537     cQry += " AND L2_ENTREGA <> ' ' "
1538     cQry += " AND D_E_L_E_T_ = ' ' "
1539
1540     cQry := ChangeQuery(cQry)
1541
1542     oPrepared:=FWPreparedStatement():New(cQry)
1543     oPrepared:SetString(1, xFilial('SL2'))
1544     oPrepared:SetString(2, cPdv)
1545     oPrepared:SetString(3, cNumOrig)
1546
1547     cQry :=oPrepared:GetFixQuery()
1548
1549     lRet := !Empty(MpSysExecScalar(cQry,"RESERVA"))
1550
1551     If oPrepared != NIL
1552         oPrepared:Destroy()
1553         oPrepared := NIL
1554     Endif
1555
1556 Endif

```

```

// Numero do PDV
// Query para realizar a busca do ultimo orçamento na SL1

```

```
1557
1558 Return  lRet
1559
1560
1561 /*{Protheus.doc} STDRestCanL1
1562 Após cancelamento ou desistencia do cancelamento voltamos os campos para seu estado padrão.
1563 @return Nil, Nulo
1564 */
1565 Function STDRestCanL1()
1566
1567 Local cMVClipad  := SuperGetMV("MV_CLIPAD")
1568 Local cMVLojaPad := SuperGetMV("MV_LOJAPAD")
1569 Local oModelCli  := Nil      // Model do cliente selecionado
1570
1571 If !Empty(cMVClipad) .And. !Empty(cMVLojaPad)
1572     oModelCli := STWCustomerSelection(cMVClipad+cMVLojaPad)
1573     STDSPBasket("SL1", "L1_CLIENTE" ,oModelCli:GetValue("SA1MASTER", "A1_COD"))
1574     STDSPBasket("SL1", "L1_LOJA" ,oModelCli:GetValue("SA1MASTER", "A1_LOJA"))
1575     STDSPBasket("SL1", "L1_TIPOCLI" ,oModelCli:GetValue("SA1MASTER", "A1_TIPO"))
1576     STDSPBasket("SL1", "L1_CGCCLI" , "")
1577     STDSPBasket("SL1", "L1_PFISICA" , "")
1578 EndIf
1579
1580 Return Nil
```

Componentes/CancelSale/STWCancelSale.prw

851 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWCANCELSALE.CH"
4
5 Static lRms := SuperGetMv("MV_LJRMS",,.F.) //Integração com a RMS
6
7 /*
8
9 Static lLjcFid := SuperGetMv("MV_LJCFID",,.F.) .AND. CrdxInt() // T0 D0: Avaliar se sera implementado o CRDxINT
10 //Indica se a recarga de cartao fidelidade esta ativa
11
12 */
13 //-----
14 /*/{Protheus.doc} STWCancelSale
15 Cancelar Venda
16
17 @param lForceCancel      Define se força o cancelamento da venda, via erro por exemploVenda em andamento?
18 @param lIsProgressSale   Venda em andamento?
19 @param cSuperior          Superior que autorizou cancelamento
20 @param cDoc              Numero Doc a Cancelar (L1_DOC)
21 @param cNumSale          Numeração venda a cancelar (L1_NUM)
22 @param lCancVenc          Cancela no server a venda vencida
23 @param cNumForce         Força numeracao da venda
24 @return lRet             Retorna se efetuou cancelamento
25 @obs
26 @sample
27 */
28 //-----
29 Function STWCancelSale(    lForceCancel    , lIsProgressSale    , cSuperior    , cDoc    , ;
30                          cNumSale        , lCancVenc          , cNFisCanc    , cNumForce    , ;
31                          cSerie          , lTemCCCD          , lTemPD       , lDocRPS      )
32
33 Local lRet      := .F.                // Retorno funcao
34 Local aParam    := {}                 // Parametros passados para a retaguarda
35 Local cNome     := ""
36 Local cCGCCli   := ""
37 Local cL1NumOri := STDGPBasket("SL1" , "L1_NUMORIG")
38 Local cL1Num    := STDGPBasket("SL1" , "L1_NUM")
39 Local cL1PDV    := STDGPBasket("SL1" , "L1_PDV")
40 Local lUseSAT   := LjUseSat()          // Sinaliza se utiliza SAT
41
42 Default lForceCancel    := .F.
43 Default lIsProgressSale := .F.
44 Default cSuperior       := ""
45 Default cDoc            := ""
46 Default cNumSale        := ""
47 Default lCancVenc       := .F.
48 Default cNFisCanc       := ""
49 Default cNumForce       := ""
50 Default cSerie          := STFGetStation("SERIE")
51 Default lTemCCCD        := .F.
52 Default lTemPD          := .F.

```

```

53 Default lDocRPS          := .F. // Cancelamento RPS
54
55 LjGrvLog( "NumOrc: "+cNumSale+"/DOC: "+cDoc + "-" + cSerie, "Cancela venda" ) //Gera LOG
56
57 Do Case
58
59   Case lForceCancel
60
61     LjGrvLog( "Cancela venda","NumOrc: "+cNumSale+"/DOC: "+cDoc+" - lForceCancel = .T.")
62     lRet := STWCSForceCancel( lCancVenc, @cNome, @cCGCCli, cNumForce )
63
64   Case lIsProgressSale      // Venda em Andamento
65
66     LjGrvLog( "Cancela venda","NumOrc: "+cNumSale+"/DOC: "+cDoc+" - Venda em Andamento(lForceCancel = .T.)")
67     lRet := STWCSProgressSale( cSuperior, cDoc, @cNome, @cCGCCli, , /*"StiPosMain"*/ ,cSerie )
68
69   Case !lIsProgressSale     // Venda Finalizada
70
71     LjGrvLog( "Cancela venda","NumOrc: "+cNumSale+"/DOC: "+cDoc+"-"+cSerie+" - Venda Finalizada(lForceCancel = .F.)")
72     lRet := STWCSFinalized( cSuperior, cDoc, cNumSale, @cNome, @cCGCCli, cNFisCanc, "StiPosMain", lDocRPS, cSerie, lTemCCCD,lTemPD)
73     If AllTrim(cNumSale) <> AllTrim(cL1Num)
74       cL1Num := cNumSale
75     EndIf
76
77 EndCase
78
79 /*/
80   Verifica se já houve importação de orçamento. Caso positivo deleta arquivo de controle
81 /*/
82 If lRet
83   //Quando a venda já foi finalizada e vai cancelar o ultimo cupom, a cesta de vendas esta vazia mas
84   //o ponteiro esta posicionada na venda a ser cancelada, neste caso busco o L1_NUMORIG da linha posicionada na L1.
85   If Empty(cL1NumOri) .AND. AllTrim(cNumSale) == AllTrim(SL1->L1_NUM)
86     cL1NumOri := SL1->L1_NUMORIG
87   EndIf
88
89   If !(Empty(cL1NumOri))
90     //Exclui arquivo .TXT
91     aParam := {{ cL1NumOri, cL1PDV},Nil,.T.}
92     STBRemoteExecute( "FR271HArq", aParam,,.T. )
93
94     //Exclui arquivo .RET
95     aParam := {{ cL1NumOri, cL1PDV},Nil,.T.,, .T.}
96     STBRemoteExecute( "FR271HArq", aParam,,.T. )
97   Else
98     STFFireEvent(ProcName(0), "STGrvMdz", {"R4",cL1Num, cNome, cCGCCli}) //"CANCELAMENTO"
99   EndIf
100
101   //Apaga o arquivo de cancelamento SAT
102   If lUseSAT
103     LjSaCtrCnc(.F.,.T.,.F.,.F.,"") //Apaga o arquivo sinal de recuperação de cancelamento
104   EndIf

```

```

105 EndIf
106
107 STFS showMessage("STCancelSale")
108
109 Return lRet
110
111
112 //-----
113 /*{Protheus.doc} STWCSPProgressSale
114 Cancela Venda em Andamento
115
116 @param cSupervisor      ID do Usuário Supervisor
117 @param cDoc             Numero do documento
118 @return lRet            Retorna se Cancelou a venda
119 @obs
120 @sample
121 */
122 //-----
123 Function STWCSPProgressSale( cSupervisor , cDoc, cNome, cGCCLi, cSerie )
124
125 Local lRet           := .T.                // Retorna se Cancelou a venda
126 Local lAllNotfiscal := .F.                // Todos os Itens são não fiscais?
127 Local lDelChange    := SuperGetMV("MV_LJTRLOC",,.F.) // Define se Usa troco localizado
128 Local oTEF20        := Nil                // Objeto TEF
129 Local oCliModel      := Nil
130 Local aSLI           := {}                //Grava status da LI_SITUA
131 Local lSaveOrc       := IIF( ValType(STFGetCfg( "lSaveOrc" , .F. )) == "L" , STFGetCfg( "lSaveOrc" , .F. ) , .F. ) //Salva venda como orcamento
132 Local lNFCETSS       := STFGetCfg("lNFCETSS", .T.) //Enviar NFC-e pelo TSS
133
134 Default cSupervisor := ""
135 Default cDoc        := ""
136 Default cNome       := ""
137 Default cGCCLi      := ""
138 Default cSerie      := ""
139
140 LjGrvLog( "DOC: "+cDoc+"/"+cSerie, "Cancela Venda em Andamento" ) //Gera LOG
141
142 If ValType(lNFCETSS) == "U" //Default é enviar NFce pelo TSS
143     lNFCETSS := .T.
144 EndIf
145
146 lRet := STBCSChkStatusPrint()
147
148 If lRet
149
150     /*/
151     Todos os Itens possuem reserva?
152     /*/
153     If STBExistItemFiscal()
154         lAllNotfiscal := .F.
155     Else
156         lAllNotfiscal := .T.

```

```

157 EndIf
158
159 If !lAllNotfiscal .AND. !lSaveOrc
160     /*/
161     Cancelamento na impressora fiscal
162     /*/
163
164     lRet := STBCSCancCupPrint( cSupervisor , cDoc, /*cNumSale*/, lNFCETSS, /*lForceCancel*/, /*cTipoCanc*/, .T.)
165     If lRet
166
167         /*/
168         Desfazimento Transação TEF
169         /*/
170         oTEF20 := STBGetTef()
171         If ValType(oTEF20) == '0'
172             oTEF20:Desfazer()
173             LjGrvLog( "DOC: "+cDoc, "Desfazimento Transação TEF" ) //Gera LOG
174             If SuperGetMV("MV_LJHMTEF", ,.F.)
175                 STFMessage("TEF", "POPUP", STR0006 )// "Transação não foi efetuada. Favor reter o cupom."
176                 STFShowMessage( "TEF")
177             EndIf
178         EndIf
179     EndIf
180 EndIf
181
182 EndIf
183
184 If lRet
185     cCGCCli := SL1->L1_CGCCLI
186     oCliModel:= STWCustomerSelection(SL1->(L1_CLIENTE+L1_LOJA))
187     cNome := oCliModel:GetValue("SA1MASTER","A1_NOME")
188
189     //Verifica se grava flag nfce por TSS
190     If SL1->(ColumnPos("L1_TPTNFCE")) > 0
191         STDSPBasket( "SL1", "L1_TPTNFCE" , IIF(!lNFCETSS, "0", "1") )
192     EndIf
193
194     If !lRms
195         LjGrvLog( "L1_NUM: "+SL1->L1_NUM, "Deleta SL1, SL2 e SL4" ) //Gera LOG
196         STDCSDeleteSale( SL1->L1_NUM , lDelChange ) // Deleta SL1, SL2 e SL4 e SE5(troco localizado)
197     EndIf
198
199     //Se a integracao da RMS esta habilitado, grava SLX e L1_SITUA = 00 (Quando for cancelamento)
200     aSL1 := {"L1_SITUA", "00"}
201     STFSaveTab( "SL1" , aSL1 )
202
203     If !STbIsPaf() // o PAF grava a SLX no comando do cancelamento do ECF
204         STDLogCanc(,cSupervisor)
205     EndIf
206
207     If lAllNotfiscal
208         STFMessage("STCancelSale","STOP", STR0001 ) //Cupom Não Fiscal Cancelado

```

```

209      LjGrvLog( "L1_NUM: "+SL1->L1_NUM, "Cupom Não Fiscal Cancelado" ) //Gera LOG
210      Else
211          STFMessage("STCancelSale","STOP", STR0002 ) //Cupom Fiscal Cancelado
212          LjGrvLog( "L1_NUM: "+SL1->L1_NUM, "Cupom Fiscal Cancelado" ) //Gera LOG
213      EndIf
214
215      EndIf
216
217      LjGrvLog( "L1_NUM: "+SL1->L1_NUM, "Reinicia variáveis para iniciar próxima venda" ) //Gera LOG
218      STFRestart()
219
220      EndIf
221
222      Return lRet
223
224
225      //-----
226      /*{Protheus.doc} STWCSFinalized
227      Cancelar Vendas Finalizadas
228
229      @param cSupervisor      ID do Usuário Supervisor
230      @param cDoc              Numero do Documento
231      @param cNumSale          Numero da venda
232      @return lRet             Retorna se Cancelou a venda
233      @obs
234      @sample
235      */
236      //-----
237      Function STWCSFinalized( cSupervisor      , cDoc      , cNumSale      , cNome      , ;
238                              cCGCCli         , cNFisCanc , cFlowAction , lCancNf      , ;
239                              cSerie          , lTemCCCD , lTemPD          )
240      Local lRet              := .T.                // Retorna se Efetuou Cancelamento
241      Local oTEF20            := Nil                // Objeto TEF
242      Local oCliModel         := NIL               // Model do cliente
243      Local aEstornoNCC       := {}                // Array de parametros para realizar estorno de NCC
244      Local lNFCETSS          := STFGetCfg("lNFCETSS", .T.) // Enviar NFC-e pelo TSS
245      Local aVendaTEF         := {}                // Acumula a quantidade transacoes TEF
246      Local nX                := 1                // Variavel de controle do For
247      Local cTipoCanc         := ""                // tipo de cancelamento off-line
248      Local lUseSAT           := LjUseSat()         // Sinaliza se utiliza SAT
249      Local cStation          := STFGetStation("CODIGO") // Estacao atual, utilizado na gravacao da SLI de controle de comandos do ECF
250      Local aParamFlow        := {}
251      Local lExecCanCup       := .T.                //Sinaliza se deve realizar o comando de cancelamento de cupom
252      Local lSTBAddFlow       := ExistFunc("STBAddFlow")
253      Local lSTDatRecovery:= IsInCallStack("STDatRecovery")
254      Local cMvLjPref         := SuperGetMV("MV_LJPPREF") //Regra para gravacao do SF2->F2_PREFIXO
255      Local cPrefE1           := ""
256      Local nMvLjTpCan        := SuperGetMV("MV_LJTPCAN",,1) //Parametro que indica se utiliza o novo processo de cancelamento
257      Local oCancNfce         := Nil                //Armazena o objeto CancNfce
258      Local lEmitNFCe         := STBGetNFCe()       //Indica se NFC-e
259      Local nPosSL4           := 0
260      Local lCmpsIdTRN        := SL4->(ColumnPos("L4_TRNID")) > 0 .And. SL4->(ColumnPos("L4_TRNPCID")) > 0 .And. SL4->(ColumnPos("L4_TRNEXID")) > 0 //Verifica se existem os campos que

```

```

        guardam os IDs das transações TEF
261 Local cTRNID      := "" //ID da Transação (Payment Hub)
262 Local cTRNPCID    := "" //ID Transação Processador (Payment Hub)
263 Local cTRNEXID    := "" //ID da Transação Externa (Payment Hub)
264 Local lRetCncTEF  := .T. // Retorno da rotina de cancelamento tef
265
266 Default cSupervisor := ""
267 Default cDoc        := ""
268 Default cNumSale    := ""
269 Default cNFisCanc   := ""
270 Default cFlowAction := "" //Recebe conteudo quando reprocessando. Exemplo: STBCSCancCupPrint|1 , sendo o ultimo parametro a qtde de vezes que ja tentou reprocessar para
    evitar looping infinito.
271 Default lCancNf     := .F. //Sinaliza que eh uma venda nao fiscal venda de vale presente ou vale credito OU PODE SER VENDA DE SERVIÇO RPS
272 Default cSerie      := STFGetStation("SÉRIE")
273
274 Default lTemCCCD     := .F.
275 Default lTempD       := .F.
276
277 If Valtype(lNFCETSS) == "U" //Default é enviar NFCE pelo TSS
278     lNFCETSS := .T.
279 EndIf
280
281 //Guarda parametros da rotina para controle de fluxo de execucao
282 Add(aParamFlow,cSupervisor)
283 Add(aParamFlow,cDoc)
284 Add(aParamFlow,cNumSale)
285 Add(aParamFlow,cNome)
286 Add(aParamFlow,cCGCCli)
287 Add(aParamFlow,cNFisCanc)
288 Add(aParamFlow,cFlowAction)
289 Add(aParamFlow,.F.) //-- Evitar erro ao iniciar o recuperar venda pq senao cSerie entra no lugar
290 Add(aParamFlow,cSerie)
291
292 Add(aParamFlow,lTemCCCD)
293 Add(aParamFlow,lTempD)
294
295
296 DbSelectArea("SL1")
297 DbSetOrder(1)//L1_FILIAL+L1_NUM
298 lRet := SL1->(DbSeek( xFilial("SL1") + cNumSale))
299
300 If !lCancNf
301     LjGrvLog( "NumOrc: "+cNumSale+"/DOC: "+cDoc+"/"+cSerie, "Cancelar Venda Finalizada" ) //Gera LOG
302 Else
303     LjGrvLog( "NumOrc: "+cNumSale+"/DOC NF: "+cNFisCanc, "Cancelar Venda Finalizada" ) //Gera LOG
304 EndIf
305 STFMessage("STWCSFinalized","ALERT", STR0007 )// "CANCELANDO A VENDA, AGUARDE..."
306 STFShowMessage("STWCSFinalized")
307
308 If Empty(cDoc) .AND. !lCancNf
309     LjGrvLog( "Cancelamento da Venda", "Cancelamento nao processado. Motivo: Nao recebeu parametro com o DOC para cancelamento" )
310     lRet := .F.

```

```

311 EndIf
312
313 If lRet
314     If STDCSChkRes(cNumSale, SL1->L1_NUMORIG) // Checa se TODOS os Itens são com Reserva
315         lRet := .F.
316         MsgAlert(STR0003)
317         STFMessage("STCancelSale","STOP", STR0003) //Não será possível cancelar o pedido pois ja foi finalizado
318         LjGrvLog(cNumSale,STR0003)
319     EndIf
320 EndIf
321
322 //FlowControl: Cancela cupom no ECF -> "STBCSCancCupPrint"
323 If lRet
324     LjGrvLog( "NumOrc: "+cNumSale+"/DOC: "+cDoc+"/"+cSerie, "Cancelar Venda Finalizada" ) //Gera LOG
325
326     If "STBCSCancCupPrint" $ cFlowAction
327         LjGrvLog( "DOC: "+cDoc+"/"+cSerie, "Reprocessando cancelamento" )
328         IIF(lSTBAddFlow,STBAddFlow(cFlowAction,aParamFlow),Nil) //Controle de fluxo, sinaliza que ira executar a rotina STBCSCancCupPrint
329
330         If STWFlwCheck("STBCSCancCupPrint",aParamFlow) //verifica se houve ou nao o cancelamento no ECF
331             lExecCanCup := .F. //Sinaliza que nao deve mais executar o comando de cancelamento no ECF
332             cFlowAction := ""
333             lRet := .T.
334         EndIf
335     EndIf
336
337     If lExecCanCup
338         IIF(lSTBAddFlow,STBAddFlow("STBCSCancCupPrint",aParamFlow),Nil) //Controle de fluxo, sinaliza que ira executar a rotina STBCSCancCupPrint
339         lRet := STBCSCancCupPrint( cSupervisor , cDoc, , lNFCETSS,,@cTipoCanc, ,cSerie,lTemCCCD,lTemPD)
340     EndIf
341 EndIf
342
343 //FlowControl: Se possuir TEF, cancela a transacao(Desfazimento Transação TEF) -> "TEFDesfazer"
344 If lRet
345
346     IIF(lSTBAddFlow,STBAddFlow("TEFDesfazer",aParamFlow),Nil)
347
348     DbSelectArea("SL4")
349     SL4->(DbSetOrder(1))
350
351     If SL4->(DbSeek(xFilial("SL4") + cNumSale))
352         While !SL4->(Eof()) .AND. SL4->L4_FILIAL + SL4->L4_NUM == xFilial("SL4") + cNumSale
353
354             If (Alltrim(SL4->L4_FORMA) == "CC" .OR. Alltrim(SL4->L4_FORMA) == "CD" .OR. Alltrim(SL4->L4_FORMA) == "PD" .OR. Alltrim(SL4->L4_FORMA) == "PX" ) .AND. ( (
!Empty(SL4->L4_DOCTEF) .AND. !Empty(SL4->L4_HORATEF) ) .OR. (lCmpsIdTRN .AND. !Empty(SL4->L4_TRNPCID)))
355                 If lCmpsIdTRN
356                     cTRNID := Alltrim(SL4->L4_TRNID) //ID da Transação (Payment Hub)
357                     cTRNPCID := Alltrim(SL4->L4_TRNPCID) //ID Transação Processador (Payment Hub)
358                     cTRNEXID := Alltrim(SL4->L4_TRNEXID) //ID da Transação Externa (Payment Hub)
359                 EndIf
360
361                 nPosSL4 := aScan(aVendaTEF,{|x| AllTrim(x[1] + x[3] ) == Alltrim(SL4->(L4_DOCTEF)) + cTRNPCID })

```

```

362
363     If nPosSL4 == 0
364         Aadd(aVendaTEF, {} )
365         nPosSL4 := Len(aVendaTEF)
366         Aadd(aVendaTEF[nPosSL4], Alltrim(SL4->L4_DOCTEF) ) //01-DOCTEF
367         Aadd(aVendaTEF[nPosSL4], SL4->L4_VALOR ) //02-Valor Total da transação
368         Aadd(aVendaTEF[nPosSL4], cTRNID ) //03-ID da Transação
369         Aadd(aVendaTEF[nPosSL4], cTRNPCID ) //04-ID Transação Processador
370         Aadd(aVendaTEF[nPosSL4], cTRNEXID ) //05-ID da Transação Externa
371         Aadd(aVendaTEF[nPosSL4], SL4->L4_DATATEF ) //06-Data da Transação
372         Aadd(aVendaTEF[nPosSL4], Alltrim(SL4->L4_FORMA) ) //07-Forma de Pagamento
373         Aadd(aVendaTEF[nPosSL4], { SL4->(Recno()) } ) //08-Recnos relacionados a transação
374     Else
375         aVendaTEF[nPosSL4][2] += SL4->L4_VALOR //Aglutina o Valor Total da transação
376         aAdd( aVendaTEF[nPosSL4][8], SL4->(Recno()) ) //Recnos relacionados a transação
377     Endif
378 Endif
379 SL4->(DbSkip())
380 Enddo
381
382 // Faz a chamada da tela para cancelamento dos cartoes
383 For nX := 1 to Len(aVendaTEF)
384     lRetCncTEF:= STBFunAdm(,lSTDSatRecovery,aVendaTEF[nX])
385     If !lRetCncTEF
386         If MsgYesNo(STR0012+ Chr(10)+Chr(12)+STR0013) //"Ocorreu erro no Cancelamento TEF." //"Deseja tentar Novamente?"
387             nX := nX - 1
388         Else
389             MsgInfo("",STR0014) //"Cupom Fiscal será cancelado. Favor cancelar a transação TEF através do Menu TEF Gerencias"
390         Endif
391     Endif
392 Next nX
393 Endif
394
395 If Len(aVendaTEF) > 0
396     oTEF20 := STBGetTef()
397     If ValType(oTEF20) == '0'
398         oTEF20:Desfazer()
399         If SuperGetMV("MV_LJHMTEF",,.F.)
400             STFMessage("TEF", "POPUP", STR0006 )//"Transação não foi efetuada. Favor reter o cupom."
401             STFSHOWMessage( "TEF")
402         EndIf
403     EndIf
404 EndIf
405 EndIf
406
407 If lRet .AND. lEmitNFce .AND. !lCancNf .AND. !lUseSAT .AND. (nMvLjTpCan == 2 .OR. nMvLjTpCan == 3)
408
409     oCancNfce := LOJCNFCE():New()
410
411     //Tenta realizar o cancelamento Online
412     oCancNfce:LjCancOnline(SL1->(L1_SERIE+L1_DOC))
413

```

```

414 If oCancNfce:aResultCancelamento[1]
415 //Grava log informando que a venda foi cancelada online
416 LjGrvLog( "NumOrc: "+cNumSale, "NFC-e cancelada ONLINE, Id: " + SL1->(L1_SERIE+L1_DOC) )
417 STFMessage("CancNFCE", "ALERT", STR0009)//"Cancelamento realizado com sucesso."
418 STFShowMessage( "CancNFCE")
419 ElseIf oCancNfce:aResultMetodo[1] .AND. oCancNfce:aResultTSS[1] //Nota transmitida porem com rejeição (Online ou Offline Barra cancelamento)
420 lRet := .F. //Avisa que a nota foi rejeitada e printa na tela o motivo da rejeição.
421 LjGrvLog( "NumOrc: "+cNumSale, "Não foi possível realizar o cancelamento. Motivo: " + oCancNfce:aResultCancelamento[3])
422 STFMessage("CancNFCE", "POPUP", STR0010 + oCancNfce:aResultCancelamento[3])//"Não foi possível realizar o cancelamento. Motivo:
423 STFShowMessage( "CancNFCE")
424 Else
425 If nMvLjTpCan == 3 .And. !oCancNfce:aResultTSS[1]
426 LjGrvLog( "NumOrc: "+cNumSale, "Não foi possível realizar o cancelamento. Motivo: Sem comunicação com TSS / Sefaz.")
427 LjGrvLog( "NumOrc: "+cNumSale, "Retorno do TSS: " + oCancNfce:aResultTSS[3])
428 STFMessage("CancNFCE", "POPUP", STR0011)//"Não foi possível realizar o cancelamento. Motivo: Sem comunicação com TSS / Sefaz."
429 STFShowMessage( "CancNFCE")
430 lRet := .F.
431 ElseIf (nMvLjTpCan == 3 .AND. !oCancNfce:aResultMetodo[1] .AND. oCancNfce:aResultTSS[1]) .OR. nMvLjTpCan == 2
432 If oCancNfce:LjCancOffLine(SL1->L1_SERIE,SL1->L1_DOC, Substr(SL1->L1_KEYNFCE,21,2) )
433 LjGrvLog( "NumOrc: "+cNumSale, "Envio de cancelamento para o TSS realizado com sucesso!")
434 lRet := .T.
435 Else
436 LjGrvLog( "NumOrc: "+cNumSale, "Não foi possível realizar o envio do cancelamento para o TSS, o cancelamento devera ser realizado pelo job LJCANCNFCE")
437 lRet := .T.
438 EndIf
439 EndIf
440 EndIf
441 EndIf
442
443 If lRet
444
445 // Solicita cancelamento no Back Office
446 If lNFCETSS .OR. Empty(cTipoCanc)
447 LjGrvLog( "NumOrc: "+cNumSale, "Solicita cancelamento no Back Office" ) //Gera LOG
448 IIF(lSTBAddFlow,STBAddFlow("STDCSRRequestCancel",aParamFlow),Nil)
449 STDCSRRequestCancel( cNumSale,,cNFisCanc,cTipoCanc)
450 EndIf
451
452 /* TO D0: Avaliar se a variavel lLjcFid sera usada
453 If lLjcFid oTEF20
454 STDCSSShopCard( cNumSale )
455 EndIf
456 */
457
458 If lRet
459
460 IIF(lSTBAddFlow,STBAddFlow("STBCSCancGP",aParamFlow),Nil)
461
462 cCGCCli := STDGPBasket("SL1","L1_CGCCLI")
463 oCliModel := STWCustomerSelection(STDGPBasket("SL1","L1_CLIENTE")+STDGPBasket("SL1","L1_LOJA"))
464 cNome := oCliModel:GetValue("SAIMASTER","A1_NOME")
465

```

```

466     If lRMS
467         //Grava SLX
468         STFSLICreate( cStation, "TRN", "STWCSFinalized|Sistema|STDLogCanc|"+cDoc, "SOBREP0E" )    //Controle de ECF: Comando de cancelamento realizado no ECF,
processando gravacao
469     EndIf
470
471     //Permito Gravar o Log somente quando não usar PAF-ECF, pois dentro da função
472     //de cancelamento do ECF tem o comando para gerar o log
473     If lEmitNFCe .Or. !STBIsPaf()
474         STDLogCanc(,,cSupervisor,,lCancNf)
475     EndIf
476
477     If ExistFunc("STWRsnLCanCup") // Motivo de Cancelamento
478         STWRsnLCanCup(SL1->L1_FILIAL,SL1->L1_NUM)
479     Endif
480
481 EndIf
482
483 //
484 // Realiza estorno das NCCs utilizadas na venda
485 //
486 If SL1->L1_CREDITO > 0
487     //////////////////////////////////////
488     //Tratamento para retornar o prefixo correto de acordo com o parâmetro MV_LJPREF//
489     //////////////////////////////////////
490     If AllTrim(Upper(cMvLjPref)) <> "SF2->F2_SERIE"
491         cPrefE1 := PadR(&(cMvLjPref),TamSX3("F2_SERIE")[01]) //como o conteudo do parametro nao e padrao, ele e macro executado.
492     Else
493         cPrefE1 := IIf( !Empty( SL1->L1_SERIE )      , SL1->L1_SERIE      , SL1->L1_SERPED      )
494     EndIf
495
496     aEstornoNCC := {      SL1->L1_FILIAL      ,;
497                       IIf( !Empty( SL1->L1_DOC      )      , SL1->L1_DOC      , SL1->L1_DOCPED      )      ,;
498                       cPrefE1      ,;
499                       SL1->L1_CLIENTE      ,;
500                       SL1->L1_LOJA      }
501
502     //Executa o estorno da NCC no server
503     IIF(lSTBAddFlow,STBAddFlow("FRTNCCEXC",aParamFlow),Nil)
504     STBRemoteExecute( "FRTNCCEXC", aEstornoNCC , , .T. )
505 EndIf
506
507 If ExistBlock("STRelCanVc") .AND. lCancNf
508     LjGrvLog( NIL , "Antes da execução do P.E. STRelCanVc",{SL1->L1_FILIAL, SL1->L1_NUM})
509     ExecBlock("STRelCanVc", .F., .F., {SL1->L1_FILIAL, SL1->L1_NUM})
510     LjGrvLog( NIL , "Retorno da execução do P.E. STRelCanVc")
511 ElseIf lEmitNFCe .And. !lCancNf .AND. !lUseSAT
512     //Imprime comprovante Nao-Fiscal referente a Solicitacao de Cancelamento de NFC-e
513     LjNFCePrtC(SL1->L1_PDV, SL1->L1_DOC, SL1->L1_SERIE, dDatabase, Time())
514 EndIf
515
516 If STFUseFiscalPrinter()

```

```

517     STFMessagem("STCancelSale","STOP", STR0002) //Cupom Fiscal Cancelado
518     LjGrvLog( "NumOrc: "+cNumSale, "Cupom Fiscal Cancelado" ) //Gera LOG
519 Else
520     STFMessagem("STCancelSale","STOP", STR0004) //Venda Cancelada
521     LjGrvLog( "NumOrc: "+cNumSale, "Venda Cancelada" ) //Gera LOG
522 EndIf
523
524 IIF(lSTBAddFlow,STBAddFlow(""),Nil) // finaliza controle de fluxo da rotina
525
526 If !lSTDsSatRecovery
527     LjGrvLog( "NumOrc: "+cNumSale, "Reinicia variáveis para iniciar próxima venda" ) //Gera LOG
528     STFRestart(.T.)
529 EndIf
530
531 Else
532     IIF(lSTBAddFlow,STBAddFlow(""),Nil) // finaliza controle de fluxo da rotina
533 EndIf
534
535 //Ponto de Entrada apos concluir o cancelamento
536 If lRet .AND. ExistBlock("STCANSale")
537     LjGrvLog( NIL , "Antes da execução do P.E. STCANSale")
538     ExecBlock( "STCANSale",.F.,.F.)
539     LjGrvLog( NIL , "Retorno da execução do P.E. STCANSale")
540 EndIf
541
542 Return lRet
543
544 //-----
545 /*{Protheus.doc} STWCSForceCancel
546 Força Cancelamento
547
548 @param lCancVenc Cancela vendas vencidas
549 @param cNumSale Numero da venda a ser cancelada
550 @return lRet          Retorna se efetuou cancelamento
551 @obs
552 @sample
553 */
554 //-----
555 Function STWCSForceCancel(lCancVenc, cNome, cGCCLi , cNumSale )
556 Local lRet          := .T.          // Retorna se efetuou cancelamento
557 Local lIsOpenReceipt := .F.          // Define se o cupom está aberto
558 Local aRetPrinter    := {}          // Armazena retorno impressora
559 Local aProfile       := STFPFILE(8)
560 Local cSuperior      := ""          // ID do Usuário Superior
561 Local aIsOpenReceipt := { "5" , "" } // Armazena retorno se o cupom está aberto
562 Local lFiscalPrinter := STFUseFiscalPrinter() // Indica se utiliza impressora fiscal
563 Local oCliModel      := NIL
564 Local lNFCETSS       := STFGetCfg("lNFCETSS", .T.) //Enviar NFC-e pelo TSS
565 Local lProgrs        := STBCSIsProgressSale()
566 Local lInutiliza     := .F. //Inutiliza a venda
567 Local lEmitNFCe      := STBGetNFCe() //valida se é NFC-e ou não
568 Local lAutomato      := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX) //Verifica se eh automacao de teste

```

```

569
570 Default lCancVenc      := .F.
571 Default cNome          := ""
572 Default cCGCCli        := ""
573 Default cNumSale       := ""
574
575 If ValType(lNFCETSS) == "U" //Default é enviar NFce pelo TSS
576     lNFCETSS := .T.
577 EndIf
578
579 LjGrvLog( "L1_NUM: "+STDGPBasket("SL1","L1_NUM"), "Força Cancelamento da venda" )
580
581 //se a venda está em progresso, verifica o TEF para desfazer a transação
582 If lProgrs
583     STBSetRetTef( STIGetCard() )
584 EndIf
585
586 //validação para ver se o usuário tem permissão para cancelar venda
587 If Len(aProfile) > 0 .And. aProfile[1]
588     /*
589     Verifica cupom aberto
590     */
591     aRetPrinter := STFFireEvent(ProcName(0)      , ; // Nome do processo
592                               "STPrinterStatus" , ; // Nome do evento
593                               aIsOpenReceipt   )
594
595     If Len(aRetPrinter) > 0 .AND. ValType(aRetPrinter[1]) == "N"
596         If aRetPrinter[1] == 7
597             lIsOpenReceipt := .T.    // Aberto
598         Else
599             lIsOpenReceipt := .F.    // Fechado
600         EndIf
601     EndIf
602
603     If lIsOpenReceipt .OR. !lFiscalPrinter
604
605         /*
606         Verifica Status da impressora
607         */
608         lRet := STBCSChkStatusPrint()
609         If !lRet
610             STFMessage("STCancelSale","STOP", STR0005) //Erro com a Impressora Fiscal. Operação não efetuada. É necessário efetuar Nota de Devolução;
611             LjGrvLog( "L1_DOC: "+STDGPBasket("SL1","L1_DOC"), "Erro com a Impressora Fiscal. Operação não efetuada. É necessário efetuar Nota de Devolução" ) //Gera LOG
612         EndIf
613
614         If lRet
615
616             //Se nao recebeu numero da venda pega do model
617             If Empty(cNumSale)
618                 cNumSale := STDGPBasket("SL1","L1_NUM")
619             EndIf
620

```

```

621      oCliModel := STWCustomerSelection(STDGPBasket("SL1","L1_CLIENTE")+STDGPBasket("SL1","L1_LOJA"))
622      cNome      := oCliModel:GetValue("SA1MASTER","A1_NOME")
623      cSuperior  := aProfile[2]
624      lInutiliza := STDGPBasket("SL1","L1_SITUA") == "65"
625
626      If lNFCETSS .OR. !Empty(cNumSale)
627          cNumDoc := STDGPBasket("SL1","L1_DOC")
628
629          cCGCCli := STDGPBasket("SL1","L1_CGCLLI")
630          If !lNFCETSS
631              LjGrvLog( "L1_NUM: "+cNumSale, "Força Cancelamento da venda", lProgrs ) //Gera LOG
632              If lProgrs
633                  lRet := STBCSCancCupPrint( cSuperior , cNumDoc,cNumSale,lNFCETSS, , , @lInutiliza)
634              EndIf // OBS: 0 Front Nem valida o retorno da impressora
635          Else
636              STBCSCancCupPrint( cSuperior , cNumDoc,@cNumSale,lNFCETSS ) // OBS: 0 Front Nem valida o retorno da impressora
637          EndIf
638      ElseIf !lNFCETSS
639          //Se não for TSS (mobile), posiciona e cancela a ultima venda
640          cNumDoc := STDCSLastSale("L1_DOC")
641          cNumSale := STDCSLastSale("L1_NUM")
642          cCGCCli := STDCSLastSale("L1_CGCLLI")
643          lInutiliza := STDGPBasket("SL1","L1_SITUA") == "65"
644
645          //TSS valida o retorno para excluir a venda
646          If !lProgrs
647              cSuperior := aProfile[2]
648              lRet := STBCSCancCupPrint( cSuperior , @cNumDoc,@cNumSale,lNFCETSS, , , @lInutiliza ) // OBS: 0 Front Nem valida o retorno da impressora
649          Endif
650      EndIf
651
652      If lRet
653          If lNFCETSS .and. Empty(cNumSale)
654              cNumSale := STDCSLastSale()
655          EndIf
656
657          If !Empty(cNumSale)
658              If lNFCETSS .OR. (!lNFCETSS .and. lProgrs)
659                  STDCSDeleteSale( cNumSale)
660                  iif (ExistFunc("STWRsnLCanCup"), STWRsnLCanCup(), nil)
661              EndIf
662          Endif
663
664          If lCancVenc
665              // Solicita cancelamento no Back Office
666              STDCSRequestCancel( cNumSale , lCancVenc )
667          EndIf
668
669          If lFiscalPrinter
670              STFMessage("STCancelSale","STOP", STR0002) //Cupom Fiscal Cancelado
671              LjGrvLog( "Num0rc: "+cNumSale, "Cupom Fiscal Cancelado" ) //Gera LOG
672          Else

```

```

673         STFMesssage("STCancelSale","STOP", STR0004) //Venda Cancelada
674         LjGrvLog( "NumOrc: "+cNumSale, "Venda Cancelada" ) //Gera LOG
675     EndIf
676
677     If lEmitNFCe .Or. !STBIsRecovered()
678         STDAjstSLX(SL1->L1_DOC) // Deleta na SLX o registro referente a NFCe
679         STFRestart() // Zera variáveis de venda
680     EndIf
681
682     If !lAutomato .AND. !STBCSIsProgressSale() .AND. lFiscalPrinter .AND. !STBIsRecovered()
683         STIGridCupRefresh()
684         STIRegItemInterface()
685     Endif
686
687 EndIf
688 EndIf
689 EndIf
690 Else
691     STFMesssage("STCancelSale","STOP", "Sem permissão para cancelar a venda!") //Venda Cancelada
692     LjGrvLog( "NumOrc: "+cNumSale, "Sem permissão para cancelar a venda!" ) //Gera LOG
693 EndIf
694
695 Return lRet
696
697
698 //-----
699 /*{Protheus.doc} STWInuNFCE
700 Cria o registro na SLX para que a NFC-e seja inutilizada
701 @type      function
702 @param      cDoc, caracter, numero da Doc Fiscal
703 @param      [cSerie], caracter, serie do Doc Fiscal
704 @param      [cPDV], caracter, codigo do PDV em uso
705 @param      [cOperador], caracter, codigo do Operador
706 @param      [cStation], caracter, codigo da Estacao
707 @return      logico, se gerou SLX e a cesta foi renovada
708 */
709 //-----
710 Function STWInuNFCE(      cDoc      , cSerie      , cPDV      , cOperador ,;
711                        cStation , cMsgErro , cNumOrc )
712
713 Local lRet := .F.
714 Local aSLX := {}
715
716 Default cDoc      := ""
717 Default cSerie     := ""
718 Default cPDV      := ""
719 Default cOperador  := ""
720 Default cStation   := ""
721 Default cMsgErro   := ""
722 Default cNumOrc    := ""
723
724 //

```

```

725 // inclui o documento na SLX //Lj7SLXDocE("65", cDoc, cSerie, cPDV, cOperador, "X0")
726 //
727 Aadd( aSLX, {"LX_FILIAL"      , xFilial("SLX")} )
728 Aadd( aSLX, {"LX_CUPOM"      , cDoc          } )
729 Aadd( aSLX, {"LX_SERIE"      , cSerie       } )
730 Aadd( aSLX, {"LX_PDV"        , cPDV        } )
731 Aadd( aSLX, {"LX_OPERADO"    , cOperador   } )
732 Aadd( aSLX, {"LX_MODALDOC"   , iif( AllTrim(cSerie) == STFFGetStation("SERIE"), "65", "55") } )
733 Aadd( aSLX, {"LX_SITUA"      , "OK"        } )
734 Aadd( aSLX, {"LX_DTMOVTO"    , DDATABASE   } )
735 Aadd( aSLX, {"LX_HORA"       , Time()      } )
736 Aadd( aSLX, {"LX_TPCANC"     , "X"         } )
737
738 If SLX->(ColumnPos("LX_MOTIVO")) > 0
739     Aadd( aSLX, {"LX_MOTIVO"    , _NoTags(cMsgErro) } )
740 EndIf
741
742 If SLX->(ColumnPos("LX_NUMORC")) > 0
743     Aadd( aSLX, {"LX_NUMORC"    , cNumOrc      } )
744 EndIf
745
746 lRet := STFSaveTab( "SLX" , aSLX, .T. /*inclui um novo registro*/ )
747
748 //
749 // atualiza a cesta
750 //
751 If lRet
752     STDSPBasket("SL1", "L1_DOC"      , CriaVar("L1_DOC")      )
753     STDSPBasket("SL1", "L1_SERIE"    , CriaVar("L1_SERIE")   )
754     STDSPBasket("SL1", "L1_KEYNFCE"  , CriaVar("L1_KEYNFCE") )
755     STDSPBasket("SL1", "L1_STORC"    , CriaVar("L1_STORC")   )
756     STDSPBasket("SL1", "L1_HORA"     , CriaVar("L1_HORA")    )
757     STDSPBasket("SL1", "L1_SITUA"    , "04"                  )
758 Else
759     LJGrvLog(Nil, "Falha ao incluir o Doc na SLX", cDoc)
760 EndIf
761
762 Return lRet
763
764
765 /*{Protheus.doc} STWDelPay
766 Exclui os pagamentos e reseta a cesta
767 @type      function
768 @param     cNum, caracter, numero do orcamento
769 @return    logico, se todos os registros do bd foram excluidos e a cesta foi resetada
770 */
771 Function STWDelPay( cNum )
772
773 Local lRet := .F.
774 Local lFinishSales := IsInCallStack("STWFinishSale")
775 Local lUsePayHub := ExistFunc("LjUsePayHub") .And. LjUsePayHub()
776

```

```

777 Default cNum := ""
778
779 //
780 // Apaga os registros de pagamento, pois ao finalizar a venda novamente, eles serao recriados
781 lRet := STDDelPay(cNum)
782
783 If lRet
784     //
785     // Limpa os campos da cesta referente a Forma de Pagamento
786     STDSPBasket("SL1", "L1_DINHEIR" , CriarVar("L1_DINHEIR") )
787     STDSPBasket("SL1", "L1_CHEQUES" , CriarVar("L1_CHEQUES") )
788     STDSPBasket("SL1", "L1_CARTAO" , CriarVar("L1_CARTAO") )
789     If lUsePayHub
790         STDSPBasket("SL1", "L1_VLRPGDG" , CriarVar("L1_VLRPGDG") )
791         STDSPBasket("SL1", "L1_VLRPGPX" , CriarVar("L1_VLRPGPX") )
792     EndIf
793     STDSPBasket("SL1", "L1_CONVENI" , CriarVar("L1_CONVENI") )
794     STDSPBasket("SL1", "L1_VALES" , CriarVar("L1_VALES") )
795     STDSPBasket("SL1", "L1_FINANC" , CriarVar("L1_FINANC") )
796     STDSPBasket("SL1", "L1_OUTROS" , CriarVar("L1_OUTROS") )
797     STDSPBasket("SL1", "L1_ENTRADA" , CriarVar("L1_ENTRADA") )
798
799     //-- Quando se utiliza a condicao de pagamento e ocorre um erro de transmissao durante a venda e a mesma e inutilizada
800     //-- nao devemos apagar o conteudo destes campos, pois retorna para a tela de fechamento da venda e o usuario ira
801     //-- tentar finalizar novamente e finalizara sem os dados da condicao de pagamento colocando como condicao negociada
802     //-- como se tivesse finalizando a venda escolhendo as formas de pagamento e nao escolhendo o legado das condicoes de pagamento
803     if !lFinishSales
804         STDSPBasket("SL1", "L1_CONDPG" , CriarVar("L1_CONDPG") )
805         STDSPBasket("SL1", "L1_FORMPG" , CriarVar("L1_FORMPG") )
806     endif
807
808     STDSPBasket("SL1", "L1_CREDITO" , CriarVar("L1_CREDITO") )
809     STDSPBasket("SL1", "L1_VENDTEF" , CriarVar("L1_VENDTEF") )
810     STDSPBasket("SL1", "L1_DATATEF" , CriarVar("L1_DATATEF") )
811     STDSPBasket("SL1", "L1_HORATEF" , CriarVar("L1_HORATEF") )
812     STDSPBasket("SL1", "L1_DOCTEF" , CriarVar("L1_DOCTEF") )
813     STDSPBasket("SL1", "L1_AUTORIZ" , CriarVar("L1_AUTORIZ") )
814     STDSPBasket("SL1", "L1_INSTITU" , CriarVar("L1_INSTITU") )
815     STDSPBasket("SL1", "L1_NSUTEF" , CriarVar("L1_NSUTEF") )
816     STDSPBasket("SL1", "L1_PARCTEF" , CriarVar("L1_PARCTEF") )
817     STDSPBasket("SL1", "L1_IMPRIME" , CriarVar("L1_IMPRIME") )
818     STDSPBasket("SL1", "L1_VLRDEBI" , CriarVar("L1_VLRDEBI") )
819
820 EndIf
821
822 Return lRet
823
824 //-----
825 /*/{Protheus.doc} STWCancNF()
826 Cancela uma venda nao fiscal, vale credito, vale presente, venda de servico RPS
827 @type      function
828 @param

```

```

829 @return
830 /*/
831 //-----
832 Function STWCancNF()
833
834 Local lRet      := .F.                //Variavel de retorno
835 Local aCanc     := STIGetCancel()     //Retorna o conteudo do array aCancel
836 Local lAutomato := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX) //Verifica se eh automacao de teste
837
838 If Len(aCanc) >= 6
839     If Len(aCanc) >= 11 .AND. !Empty(aCanc[10]) .AND. !Empty(aCanc[11]) //RPS
840         lRet := STWCSFinalized( aCanc[3] , aCanc[10] , aCanc[5], , , aCanc[6], , .T.,aCanc[11])
841     Else
842         lRet := STWCSFinalized( aCanc[3] , aCanc[4] , aCanc[5], , , aCanc[6], , .T. )
843     Endif
844 EndIf
845
846 If !lAutomato
847     STIGridCupRefresh() // Sincroniza a Cesta com a interface
848     STIRegItemInterface()
849 EndIf
850
851 Return lRet

```

Componentes/Cashier/STBCloseCash.prw

46 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBCLOSECASH.CH"
4
5 //-----
6 /*{Protheus.doc} STBAutCloseCash
7 Verifica Autorizacao(permissao) de usuario e caixa para
8 Abrir/Fechar o caixa
9 @param
10 @return aRet Retorna permissao sim ou nao/ Supervisor
11 @obs
12 @sample
13 */
14 //-----
15 Function STBAutCloseCash()
16
17 Local cSupervisor := "" // Usuario Supervisor
18 Local aRet := {.F.,""} // Retorno
19
20 // Verifica permissao de usuario
21 If ChkPsw(41)
22
23 //Verifica permissao de caixa
24 aRet := STFProFile( 4 )// 4 - Permissao para abrir e fechar o caixa
25
26
27 If !aRet[1]
28
29 If lUsaDisplay
30 // Inicia Evento
31 STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "2C"+ STR0001 } ) //"Senha invalida ou acesso negado"
32 End
33
34 // Usuario / sem permissao para Abrir/Reabrir/Fechar o Caixa. / Atenção
35 STFMessage("OpenCash","STOP","Atencao, Usuario " + cUserName + STR0002) //Sem permissao para Abrir/Fechar o Caixa
36
37 If lUsaDisplay
38 STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "2E"+ STR0003 } ) //"Codigo do Produto: "
39 STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "1E"+ " " " } )
40 End
41
42 Endif
43
44 EndIf
45
46 Return aRet

```

Componentes/Cashier/STBConfCash.prw

46 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBCONF CASH.CH"
4
5 //-----
6 /*{Protheus.doc} STBValOpCash
7 Valida se o caixa ja esta aberto, caso não esteja, libera o usuario
8 para abertura do caixa, caso contrario bloqueia
9
10 @param
11 @return      cChave - Retorna a chave da SLW
12 @obs
13 @sample
14 */
15 //-----
16 Function STBValOpCash()
17
18 Local cChave      := ""                                //Recebe a chave da SLW que esta pendente
19 Local aId         := STBInfoEst( 1, .F., .T. )         //Array com informacoes da SLG
20 Local cNumMov     := AllTrim(STDNumMov())              //Retorno o numero do movimento atual
21 Local uRet        := Nil                               //Retorno da funcao LjUltMovAb
22
23 lContinue := STBRemoteExecute( "LjUltMovAb"           ,;
24                               {2,aId[1][1],"", "", "",.T.} ,;
25                               Nil                      ,;
26                               .F.                      ,;
27                               @uRet                   )
28
29 cChave := uRet
30
31 If lContinue
32
33     If Empty(cChave) .Or. MsgNoYes(STR0001,STR0002) //"0 fechamento deste caixa não foi atualizado na retaguarda, deseja continuar com a abertura do caixa ?" ## "Abertura de
        caixa"
34
35         //Executa a função no PDV
36         If Empty(cNumMov)
37
38             cChave := STDUtMovAb(2,aId[1][1],"", "", "",.T.)
39
40         EndIf
41
42     EndIf
43
44 EndIf
45
46 Return cChave

```

Componentes/Cashier/STBOpenCash.prw

322 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBOPENCASH.CH"
4
5 Static      nHandleCX      := 0              // Handle do arquivo de caixa criado na abertura de caixa
6
7 //-----
8 /*{Protheus.doc} STBOpenCash
9 Verifica se o caixa esta aberto
10
11 @param
12 @return lRet   retorna se o caixa esta aberto
13
14 @obs
15 @sample
16 */
17 //-----
18
19 Function STBOpenCash()
20
21 Local lRet      := .F.  // Retorno
22 Local cNumMov   := ""   // Numero do movimento
23 Local cNumCashier := "" // Numero do caixa
24
25 cNumCashier     := xNumCaixa()
26 cNumMov         := STDNumMov()
27
28 If cNumMov <> ""
29     lRet := .T.
30 EndIf
31
32 Return lRet
33
34
35 //-----
36 /*{Protheus.doc} STBInfoEst
37 Retorna Informacoes referente a estacao
38
39 @param  nOpc      1: Retorna array com dados | 2: retorna string com dados
40 @param  lFormat   Retorna formatacao do SX3
41 @param  lRetSize  Define se no array de retorno sera add tamanho do campo
42
43 @return uRet      [1]CAIXA [2]ESTACAO [3]SERIE [4]PDV
44
45 @obs
46 @sample
47 */
48 //-----
49
50 Function STBInfoEst(      nOpc      ,  lFormat ,  lRetSize)
51
52 Local aStation := {}      // Array com informacoes da estacao

```

```

53 Local aTMP          := {}          // Array temporario
54 Local uRet          := ""          // Variavel indefinida para retorno
55
56 Default nOpc         := 1          // 1: Retorna array com dados | 2: retorna string com dados
57 Default lFormat      := .F.        // Retorna formatacao do SX3?
58 Default lRetSize     := .F.        // Define se no array de retorno sera add tamanho do campo
59
60 ParamType 0 Var      nOpc          As Numeric      Default 1
61 ParamType 1 var      lFormat      As Logical      Default .F.
62 ParamType 2 var      lRetSize     As Logical      Default .F.
63
64 // Guarda no array temporario as informacoes da estacao
65 aTMP := STFGetStat({"CODIGO","SERIE","PDV","SERNFIS"})
66
67 If Len(aTMP) > 0
68
69     //Caso nao se utilize o retorno de tamanho de campos ou a opcao de retorno seja de string
70     If !lRetSize .OR. nOpc <> 1
71
72         aStation := Array(5)
73
74         If !lFormat
75             aStation[1] := AllTrim(xNumCaixa())
76             aStation[2] := AllTrim(aTMP[1])
77             aStation[3] := AllTrim(aTMP[2])
78             aStation[4] := AllTrim(aTMP[3])
79             aStation[5] := AllTrim(aTMP[4])
80         Else
81             aStation[1] := PadR(AllTrim(xNumCaixa()),TamSX3("L1_OPERADO")[1])
82             aStation[2] := PadR(AllTrim(aTMP[1]),TamSX3("LG_CODIGO")[1])
83             aStation[3] := PadR(AllTrim(aTMP[2]),TamSX3("LG_SERIE")[1])
84             aStation[4] := PadR(AllTrim(aTMP[3]),TamSX3("LG_PDV")[1])
85             aStation[5] := PadR(AllTrim(aTMP[4]),TamSX3("LG_SERNFIS")[1])
86         Endif
87
88         If nOpc == 1
89             uRet := aStation
90         Else
91             aEval(aStation,{|x| uRet += x})
92         Endif
93     Else
94
95         aStation := Array(5,2)
96
97         If !lFormat
98             aStation[1][1] := AllTrim(xNumCaixa())
99             aStation[2][1] := AllTrim(aTMP[1])
100             aStation[3][1] := AllTrim(aTMP[2])
101             aStation[4][1] := AllTrim(aTMP[3])
102             aStation[5][1] := AllTrim(aTMP[4])
103         Else
104

```

```

105         aStation[1][1] := PadR(AllTrim(xNumCaixa()),TamSX3("L1_OPERADO")[1])
106         aStation[2][1] := PadR(AllTrim(aTMP[1]),TamSX3("LG_CODIGO")[1])
107         aStation[3][1] := PadR(AllTrim(aTMP[2]),TamSX3("LG_SERIE")[1])
108         aStation[4][1] := PadR(AllTrim(aTMP[3]),TamSX3("LG_PDV")[1])
109         aStation[5][1] := PadR(AllTrim(aTMP[4]),TamSX3("LG_SERNFIS")[1])
110     Endif
111
112     aStation[1][2] := TamSX3("L1_OPERADO")[1]
113     aStation[2][2] := TamSX3("LG_CODIGO")[1]
114     aStation[3][2] := TamSX3("LG_SERIE")[1]
115     aStation[4][2] := TamSX3("LG_PDV")[1]
116     aStation[5][2] := TamSX3("LG_SERNFIS")[1]
117
118     If n0pc == 1
119         uRet := aStation
120     Else
121         aEval(aStation,{|x| uRet += x[1]})
122     EndIf
123
124 EndIf
125
126 EndIf
127
128 Return uRet
129
130
131 //-----
132 /*{Protheus.doc} FR271BSemaforo
133 Cria um Semaforo em Arquivo
134
135 @param cType          Tipo operacao
136 @param cStation       Estacao
137
138 @return lRet          Retorna se conseguiu gravar semaforo
139 @obs
140 @sample
141 */
142 //-----
143 Function STBTraffic( cType, cStation )
144
145 Local lRet          := .T.          // Retorno
146 Local lMobile       := STFGetCfg("lMobile", .F.)    //Smart Client Mobile
147
148 Default cType       := "CON"        // Tipo operacao
149 Default cStation    := ""           // Estacao
150
151 ParamType 0 Var     cType           As Character    Default "CON"
152 ParamType 1 var     cStation        As Character    Default ""
153
154 lMobile := ValType(lMobile) == "L" .AND. lMobile
155
156 //Em versoes Mobile nao faz controle por arquivo

```

```

157 //fisico apenas por registros em Base de dados
158 If !lMobile
159
160     // Se nao existir o diretorio cria
161     If !ExistDir("\SEMAFORO\SIGAFRT")
162         MakeDir("\SEMAFORO\SIGAFRT")
163     EndIf
164
165     // Tenta criar o arquivo de semaforo
166     nHandleCX := MSFCreate("\SEMAFORO\SIGAFRT"+StrTran(cType+cFilAnt+cStation," ","")+".L"+StrTran(cEmpAnt," ",""))
167
168     IF nHandleCX < 0
169         // Nao conseguiu criar arquivo de semaforo
170         lRet := .F.
171     EndIf
172
173 EndIf
174
175 // Grava SLI
176 STFSLICreate( cStation, "CON", "||||", "SOBREPOE" )
177
178 Return(lRet)
179
180
181 //-----
182 /*/{Protheus.doc} FR271BSemaforo
183 Retorna o Handle do arquivo de caixa criado na abertura de caixa
184
185 @return nHandleCX Retorna o Handle do arquivo de caixa criado na abertura de caixa
186 @obs
187 @sample
188 */
189 //-----
190 Function STBGetCXHnd()
191 Return nHandleCX
192
193
194 //-----
195 /*/{Protheus.doc} STBAutoOpenCash
196 Verifica Autorizacao(permissao) de usuario e caixa para
197 Abrir/Fechar/Reabrir o caixa
198
199 @param
200
201 @return aRet[1]Retorna permissao
202 @return aRet[2]Supervisor/Usuario
203 @obs
204 @sample
205 */
206 //-----
207 Function STBAutoOpenCash()
208

```

```

209 Local cSupervisor   := ""           // Usuario Supervisor
210 Local lRet          := {}           // Validacao
211 Local aRet          := {.F., ""} // Retorno com validacao se esta autorizado e usuario supervisor
212 Local aMov          := {}           // array com informacoes do movimento
213
214 // Verifica permissao de usuario
215 If ChkPsw(41)
216
217     //Verifica permissao de caixa
218     aMov := STDNumMov( {"LW_NUMMOV","LW_DTFECHA"}} )
219
220     // Se o caixa ja foi fechado na data atual, Ver permissao para reabrir
221     If ValType(aMov) == "A" .AND. Len(aMov) > 1
222         If aMov[2] == dDataBase
223             aRet := STFProFile(16) // 16 - Permissao para Reabrir o Caixa
224         EndIf
225     Else
226         aRet := STFProFile(4)// 04 - Permissao para abrir e fechar o caixa
227     Endif
228
229     If !aRet[1]
230
231         If lUsaDisplay
232             // Inicia Evento
233             STFFireEvent(ProcName(0), "STDdisplay", { StatDisplay(), "2C"+ STR0001 } ) //"Senha invalida ou acesso negado"
234         End
235
236         // Usuario / sem permissao para Abrir/Reabrir/Fechar o Caixa. / Atenção
237         STFMessage("OpenCash","STOP", STR0003 + cUserName + STR0002) //"Atencao, Usuario, Sem permissao para Abrir/Reabrir/Fechar o Caixa."
238     Endif
239
240 EndIf
241 EndIf
242
243 Return aRet
244
245
246 //-----
247 /*/{Protheus.doc} STBDtCash
248 Pega data para realizar abertura do caixa
249
250 @param
251 @return aDtHr Retorna array com data e hora [1]Data [2]hora
252 @obs
253 @sample
254 */
255 //-----
256 Function STBDtCash()
257
258 Local      aDtHr      := Array(2)
259
260 aDtHr[1]    := Date()

```

```

261 aDthR[2] := Time()
262
263 Return aDthR
264
265
266 //-----
267 /*{Protheus.doc} STBOpenLX
268 Emite leitura X na abertura do caixa
269 segundo a regra de negocios
270
271 @param
272 @return lRet Retorno
273 @obs
274 @sample
275 */
276 //-----
277 Function STBOpenLX()
278
279 Local lRet := .T. // Retorno
280 Local dDtLastMov := STDGtLstMov() // Pega data do ultimo fechamento antes de abrir
281 Local lEmitNFCe := LjEmitNFCe() //Indica a utilizacao da NFC-e
282
283 // Verifica Legislacao se e necessario a Emissao
284 // de uma Leitura X na primeira abertura do Caixa no dia
285 If !lEmitNFCe .AND. LJAnalisaLeg(42)[1]
286     // Se a data do ultimo fechamento for menor que a data atual emite leitura X
287     If dDtLastMov < dDatabase .and. STDEmitLeitX() // Verifica se já imprimiu uma vez a Leitura X
288
289         LJMsgLeg( LJAnalisaLeg(42) )
290         // Inicia Evento
291         STFFireEvent(ProcName(0), "STReadingX", {} ) // Leitura X
292
293     EndIf
294
295 EndIf
296
297 Return lRet
298
299 //-----
300 /*{Protheus.doc} STDGtFstMov
301 Verifica se a leitura x ja foi impressa quando a tela de venda
302 não é fechado de um dia para outro.
303 @return lRet
304 @obs
305 @sample
306 */
307 //-----
308
309 Static Function STDEmitLeitX()
310 Local aSLWArea := {}
311 Local lRet := .T.
312 Local cPdv := PadR(AllTrim(LjGetStation("PDV")),TamSX3("LG_PDV"))[1])

```

```
313
314 DBSelectArea("SLW")
315 aSLWArea := SLW->(GetArea())
316 SLW->(DBSetOrder(2))
317
318 lRet := !SLW->(DbSeek(xFilial("SLW")+cPdv+DToS(dDataBase)))
319
320 RestArea(aSLWArea)
321
322 Return lRet
```

Componentes/Cashier/STDCashName.prw

21 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 //-----
5 /*/{Protheus.doc} STDCashName
6 Retorna o nome do caixa
7
8 @return  cCashName      Retorna o codigo do caixa
9 @obs
10 @sample
11 /*/
12 //-----
13 Function STDCashName()
14 Local lPDVFirst      := FindFunction("STFCfgIntegration") .AND. (STFCfgIntegration() == "FIRST")//Pdv Mobile
15 Local cCashName      := Upper(cUsername)                // Retorna se todos os itens são de reserva
16
17 If lPDVFirst .and. !Empty(STFGetStat("CODIGO"))
18     cCashName := "CAIXA_PDV_" + STFGetStat("CODIGO") //o nome do usuário deverá ser gravado no cadastro de estacao ou parametro
19 EndIf
20
21 Return cCashName
```

Componentes/Cashier/STDCloseCash.prw

289 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static cLastNumMov := ""
5 Static aDtConf     := {}
6
7 //-----
8 /*{Protheus.doc} STDCloseCash
9 Fecha o caixa
10
11 @param cCash      Caixa
12 @param dDtClose   Data de fechamento
13 @param cHrClose   Hora de fechamento
14
15 @return lRet Retorna se fechou o caixa
16 @obs
17 @sample
18 */
19 //-----
20 Function STDCloseCash( cCash      , dDtClose   , cHrClose )
21
22 Local aArea      := GetArea()           // Guarda área
23 Local lRet       := .F.                 // Retorno
24 Local lCashName := FindFunction("STDCashName") // Ver se a função STDCashName existe no rpo
25 Local cNomeCaixa := Upper(cUsername)    // Usuario
26
27
28 Default cCash      := ""                // Caixa
29 Default dDtClose   := CtoD(" / / ")    // Data do fechamento
30 Default cHrClose   := " : "            // Hora do fechamento
31
32
33 ParamType 0 Var      cCash      As Character      Default ""
34 ParamType 1 Var      dDtClose   As Date           Default CtoD(" / / ")
35 ParamType 2 var      cHrClose   As Character      Default " : "
36
37
38 If lCashName //inicio
39     cNomeCaixa := STDCashName()
40 EndIf
41
42
43 DbSelectArea("SA6")
44 DbSetOrder(2)//A6_FILIAL+A6_NOME
45 If DbSeek( xFilial("SA6") + Upper(cNomeCaixa) ) //final
46
47     RecLock("SA6",.F.)
48
49     SA6->A6_DATAABR := CtoD(" / / ")
50     SA6->A6_HORAABR := " : "
51     SA6->A6_DATAFCH := dDtClose
52     SA6->A6_HORAFCH := cHrClose

```

```

53
54     MsUnLock()
55
56     lRet := .T.
57
58 EndIf
59
60 RestArea(aArea) // Restaura area
61
62 Return lRet
63
64 //-----
65 /*/{Protheus.doc} STDCloseMult
66 Fecha caixas associados (tratamento multi-moeda)
67
68 @param cCash          Caixa
69 @param dDtClose       Data de fechamento
70 @param cHrClose       Hora de fechamento
71
72 @return lRet Retorna se fechou os caixas
73 @obs
74 @sample
75 */
76 //-----
77 Function STDCloseMult( cCash      , dDtClose      , cHrClose      )
78
79 Local aArea          := GetArea()                // Guarda area
80 Local lRet           := .F.                      // Retorno
81 Local cCodigo        := ""                      // Codigo do caixa principal
82 Local cAgencia       := ""                      // Agencia do caixa principal
83 Local cConta         := ""                      // Conta do caixa principal
84
85 Default cCash        := ""                      // Caixa
86 Default dDtClose     := CtoD(" / / ")          // Data de fechamento
87 Default cHrClose     := " : "                  // Hora de fechamento
88
89
90 ParamType 0 Var      cCash          As Character      Default ""
91 ParamType 1 Var      dDtClose       As Date          Default CtoD(" / / ")
92 ParamType 2 var      cHrClose       As Character      Default " : "
93
94 // Fecha caixas associados(tratamento multi-moeda)
95 xNumCaixa()
96 cCodigo      := SA6->A6_COD
97 cAgencia     := SA6->A6_AGENCIA
98 cConta      := SA6->A6_NUMCON
99
100 DbSelectArea("SA6")
101 DbSetOrder(1) //A6_FILIAL+A6_COD+A6_AGENCIA+A6_NUMCON
102 If DbSeek( xFilial("SA6") + Upper(cCash) )
103
104     While !Eof() .And. ( xFilial()+cCodigo ) == ( SA6->A6_FILIAL+SA6->A6_COD )

```

```

105
106     If AllTrim(cCodigo + cAgencia + cConta) == AllTrim(SA6->A6_COD + SA6->A6_AGENCIA + SA6->A6_NUMCON)
107         SA6->(dbSkip())
108     Loop
109 EndIf
110
111     RecLock("SA6",.F.)
112
113     SA6->A6_DATAABR      := CtoD(" / / ")
114     SA6->A6_HORAABR      := " : "
115     SA6->A6_DATAFCH      := dDtClose
116     SA6->A6_HORAFCFCH    := cHrClose
117
118     MsUnLock()
119
120     lRet := .T.
121
122     SA6->(dbSkip())
123
124 EndDo
125
126 EndIf
127
128 RestArea(aArea) // Restaura area
129
130 Return lRet
131
132
133 //-----
134 /*{Protheus.doc} STDSaveMov
135 Grava numero do movimento e Numero do Cupom Final
136 no Fechamento do caixa
137
138 @return lRet   Retorna se realizou a gravacao
139 @obs
140 @sample
141 */
142 //-----
143 Function STDCSaveMov( aDthHr , cCupomFim )
144
145 Local aArea      := GetArea()           //Guarda area
146 Local aStation   := STBInfoEst( 1, .T. ) //Informacoes da estacao // [1]-CAIXA [2]-ESTACAO [3]-SERIE [4]-PDV
147 Local lRet       := .F.                 //Retorno
148 Local cChave     := ""                  //Chave de Pesquisa
149 Local cNumMov     := AllTrim(STDNumMov()) //Numero do Movimento
150 Local lConfCaixa  := SuperGetMV( "MV_LJCONF",,.F. ) //Parametro da conferencia de caixa
151 Local cConfere    := ""                  //Verifica se deve ser gravado 1 ou 2 no LW_CONFERE
152 Local nIndice     := 1                   //Numero do indice
153 Local cNomeUs     := AllTrim(UsrRetName(__cUserID)) //Nome do usuario padrao
154
155 Default aDthHr    := STBDtCash()
156 Default cCupomFim := ""

```

```

157
158 ParamType 0 Var      aDtHr           As Array      Default      STBDtCash()
159 ParamType 1 Var      cCupomFim       As Character   Default      ""
160
161 If !lConfCaixa
162     //Pesquisar o proximo movimento em aberto que deve ser finalizado
163     cChave := xFilial("SLW") + aStation[4] + aStation[1]
164     nIndice := 1 //LW_FILIAL+LW_PDV+LW_OPERADO+DTOS(LW_DTABERT)+LW_NUMMOV
165 Else
166     cChave := STDutMovAb(1,aStation[1],aStation[4],aStation[2],cNumMov,.T.)
167     nIndice := 3 //LW_FILIAL+LW_PDV+LW_OPERADO+DTOS(LW_DTABERT)+LW_ESTACAO+LW_NUMMOV
168 EndIf
169
170 If !Empty(cNumMov)
171     cLastNumMov := cNumMov
172 EndIf
173
174 DbSelectArea("SLW")
175 DbSetOrder(nIndice)
176 If DbSeek( cChave )
177
178     If !lConfCaixa
179         While !SLW->(Eof()) .AND. ( SLW->(LW_FILIAL + LW_PDV + LW_OPERADO) == cChave ) .AND. ( DtoC(SLW->LW_DTFECHA) <> " / / " )
180             SLW->(dbSkip())
181         EndDo
182     EndIf
183
184     If !SLW->(Eof())
185
186         If lConfCaixa
187             cConfere := "2"
188         Else
189             cConfere := "1"
190         EndIf
191
192         // Grava SLW
193         Reclock("SLW",.F.)
194
195         Replace SLW->LW_DTFECHA      With aDtHr[1]
196         Replace SLW->LW_HRFECHA      With aDtHr[2]
197         Replace SLW->LW_NUMFIM       With cCupomFim
198         Replace SLW->LW_SITUA        With Replicate("0",TamSX3("LW_SITUA"))[1])
199         Replace SLW->LW_TIPFECH      With "2"
200         Replace SLW->LW_CONFERE      With cConfere
201
202         If Empty(SLW->LW_ORIGEM)
203             Replace SLW->LW_ORIGEM With cModulo
204         EndIf
205
206         MsUnlock()
207
208         aDtConf := {SLW->LW_DTABERT,SLW->LW_HRABERT,SLW->LW_DTFECHA,SLW->LW_HRFECHA}

```

```

209
210         lRet := .T.
211
212         If lConfCaixa
213             STFSLICreate(aStation[2], "FCH", "2" + StrZero(SLW->(IndexOrd()), 2) + cChave, "ABANDONA", .T., cNomeUs, aDtHr[1], aDtHr[2])
214
215         EndIf
216
217     EndIf
218
219 EndIf
220 EndIf
221
222 RestArea(aArea)
223
224 Return lRet
225
226 //-----
227 /*{Protheus.doc} STDGLstNumMov
228 Fornece o ultimo numero do movimento.
229
230 @return  cLastNumMov - Código da ultima movimentacao
231 @obs
232 @sample
233 */
234 //-----
235 Function STDGLstNumMov( )
236
237 Return cLastNumMov
238
239 //-----
240 /*{Protheus.doc} STDGDtAbFch
241 Retorna um array com as informacoes de data e hora de abertura e fechamento.
242
243 @return  aDtConf[1] - Data de Abertura;
244          aDtConf[2] - Hora de Abertura;
245          aDtConf[3] - Data de Fechamento;
246          aDtConf[4] - Hora de Fechamento
247 @obs
248 @sample
249 */
250 //-----
251 Function STDGDtAbFch( )
252
253 Return aDtConf
254
255 //-----
256 /*{Protheus.doc} STDDtAbCx
257 Retorna informacao de aberta e fechamento de caixa
258
259 @return  aDtConf[1] - Data de Abertura;
260          aDtConf[2] - Hora de Abertura;

```

```

261      aDtConf[3] - Data de Fechamento;
262      aDtConf[4] - Hora de Fechamento
263 @obs
264 @sample
265 */
266 //-----
267 Function STDDtAbCx()
268
269 Local aArea      := GetArea()           //Guarda area
270 Local aStation   := STBInfoEst( 1, .T. ) //Informacoes da estacao // [1]-CAIXA [2]-ESTACAO [3]-SERIE [4]-PDV
271 Local cChave     := ""                 //Chave de Pesquisa
272 Local cNumMov    := AllTrim(STDNumMov()) //Numero do Movimento
273 Local nIndice    := 1                 //Numero do indice
274 Local aDtConf    := {}
275
276 cChave := STDutMovAb(1,aStation[1],aStation[4],aStation[2],cNumMov,.T.)
277 nIndice := 3 //LW_FILIAL+LW_PDV+LW_OPERADO+DTOS(LW_DTABERT)+LW_ESTACAO+LW_NUMMOV
278
279 DbSelectArea("SLW")
280 DbSetOrder(nIndice)
281 If DbSeek( cChave )
282     aDtConf := {SLW->LW_DTABERT,SLW->LW_HRABERT,SLW->LW_DTFECHA,SLW->LW_HRFECHA}
283 Else
284     aDtConf := Array(4)
285 EndIf
286
287 RestArea(aArea)
288
289 Return aDtConf

```

Componentes/Cashier/STDConfCash.prw

1060 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDCONF CASH.CH"
4
5 Static lDetADMf      := SuperGetMV("MV_LJDESM",.T.,.F.) //Utiliza detalhamento por administradora financeira
6 Static dDtMov        := ""                               //Data de movimento do caixa
7
8 #DEFINE POS_FP        1
9 #DEFINE POS_DESCFP    2
10 #DEFINE POS_CODADM    3
11 #DEFINE POS_QTDE      5
12 #DEFINE POS_MOEDA     6
13 #DEFINE POS_VALDIG    7
14 #DEFINE POS_VALAPU    8
15
16
17 //-----
18 /*{Protheus.doc} STDUtMovAb
19 Funcao para retornar a chave do primeiro movimento pend. fechamento
20 encontrado no controle de movimento de abertura e fecha. de caixa.
21
22 @param  n0pc          - Numero de operacao 1 ou 2
23 @param  c0per         - Numero do caixa
24 @param  cPDV          - Numero do PDV
25 @param  cEstacao      - Numero da estacao
26 @param  cNumMov       - Numero da movimentacao
27 @return  cChave       - Retorna a chave do movimento pendente
28 @obs
29 @sample
30 */
31 //-----
32 Function STDUtMovAb(   n0pc      ,c0per,cPDV,cEstacao,;
33                      cNumMov)
34
35 Local cChave          := ""                //Chave
36 Local cTab            := "SLW"            //Tabela a ser processada
37 Local aEstru          := {}               //Estrutura de tabela
38 Local cArqInd         := ""               //Arquivo para o indice temporario
39 Local cIndice         := ""               //Campos do indice
40 Local cFiltro         := ""               //Filtro do novo indice
41 Local nIndice         := 0                //Sequencia dos indices
42 Local aAreaSLW        := SLW->(GetArea()) //Area SLW
43 Local nIndAnt         := SLW->(IndexOrd()) //Indice atual
44
45 Default n0pc          := 0
46 Default c0per         := ""
47 Default cPDV          := ""
48 Default cEstacao      := ""
49 Default cNumMov       := ""
50
51
52 ParamType 0 Var      n0pc                As Numeric          Default 0

```

```

53 ParamType 1 var      cOper          As Character      Default ""
54 ParamType 2 var      cPDV           As Character      Default ""
55 ParamType 3 var      cEstacao        As Character      Default ""
56 ParamType 4 var      cNumMov         As Character      Default ""
57
58 //Determinando a estrutura
59 dbSelectArea(cTab)
60 aEstru := (cTab)->(dbStruct())
61
62 //Formatar variaveis
63 cOper      := PadR(cOper      ,TamSX3("LW_OPERADO")[1])
64 cPDV       := PadR(cPDV       ,TamSX3("LW_PDV")[1])
65 cEstacao   := PadR(cEstacao   ,TamSX3("LW_ESTACAO")[1])
66 cNumMov    := PadR(cNumMov    ,TamSX3("LW_NUMMOV")[1])
67
68 Do Case
69
70 //Pesquisa completa - busca dados da abertura vigente, em processo de fechamento
71 Case nOpc == 1
72     If Empty(cOper) .OR. Empty(cPDV) .OR. Empty(cEstacao) .OR. Empty(cNumMov)
73         Return cChave
74     Endif
75     cArqInd := CriaTrab(,.F.)
76     cIndice := "LW_FILIAL+DTOS(LW_DTFECHA)+LW_TIPFECH+LW_OPERADO+LW_PDV+LW_ESTACAO+LW_NUMMOV"
77     cFiltro := "LW_FILIAL = '" + xFilial(cTab) + "' .AND. "
78     cFiltro += "!LW_TIPFECH $ '2|3|4|5|6' .AND. "
79     cFiltro += "LW_OPERADO = '" + RTrim(cOper) + "' .AND. "
80     cFiltro += "LW_PDV = '" + RTrim(cPDV) + "' .AND. "
81     cFiltro += "LW_ESTACAO = '" + RTrim(cEstacao) + "' .AND. "
82     cFiltro += "LW_NUMMOV = '" + RTrim(cNumMov) + "' "
83     cFiltro += ".AND. LW_ORIGEM <> 'LOJ' "
84
85 //Pesquisa incompleta - busca movimentos jah fechados, por simplificados
86 Case nOpc == 2
87     If Empty(cOper)
88         Return cChave
89     Endif
90     cArqInd := CriaTrab(,.F.)
91     cIndice := "LW_FILIAL+DTOS(LW_DTFECHA)+LW_TIPFECH+LW_OPERADO+LW_PDV+LW_ESTACAO+LW_NUMMOV"
92     cFiltro := "LW_FILIAL = '" + xFilial(cTab) + "' "
93     cFiltro += ".AND. !LW_TIPFECH $ '2|3|4|5|6' "
94     cFiltro += ".AND. LW_OPERADO = '" + RTrim(cOper) + "' "
95     If !Empty(cPDV)
96         cFiltro += ".AND. LW_PDV = '" + RTrim(cPDV) + "' "
97     Endif
98     If !Empty(cEstacao)
99         cFiltro += ".AND. LW_ESTACAO = '" + RTrim(cEstacao) + "' "
100    Else
101        //0 campo LW_ESTACAO vazio caracteriza registros anteriores ao update, desconsiderar
102        cFiltro += ".AND. !Empty(LW_ESTACAO) "
103    Endif
104    If !Empty(cNumMov)

```

```

105         cFiltro += ".AND. LW_NUMMOV = '" + RTrim(cNumMov) + "' "
106     Endif
107     cFiltro += ".AND. LW_ORIGEM <> 'LOJ' "
108
109     OtherWise
110         Return cChave
111
112 EndCase
113
114 IndRegua("SLW",cArqInd,cIndice,,cFiltro,"")
115 dbSelectArea("SLW")
116 nIndex := RetIndex("SLW")
117 #IFDEF TOP
118     SLW->(dbSetIndex(cArqInd + OrdBagExt()))
119 #ENDIF
120 SLW->(dbSetOrder(nIndex + 1))
121 SLW->(dbGoTop())
122 Do While !SLW->(Eof())
123     //Se o tipo de fechamento nao for simplificado ou completo, e a
124     //data de fechamento nao estiver em branco, desconsiderar
125     If !SLW->LW_TIPFECH $ "1|2" .AND. !Empty(SLW->LW_DTFECHA)
126         SLW->(dbSkip())
127         Loop
128     Else
129         dDtMov := SLW->LW_DTABERT
130         cChave := SLW->(LW_FILIAL + LW_PDV + LW_OPERADO + DtoS(LW_DTABERT) + LW_ESTACAO + LW_NUMMOV)
131         Exit
132     EndIf
133 EndDo
134
135 //Retornar ao indice anterior e eliminar o arquivo de indice
136
137
138 SLW->(DBCcloseArea())
139 dbSelectArea("SLW")
140 RetIndex("SLW")
141 SET FILTER TO
142 SLW->(dbSetOrder(nIndAnt))
143
144 If File(cArqInd + OrdBagExt())
145     fErase(cArqInd + OrdBagExt())
146 Endif
147
148 RestArea(aAreaSLW)
149
150 Return cChave
151
152
153 //-----
154 /*{Protheus.doc} STDCreatGd
155 Funcao que retorna os dados da SLT para se realizar a conferencia.
156

```

```

157 @param
158 @return      aNumerario[x,1] - Sigla da Forma de Pagamento;
159              aNumerario[x,2] - Descricao da Forma de Pagamento;
160              aNumerario[x,3] - Codigo da Administradora Financeira;
161              aNumerario[x,4] - ???
162              aNumerario[x,5] - Quantidade Utilizada da forma de pagamento.
163              aNumerario[x,6] - Moeda utilizada
164              aNumerario[x,7] - Valor informado pelo usuario
165              aNumerario[x,8] - Valor apurado pelo sistema
166 @obs
167 @sample
168 /*/
169 //-----
170 Function STDCreatGd()
171
172 Local aArea      := GetArea()           //Retrato da workarea
173 Local lMoedaEst  := .F.                 //Utiliza moeda estrangeira
174 Local aFPADM     := {}                  //Forma de pagamento das administradoras financeiras
175 Local lConfCega  := SuperGetMV("MV_LJEXAPU",.T.,.F.) == .F. //Utiliza a conferencia cega?
176 Local aNumerario := {}                 //1. Forma de pagamento 2. Descricao da FP 3. Quantidade 4. Moeda 5. Valor a ser apontado
177 Local aID        := STBInfoEst(1,.F.,.T.) //[[1]-CAIXA [2]-ESTACAO [3]-SERIE [4]-PDV [5]-SERIE NAO FISCAL + TAMANHO DE CADA CAMPO
178 Local cNumMov    := STDNumMov()         //Numero da movimentacao
179 Local lApura      := IsInCallStack("STDGRVSLT") //Apurar o valor das vendas quando for gerar a tabela SLT quando estiver fechamento a
180 Local dMovimento := POSICIONE("SA6",1,XFILIAL("SA6")+xNumCaixa(),"A6_DATAABR") //Data do movimento aberto
181 Local cFiltro     := ""                 //Armazena o filtro a ser aplicado na tabela
182 Local aFechaCx    := STDDtAbCx()       //Pega os dados de abertura do caixa Data/Hora
183
184 If Len(aFechaCx) > 0 .AND. !Empty(aFechaCx[1])
185     dMovimento := aFechaCx[1] //Pega a data de abertura da tabela SLW
186 Else
187     LjGrvLog("Conferencia de Caixa","Nao sera possivel efeturar a conferencia de caixa pois nao retornou a data de abertura da SLW - LW_DTABERT")
188 EndIf
189
190 /* Carrega as adm financeiras */
191 aFPADM := STDLoadAdm()
192
193 /* Carrega as formas de pagamento */
194 aNumerario := STDFormsPay(aFPADM)
195
196 //Filtro a ser aplicado na tabela
197 cFiltro := "L1 FILIAL = '" + xFilial("SL1") + "' .AND. L1_OPERADO = '" + PadR(aID[1][1],aID[1][2]) + "' .AND. DToS(L1_EMISSAO) >='" + DtoS(dMovimento) + "' .AND.
198     DToS(L1_EMISSAO) <= '" + DToS(dDataBase) + "'"
199
200 DbSelectArea("SL1")
201 SL1->(DbSetFilter({}|&cFiltro},cFiltro))
202 SL1->(DbGoTop())
203
204 //Caso nao seja conferencia cega, levantar os valores apurados, senao apenas as quantidades
205 Do While !SL1->(Eof())

```

```

205
206   If (AllTrim(SL1->L1_SERIE) # AllTrim(aID[3][1]) .AND. AllTrim(SL1->L1_SERPED) # AllTrim(aID[5][1]) .AND. Empty(SL1->L1_DOCRPS)) .OR.;
207   (AllTrim(SL1->L1_PDV) # AllTrim(aID[4][1]) .OR. AllTrim(SL1->L1_ESTACAO) # AllTrim(aID[2][1])) .OR.;
208   (SL1->L1_SITUA == "07" .OR. (!Empty(cNumMov) .AND. AllTrim(SL1->L1_NUMMOV) # AllTrim(cNumMov))) .OR.;
209   (SL1->L1_STORC $ "C|E")
210
211   SL1->(dbSkip())
212   Loop
213
214   Else
215
216     If SL1->L1_CREDITO > 0
217       aNumerario := STDCredVal(aNumerario)
218     EndIf
219
220     /* Totalizar as formas de pagamento */
221     aNumerario := STDTotPay(aNumerario, @lMoedaEst)
222
223   EndIf
224
225   SL1->(dbSkip())
226
227 EndDo
228
229 SL1->(dbClearFilter())
230
231 //Totalizadores de Movimentação não venda (Recebimento/Sangria/Troco e etc)
232 aNumerario := STDConfMov({xFilial("SE5"),DtoS(dMovimento),PadR(aID[1][1],aID[1][2])},aNumerario, cNumMov )
233
234 /* Ordenar a sequencia por forma de PG e moeda */
235 aNumerario := STDOrderPay(aNumerario,lMoedaEst)
236
237 RestArea(aArea)
238
239 Return aNumerario
240
241
242 //-----
243 /*{Protheus.doc} STDGrvSLT
244 Apos realizar a conferencia, gravar a SLT
245
246 @param
247 @return      Nil
248 @obs
249 @sample
250 */
251 //-----
252 Function STDGrvSLT(aDados,dDataMov,lFechaSimp)
253
254 Local lRet           := .T.           //Retorno da funcao
255 Local lMoedaC        := .F.           //Moeda corrente
256 Local cFP            := ""            //Forma de pagamento

```

```

257 Local cAgencia      := "" //Agencia
258 Local cChave        := "" //Chave
259 Local nTamFP         := TamSX3("LT_FORMPG")[1] //Tamanho do campo da forma de pagamento
260 Local nTamMV         := TamSX3("LT_NUMMOV")[1] //Tamanho do campo de movimento
261 Local cCampos        := "" //Campos do indice utilizado, para comparacoes
262 Local nI             := 0 //Variavel de Loop
263 Local aLstCxAb       := {} //Lista de caixas abertos - Agencias (moedas)
264 Local cAgenC         := PadR(".",TamSX3("A6_AGENCIA")[1]) //Codigo da agencia corrente
265 Local aCaixaC        := Array(5) //Caixa na moeda corrente - 1.Filial 2.Codigo 3.Agencia 4.Dt Abertura 5.Aberto?
266 Local lUsaFecha      := SuperGetMV("MV_LJCONF",.T.,.F.) //Usa conferencia de cx
267 Local lNovo          := .F. //Utilizada no ReckLok para um novo registro ou apenas atualizar
268 Local aDataHr        := STDGDtAbFch( ) //Array com data e hora
269 Local aID            := STBInfoEst(1,.F.,.T.) //[1]-CAIXA [2]-ESTACAO [3]-SERIE [4]-PDV + TAMANHO DE CADA CAMPO CORRESPONDENTE
270 Local lConfCega      := SuperGetMV("MV_LJEXAPU",.T.,.F.) == .F. //Utiliza a conferencia cega?
271 Local lExistGNum     := ExistFunc("STIGetNume")
272 Local aNumerario     := Iif(lExistGNum,STIGetNume(),{}) //Preenche os numerarios com as forma de pagamento
273 Local nModoChama     := 2 //Compatibilidade 'a conferencia de caixa do FrontLoja, vide FRTA272B ( 1 = LOJA260, 2 = FRTA271 )
274 Local nVlrTot        := 0 //Valor total acumulado por fechamento
275 Local nQtdTot        := 0 //Qtde total de vendas por fechamento
276 Local nPosANum       := 0 //Posição do numerario resgatado pelo aScan
277
278 Default aDados       := {}
279 Default dDataMov      := CtoD(" / / ")
280 Default lFechaSimp    := .F.
281
282 ParamType 0 Var aDados As Array Default {}
283 ParamType 1 Var dDataMov As Date Default CtoD(" / / ")
284 ParamType 2 Var lFechaSimp As Logical Default .F.
285
286 //Pesquisa caixa na moeda principal
287 dbSelectArea("SA6")
288 SA6->(dbSetOrder(1))//A6_FILIAL+A6_COD+A6_AGENCIA+A6_NUMCON
289 SA6->(dbSeek(xFilial("SA6") + PadR(aID[1][1],aID[1][2]) + cAgenC))
290 If !SA6->(Found())
291     LjGrvLog("Conferencia de Caixa","Caixa não encontrado na tabela SA6, verifique as informações de: Agencia e Conta.")
292     LjGrvLog("Conferencia de Caixa","Tabela SLT não sera gravada.")
293     lRet := .F.
294 Else
295     aCaixaC[1] := SA6->A6_FILIAL
296     aCaixaC[2] := SA6->A6_COD
297     aCaixaC[3] := SA6->A6_AGENCIA
298     aCaixaC[4] := SA6->A6_DATAABR
299     aCaixaC[5] := IIf(Empty(SA6->A6_DATAABR),.F.,.T.)
300 EndIf
301
302 If lRet
303     //Pesquisar se o movimento do caixa foi aberto em mais de
304     //uma moeda (localizadas). Para criar o array de moedas
305     //utilizadas no processo de abertura de caixa.
306     Do While !SA6->(Eof()) .AND. SA6->(A6_FILIAL + A6_COD) == (aCaixaC[1] + aCaixaC[2])
307         //Se for a agencia principal, saltar
308         If SA6->A6_AGENCIA == aCaixaC[3]

```

```

309         SA6->(dbSkip())
310         Loop
311     EndIf
312     //Exigir que as datas de abertura sejam as mesmas para que as moedas nao
313     //correntes sejam consideradas (caso o caixa esteja aberto)
314     If nModoChama == 2 .AND. aCaixaC[5]
315         If SA6->A6_DATAABR == aCaixaC[4] .AND. IsMoney(SA6->A6_AGENCIA)
316             aAdd(aLstCxAb, AllTrim(SA6->A6_AGENCIA))
317         EndIf
318     Else
319         If IsMoney(SA6->A6_AGENCIA)
320             aAdd(aLstCxAb, SA6->A6_AGENCIA)
321         EndIf
322     EndIf
323 EndDo
324
325 //Gravar a conferencia SLT
326 dbSelectArea("SLT")
327 SLT->(dbSetOrder(4)) //LT_FILIAL+LT_OPERADO+DTOS(LT_DTFECHA)+LT_FORMPG+LT_PDV+LT_NUMMOV+LT_ADMIFIN
328 For nI := 1 to Len(aDados)
329     If STFGetCfg("lMultCoin")
330         lMoedaC := IIf(aScan(aLstCxAb, {|x| AllTrim(x) == AllTrim(aDados[nI][POS_MOEDA])}) == 0, .T., .F.)
331
332         If lMoedaC
333             cFP      := aDados[nI][1]
334             cAgencia := cAgenC
335         Else
336             cFP      := aDados[nI][1] + cValToChar(RetMoeda(aDados[nI][POS_MOEDA]))
337             cAgencia := PadR(AllTrim(aDados[nI][POS_MOEDA]), TamSX3("A6_AGENCIA"))[1])
338         EndIf
339     Else
340         lMoedaC := .T.
341         cFP      := aDados[nI][1]
342         cAgencia := cAgenC
343     EndIf
344
345     If lExistGNum
346         If lDetADMF
347             nPosANum := aScan(aNumerario, {|x| x[POS_FP] == aDados[nI][POS_FP] .And. x[POS_CODADM] == aDados[nI][POS_CODADM] })
348         Else
349             nPosANum := aScan(aNumerario, {|x| x[POS_FP] == aDados[nI][POS_FP] })
350         EndIf
351     Else
352         LjGrvLog("Gravação da Tabela SLT", "0 Campo LT_VLRAPU Não será gravado, atualizar o fonte STIConfCash com data superior a 08/11/2018" )
353     EndIf
354
355     RecLock("SLT", .T.)
356     SLT->LT_FILIAL      := xFilial("SLT")
357     SLT->LT_OPERADO     := aID[1][1]
358     SLT->LT_DTFECHA     := aDataHr[3]
359     SLT->LT_FORMPG      := cFP
360     If !lFechaSimp

```

```

361         SLT->LT_VLRDIG := Val(aDados[nI][POS_VALDIG])
362     Else
363         SLT->LT_VLRDIG := 0
364     Endif
365     SLT->LT_SANPAR := 0
366     SLT->LT_AGENCIA := cAgencia
367     SLT->LT_NUMMOV := STDGLstNumMov()
368     SLT->LT_DTMOV := aDataHr[1] //Data de abertura do movimento.
369     SLT->LT_MOEDA := STBGetCurrency()
370     SLT->LT_ESTACAO := aID[2][1]
371     SLT->LT_PDV := aID[4][1]
372     If !lFechaSimp
373         SLT->LT_CONFERE := "1"
374     Else
375         SLT->LT_CONFERE := "2"
376     Endif
377     If lDetADMF
378         SLT->LT_ADMIFIN := aDados[nI][POS_CODADM]
379     Endif
380     SLT->LT_QTDE := aDados[nI][POS_QTDE]
381
382     If nPosANum > 0
383         SLT->LT_VLRAPU := aNumerario[nPosANum][POS_VALAPU]
384         nQtdTot += aNumerario[nPosANum][POS_QTDE]
385         nVlrTot += aNumerario[nPosANum][POS_VALAPU]
386         //Quarda chave para alterar Tabela SLW
387         cChave := SLT->LT_PDV + SLT->LT_OPERADO + DTOS(SLT->LT_DTFECHA) + SLT->LT_ESTACAO + SLT->LT_NUMMOV
388     Endif
389     SLT->LT_SITUA := "00"
390
391     MsUnlock()
392
393 Next nI
394
395
396 //Grava Valor e quantidade de venda
397 //para posterior comparacao na Retaguarda
398 STDGrvVlrQtd( xFilial("SLW")+cChave , nQtdTot , nVlrTot )
399
400 Endif
401
402 Return Nil
403
404
405 //-----
406 /*{Protheus.doc} STDGrvVlrQtd
407 Grava quantidade e valores de vendas na SLW
408 @param cChave Chave de pesquisa
409 @param nQtdTot Quantidade total de vendas
410 @param nVlrTot Valor total de vendas
411 @return lSet - Retorna se o campo foi gravado ou não
412 @obs

```

```

413 @sample
414 /*/
415 //-----
416 Static Function STDGrvVlrQtd( cChave , nQtdTot , nVlrTot )
417
418 Local aArea      := GetArea()    //Salva area
419 Local lRet       := .F.         //Retorno da função
420 Local lContinua  := .F.         //Continua Fluxo
421
422 Default cChave   := ""          // Chave de pesquisa
423 Default nQtdtot  := 0           // Quantidade total de vendas
424 Default nVlrTot  := 0           // Valor total de vendas
425
426 If SLW->(ColumnPos("LW_QTDTOT")) > 0 .AND. SLW->(ColumnPos("LW_VLRTOT")) > 0
427     lContinua := .T.
428 EndIf
429
430 DbSelectArea("SLW")
431 DbSetOrder(3) //LW_FILIAL+LW_PDV+LW_OPERADO+DTOS(LW_DTABERT)+LW_ESTACAO+LW_NUMMOV
432 If lContinua .AND. DbSeek( cChave )
433
434     If RecLock("SLW", .F.)
435
436         SLW->LW_QTDTOT := nQtdTot
437         SLW->LW_VLRTOT := nVlrTot
438
439         SLW->(MsUnLock())
440         lRet := .T.
441
442     EndIf
443
444 EndIf
445
446 RestArea(aArea)
447
448 Return lRet
449
450
451 //-----
452 /*/{Protheus.doc} STDFormsPay
453 Carrega as formas de pagamento
454
455 @param aFPADM Forma de Pgto
456 @return aNumerario - Numerarios
457 @obs
458 @sample
459 /*/
460 //-----
461 Function STDFormsPay( aFPADM )
462
463 Local cChave      := xFilial("SX5") + "24"    // Chave
464 Local aTMP        := {}                      // Array Temp

```

```

465 Local aNumerario      := {}           // Numerarios
466 Local nI               := 0           // Contador
467
468 Default aFPADM          := {}
469
470 ParamType 0 Var aFPADM   As Array     Default {}
471
472 dbSelectArea("SX5")
473 SX5->(dbSetOrder(1))//X5_FILIAL+X5_TABELA+X5_CHAVE
474 SX5->(dbSeek(cChave))
475
476 If SX5->(Found())
477     Do While !SX5->(Eof()) .AND. RTrim(SX5->(X5_FILIAL + X5_TABELA)) == cChave
478         aTMP := {}
479         If !SX5->(Deleted())
480             If !lDetADMF
481
482                 aAdd(aNumerario,{SX5->X5_CHAVE,SX5->X5_DESCRI,""," ",0,1,0,0})
483
484             Else
485
486                 /* Levantar todas as adm financeiras associadas com esta forma de pagamento */
487                 For nI := 1 to Len(aFPADM)
488                     If AllTrim(aFPADM[nI][1]) == AllTrim(SX5->X5_CHAVE)
489                         aAdd(aTMP,{aFPADM[nI][2],aFPADM[nI][3]})
490                     EndIf
491                 Next nI
492
493                 /* Listar todas as administradoras financeiras na lista de numerarios */
494                 If Len(aTMP) == 0
495                     aAdd(aNumerario,{SX5->X5_CHAVE,SX5->X5_DESCRI,""," ",0,1,0,0})
496                 Else
497                     For nI := 1 to Len(aTMP)
498                         aAdd(aNumerario,{SX5->X5_CHAVE,aTMP[nI][2],aTMP[nI][1],aTMP[nI][2],0,1,0,0})
499                     Next nI
500                 EndIf
501             EndIf
502         EndIf
503     EndIf
504     SX5->(dbSkip())
505 EndDo
506 EndIf
507
508 Return aNumerario
509
510
511 //-----
512 /*/{Protheus.doc} STDOrderPay
513 Ordenar a sequencia por forma de PG e moeda
514
515 @param aNumerario - Forma de pagamento
516 @param lMoedaEst - Moeda corrente

```

```

517 @return      aNumerario - Retorna as formas de pagamento ja ordenadas.
518 @obs
519 @sample
520 /*/
521 //-----
522 Function STDOrderPay(aNumerario,lMoedaEst)
523
524 Default aNumerario      := {}
525 Default lMoedaEst       := .F.
526
527 ParamType 0 Var aNumerario      As Array      Default {}
528 ParamType 1 Var lMoedaEst       As Logical    Default .F.
529
530 If Len(aNumerario) > 0
531
532     If lMoedaEst .AND. !lDetADMF
533         aNumerario := aSort(aNumerario,,,{|x,y| (x[POS_FP] + x[POS_MOEDA]) < (y[POS_FP] + y[POS_MOEDA])})
534     ElseIf !lMoedaEst .AND. lDetADMF
535         aNumerario := aSort(aNumerario,,,{|x,y| (x[POS_FP] + x[POS_CODADM]) < (y[POS_FP] + y[POS_CODADM])})
536     ElseIf lMoedaEst .AND. lDetADMF
537         aNumerario := aSort(aNumerario,,,{|x,y| (x[POS_FP] + x[POS_MOEDA] + x[POS_CODADM]) < (y[POS_FP] + y[POS_MOEDA] + y[POS_CODADM])})
538     EndIf
539
540 EndIf
541
542 Return aNumerario
543
544
545 //-----
546 /*/{Protheus.doc} STDCredVal
547 Trata o valor do credito da venda.
548
549 @param aNumerario Numerario
550 @return aNumerario Numerario
551 @obs
552 @sample
553 /*/
554 //-----
555 Function STDCredVal(aNumerario)
556
557 Local nValFP      := 0      //Valor por forma de pagamento
558 Local nPos        := 0      //Posicionador
559 Local nPos02      := 0      //Posicionador
560
561 Default aNumerario      := {}
562
563 ParamType 0 Var aNumerario      As Array      Default {}
564
565 nValFP := SL1->L1_CREDITO
566
567 If SL1->L1_MOEDA > 1 .AND. !STFGetCfg("lMultCoin")
568     nValFP := xMoeda( nValFP, SL1->L1_MOEDA, 1, SL1->L1_EMISSAO )

```

```

569 EndIf
570
571 If (nPos := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == "CR"})) > 0
572   If STFCfg("lMultCoin")
573     /* Caso o movimento esteja na moeda corrente, apenas totalizar */
574     If SL1->L1_MOEDA == 0 .OR. SL1->L1_MOEDA == 1
575       aNumerario[nPos][POS_QTDE] += 1
576       aNumerario[nPos][POS_VALAPU] += nValFP
577     Else
578       /*Verificar se a moeda associada a forma de pagamento esta na array, caso nao esteja, incluir
579       Aplicar um filtro para verificar a existencia da agencia (moeda) na forma de pagamento associada*/
580       nPos02 := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == "CR" .AND. x[POS_MOEDA] == SL1->L1_MOEDA})
581       If nPos02 == 0
582         aAdd(aNumerario,{"CR", STDDescFP("CR"), 0, SL1->L1_MOEDA, 0, 0, "", ""})
583         nPos := Len(aNumerario)
584       Else
585         nPos := nPos02
586       EndIf
587       aNumerario[nPos][POS_QTDE] += 1
588       aNumerario[nPos][POS_VALAPU] += nValFP
589     EndIf
590   Else
591     aNumerario[nPos][POS_QTDE] += 1
592     aNumerario[nPos][POS_VALAPU] += nValFP
593   EndIf
594 EndIf
595
596 Return aNumerario
597
598 //-----
599 /*{Protheus.doc} STDLoadAdm
600 Carrega as administradoras financeiras
601
602 @param
603 @return      aFPADM - Retorna Tipo, Codigo e Descricao da adm financeira
604 @obs
605 @sample
606 */
607 //-----
608 Function STDLoadAdm()
609
610 Local aFPADM      := {}      // Array formas de Pgto
611
612 If lDetADMF
613   dbSelectArea("SAE")
614   SAE->(dbGoTop())
615
616   Do While !SAE->(Eof())
617     If aScan(aFPADM,{|x| x[1] == AllTrim(SAE->AE_TIPO) .AND. AllTrim(x[2]) == AllTrim(SAE->AE_COD)}) == 0
618       aFPADM += {SAE->AE_TIPO, SAE->AE_COD, SAE->AE_DESC, SAE->AE_VAL}
619     EndIf
620   EndWhile
621 EndIf

```

```

621         aAdd(aFPADM,{AllTrim(SAE->AE_TIPO),SAE->AE_COD,SAE->AE_DESC})
622
623     EndIf
624
625     SAE->(dbSkip())
626 EndDo
627
628 EndIf
629
630 Return aFPADM
631
632 //-----
633 /*{Protheus.doc} STDTotPay
634 Totaliza todas as formas de pagamento que
635
636 @param  cFM - Chave
637 @return  cRet - Retorna a descricao da forma de pagamento
638 @obs
639 @sample
640 */
641 //-----
642 Function STDTotPay(aNumerario)
643
644 Local lHabTroco      := SuperGetMV("MV_LJTROCO",,.F.)    //Habilita troco
645 Local cCodADM        := ""                                //Codigo administadora financeira
646 Local nTamCADM       := TamSX3("AE_COD")[1]             //Tamanho do codigo da adm financeira
647 Local aLstFPQ        := {}                               //Variavel Provisoria
648 Local lFiltrarADMF    := .F.                             //Filtrar por administradora financeira
649 Local cTMP           := ""                               //Temporaria
650 Local nPos           := 0                                //Posicionador
651 Local nPos02         := 0                                //Posicionador
652 Local lSomaQtde       := .T.                             //Variavel de controle de soma das quantidades de vendas
653 Local lMoedaEst       := .F.                             //Utiliza moeda estrangeira
654 Local nValFP         := 0                                //Valor por forma de pagamento
655
656 Default aNumerario   := {}
657
658 ParamType 0 Var aNumerario As Array Default {}
659
660 dbSelectArea("SL4")
661 SL4->(dbSetOrder(1))//L4_FILIAL+L4_NUM+L4_ORIGEM
662 SL4->(dbSeek(xFilial("SL4") + SL1->L1_NUM + Space(TamSX3("L4_ORIGEM")[1])))
663 Do While !SL4->(Eof()) .AND. RTrim(SL4->(L4_FILIAL + L4_NUM)) == RTrim(xFilial("SL4") + SL1->L1_NUM) .AND. Empty(SL4->L4_ORIGEM)
664
665     If lHabTroco
666         nValFP := SL4->L4_VALOR
667     Else
668         nValFP := SL4->(L4_VALOR - L4_TROCO)
669     EndIf
670
671     cCodADM := Substr( SL4->L4_ADMINIS, 1, nTamCADM )
672

```

```

673  /* Se ja existe a FP no array aLstFPQ então não soma a quantidade 1 na FP */
674  If aScan(aLstFPQ,{|x| x[1] == AllTrim(SL4->L4_FILIAL) .AND. x[2] == AllTrim(SL4->L4_NUM) .AND. ;
675    x[3] == AllTrim(SL4->L4_FORMA) .AND. x[4] == AllTrim(cCodADM) .AND. x[5] == AllTrim(SL4->L4_MOEDA)}) == 0
676
677    aAdd(aLstFPQ,{AllTrim(SL4->L4_FILIAL),AllTrim(SL4->L4_NUM),AllTrim(SL4->L4_FORMA),AllTrim(cCodADM),AllTrim(SL4->L4_MOEDA)})
678    lSomaQtde := .T.
679
680  Else
681    lSomaQtde := .F.
682  EndIf
683
684  /*Verificar se o pagamento foi realizado em moeda estrangeira e necessita conversao*/
685  If SL4->L4_MOEDA > 1 .AND. !STFGetCfg("lMultCoin")
686    nValFP := xMoeda( nValFP, SL4->L4_MOEDA, 1, SL1->L1_EMISSAO )
687  EndIf
688
689  /* Fazer o posicionamento na array */
690  If !lDetADMf
691    nPos := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA)})
692  Else
693    If !Empty(cCodADM)
694      lFiltroADMf := .T.
695      If (nPos := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA) .AND. AllTrim(x[POS_CODADM]) == cCodADM})) == 0
696        /* Caso a adm financeira nao tenha sido encontrada, pesquisar apenas pela forma de pagamento */
697        nPos := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA)})
698      EndIf
699    Else
700      nPos := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA)})
701    EndIf
702  EndIf
703
704  If nPos > 0
705    If STFGetCfg("lMultCoin")
706
707      /* Caso o movimento esteja na moeda corrente, apenas totalizar */
708      If SL4->L4_MOEDA == 0 .OR. SL4->L4_MOEDA == 1
709        aNumerario[nPos][POS_QTDE] += IIf(lSomaQtde,1,0)
710        aNumerario[nPos][POS_VALAPU] += nValFP
711      Else
712        /*Verificar se a moeda associada a forma de pagamento esta na array, caso nao esteja, incluir
713        Aplicar um filtro para verificar a existencia da agencia (moeda) na forma de pagamento associada*/
714        If lFiltroADMf
715          /* Filtro : Forma Pagto + Agencia + Adm Financeira */
716          If (nPos02 := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA) .AND. x[POS_MOEDA] == SL4->L4_MOEDA .AND. ;
717            AllTrim(x[POS_CODADM]) == cCodADM})) == 0
718
719            /* Caso a adm financeira nao tenha sido encontrada, pesquisa apenas pela forma de pagamento e moeda */
720            nPos02 := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA) .AND. x[POS_MOEDA] == SL4->L4_MOEDA})
721
722          EndIf
723        Else
724          /* Filtro : Forma Pagto + Agencia */

```

```

725         nPos02 := aScan(aNumerario,{|x| AllTrim(x[POS_FP]) == AllTrim(SL4->L4_FORMA) .AND. x[POS_MOEDA] == SL4->L4_MOEDA})
726     EndIf
727
728     If nPos02 == 0
729         lMoedaEst := .T.
730         If lDetADMF .AND. !Empty(cCodADM)
731             cTMP := GetAdvFVal("SAE","AE_DESC",xFilial("SAE") + RTrim(cCodADM),1)
732         EndIf
733         If !lDetADMF
734             aAdd(aNumerario,{SL4->L4_FORMA, STDDescFP(SL4->L4_FORMA), 0, SL4->L4_MOEDA, 0, 0, cCodADM, cTMP})
735         Else
736             aAdd(aNumerario,{SL4->L4_FORMA, STDDescFP(SL4->L4_FORMA), cCodADM, cTMP, 0, SL4->L4_MOEDA, 0, 0})
737         EndIf
738         nPos := Len(aNumerario)
739     Else
740         nPos := nPos02
741     EndIf
742     aNumerario[nPos][POS_QTDE] += IIf(lSomaQtde,1,0)
743     aNumerario[nPos][POS_VALAPU] += nValFP
744 EndIf
745 Else
746     aNumerario[nPos][POS_QTDE] += IIf(lSomaQtde,1,0)
747     aNumerario[nPos][POS_VALAPU] += nValFP
748 EndIf
749 EndIf
750 SL4->(dbSkip())
751 EndDo
752
753 Return aNumerario
754
755
756 //-----
757 /*{Protheus.doc} STDDescFP
758 Retorna a descricao da forma de pagamento
759
760 @param cFM - Chave
761 @return cRet - Retorna a descricao da forma de pagamento
762 @obs
763 @sample
764 */
765 //-----
766 Function STDDescFP( cFM )
767
768 Local cRet := "" //Retorno
769
770 Default cFM := ""
771
772 ParamType 0 var      cFM          As Character          Default ""
773
774 dbSelectArea("SX5")
775 SX5->(dbSetOrder(1)) //X5_FILIAL+X5_TABELA+X5_CHAVE
776 SX5->(dbSeek(xFilial("SX5") + "24" + AllTrim(cFM)))

```

```

777 If SX5->(Found())
778     cRet := SX5->X5_DESCRI
779 EndIf
780
781 Return cRet
782
783
784 //-----
785 /*{Protheus.doc} RetMoeda
786 função statica: Retorna o código da moeda associada a um símbolo ou nome.
787 @param cStr - Símbolo ou nome da moeda a se obter o código correspondente
788 @param nOpc - Opcao
789 @param nTipoRet - Tipo de retorno 1 numérico - 2. Retorno caracter
790 @param lFmtStr - A string de retorno deve ser fommatada?
791 @return uMoeda - Código da moeda em formato numerico ou caracter
792 @obs
793 @sample
794 */
795 //-----
796 Static Function RetMoeda(cStr,nOpc,nTipoRet,lFmtStr)
797
798 Local aAreaSM2      := SM2->(GetArea())
799 Local ni            := 1
800 Local uMoeda        := 0
801 Local cAcentua      := "ÃÊÏÔÜÄËÏÖÅÊÏÖÄËÏÖÅÖ" //Acentuacoes
802 Local cAcentuaCorr  := "AEIOUAEIOUAEIOUAEIOUAEIOUAO" //Correspondentes
803 Local cTMP          := ""
804 Local cTMP02        := ""
805 Local nx            := 0
806 Local nPos          := 0
807 Local nTamMoeda     := TamSX3("L1_MOEDA")[1]
808
809 Default cStr        := ""
810 Default nOpc        := 1
811 Default nTipoRet     := 1
812 Default lFmtStr     := .T.
813
814 If ValType(cStr) # "C" .OR. ValType(nOpc) # "N" .OR. Empty(cStr) .OR. ValType(nTipoRet) # "N"
815     Return uMoeda
816 Else
817     If (nOpc < 1 .OR. nOpc > 2) .OR. (nTipoRet < 1 .OR. nTipoRet > 2)
818         Return uMoeda
819     Endif
820 EndIf
821
822 dbSelectArea("SM2")
823 Do While SM2->(FieldPos("M2_MOEDA" + cValToChar(ni))) > 0
824     Do Case
825         Case nOpc == 1
826             //Igualar caixa e remover espacos
827             cTMP := Upper(AllTrim(SuperGetMV("MV_SIMB" + cValToChar(ni),.F.,"")))
828             cTMP02 := Upper(AllTrim(cStr))

```

```

829     Case n0pc == 2
830         //Igualar caixa e remover espacos
831         cTMP := Upper(AllTrim(SuperGetMV("MV_MOEDA" + cValToChar(ni),.F.,"")))
832         cTMP02 := Upper(AllTrim(cStr))
833     Otherwise
834         Return uMoeda
835 EndCase
836 //Remover acentuacao do parametro e variavel, caso exista
837 For nx := 1 to Len(If(Len(cTMP) >= Len(cTMP02),cTMP,cTMP02))
838     If nx <= Len(cTMP)
839         If (npos := At(Substr(cTMP,nx,1),cAcentua)) > 0
840             cTMP := If(nx > 1, Substr(cTMP,1,nx - 1,"") + Substr(cAcentuaCorr,npos,1) + Right(cTMP,Len(cTMP) - nx)
841         Endif
842     Endif
843     If nx <= Len(cTMP02)
844         If (npos := At(Substr(cTMP02,nx,1),cAcentua)) > 0
845             cTMP02 := If(nx > 1, Substr(cTMP02,1,nx - 1,"") + Substr(cAcentuaCorr,npos,1) + Right(cTMP02,Len(cTMP) - nx)
846         Endif
847     Endif
848 Next ni
849 If cTMP == cTMP02
850     uMoeda := ni
851     Exit
852 Endif
853 ni++
854 EndDo
855 //Caso tenha que ser retornado como caracter
856 If nTipoRet == 2
857     If lFmtStr
858         uMoeda := StrZero(uMoeda,nTamMoeda)
859     Else
860         uMoeda := cValToChar(uMoeda)
861     Endif
862 Endif
863 RestArea(aAreaSM2)
864
865 Return uMoeda
866
867 //-----
868 /*{Protheus.doc} STDConfMov
869 Realiza a conferencia de caixa para movimentações SE5 (não vendas)
870 @param      cChave - Chave de localização das movimentações na tabela SE5
871             aNumerario - Array com os numerarios da venda
872             cNumMov - Numero do movimento atual do caixa
873 @return     aRet - Array com todos os registros do fechamento de caixa.
874 @obs
875 @sample
876 */
877 //-----
878 Function STDConfMov(aChave,aNumerario,cNumMov)
879 Local aRet      := {}
880 Local cForma    := ""

```

```

881 Local cDesForma      := ""
882 Local nValFP         := 0
883 Local cNatRece       := &(SuperGetMV("MV_NATRECE"))
884 Local nOpcConf       := SuperGetmv("MV_LJOPCON",,2) //1-Conferencia por forma e totalizador (Recebimento/Sangria/Suprimento); 2-Conferencia por forma de pagamento;
885 Local cMoedaSimb     := SuperGetMV( "MV_SIMB" + Str(STBGetCurrency(),1 ) ) // Simbolo da moeda corrente
886 Local cMvNatSang     := ""
887 Local cMvNatTrc      := ""
888 Local cfiltro        := "" //Variavel para filtro nas tabelas SE5 e MHK
889 Local cCodAdmins     := "" //Codigo da Administradora
890 Local cDescAdmins     := "" //Descrição da administradora
891
892 Default aNumerario   := {}
893 Default aChave       := {}
894 Default cNumMov      := STDGLstNumMov( )
895
896 //Natureza das movimentações:
897 If FindFunction("LjMExeParam")
898     cMvNatSang := LjMExeParam("MV_NATSANG",,"SANGRIA") //Função que trata macroexecucao
899     cMvNatTrc  := LjMExeParam("MV_NATTROC",,"TROCO")      //Função que trata macroexecucao
900 Else
901     cMvNatSang := SuperGetMv("MV_NATSANG",,"SANGRIA") //Natureza da Sangria
902     If AllTrim(cMvNatSang) == "SANGRIA"
903         cMvNatSang := "SANGRIA"
904     EndIf
905
906     cMvNatTrc := SuperGetMv("MV_NATTROC",,"TROCO") //Natureza do Troco
907     If AllTrim(cMvNatTrc) == "TROCO"
908         cMvNatTrc := "TROCO"
909     EndIf
910 EndIf
911
912 aRet := aClone(aNumerario)
913 cFiltro := "E5_FILIAL = '" + aChave[1] + "' .AND. DToS(E5_DATA) >= '" + aChave[2] + "' .AND. DToS(E5_DATA) <= '" + DToS(dDataBase) + "' " +;
914 " .AND. E5_BANCO = '" + aChave[3] + "' .AND. E5_NUMMOV = '" + cNumMov + "' .AND. AllTrim(SE5->E5_TIPODOC) <> 'VL' .AND. AllTrim(SE5->E5_TIPODOC) <> 'ES' .AND.
    AllTrim(SE5->E5_TIP0) <> 'FI' "
915
916 DbSelectArea("SE5")
917 SE5->(DbSetFilter({}|&cFiltro},cFiltro))
918 SE5->(dbGoTop())
919
920 While !SE5->(EOF())
921
922     cForma      := ""
923     cDesForma   := ""
924     nValFP      := SE5->E5_VALOR
925
926     If AllTrim(SE5->E5_TIPODOC) == "TR" .And. Upper(AllTrim(SE5->E5_MOEDA)) == "TC" .And. Upper(AllTrim(SE5->E5_NATUREZ)) == cMvNatTrc //Troco
927         If nOpcConf == 2
928             cForma      := AllTrim(cMoedaSimb)
929             cDesForma   := STDDescFP(cForma)
930         Else
931             cForma      := "TC"

```

```

932     EndIf
933     cDesForma := iIf(!Empty(cDesForma), cDesForma, STR0001) //"ENTRADA DE TROCO"
934
935 ElseIf AllTrim(SE5->E5_TIP00C) == "TR" .And. Upper(AllTrim(SE5->E5_NATUREZ)) == cMvNatSang //Sangria
936     If nOpcConf == 2
937         cForma      := AllTrim(SE5->E5_MOEDA)
938         cDesForma    := STDDescFP(cForma)
939         nValFP       := nValFP * -1 // Transforma o valor em negativo para subtrair o valor e a quantidade
940     Else
941         cForma      := "SG"
942     EndIf
943     cDesForma := iIf(!Empty(cDesForma), cDesForma, STR0002) //"SANGRIA"
944 ElseIf Upper(AllTrim(SE5->E5_HISTOR)) == Upper(AllTrim("CORRESPONDENTE BANCARIO")) .Or. (Upper(AllTrim(SE5->E5_NATUREZ)) == Upper(AllTrim(SuperGetMv("MV_NATCB", ""))))
945     .Or. Upper(AllTrim(SE5->E5_NATUREZ)) == SuperGetMV("MV_NATTEF") )
946     cForma      := "CB"
947     cDesForma := STR0004 //"CORRESPONDENTES BANCARIOS"
948
949 ElseIf SubSTR(Upper(AllTrim(SE5->E5_HISTOR)),1,18) == Upper(AllTrim("RECARGA DE CELULAR"))
950     cForma      := "RCE"
951     cDesForma := STR0005 //"RECARGA DE CELULAR"
952 EndIf
953
954 If !Empty(cForma)
955     If (nPos := aScan(aRet,{|x| AllTrim(x[POS_FP]) == AllTrim(cForma)}) ) > 0
956         //Quando dinheiro e for sangria nao deve somar nem subtrair a quantidede
957         aRet[nPos][POS_QTDE] += IIf(nValFP < 0, IIf(AllTrim(cMoedaSimb) == AllTrim(cForma),0,-1),1)
958         aRet[nPos][POS_VALAPU] += nValFP
959     Else
960         If !lDetADMF
961             aAdd(aRet,{PadR(cForma,6),PadR(cDesForma,TamSX3("X5_DESCRI")[1]),0,0,0,0,"",""})
962         Else
963             aAdd(aRet,{PadR(cForma,6),PadR(cDesForma,TamSX3("X5_DESCRI")[1]),"", "",0,0,0,0})
964         Endif
965         aRet[Len(aRet)][POS_QTDE] += 1
966         aRet[Len(aRet)][POS_VALAPU] := nValFP
967     Endif
968 EndIf
969 SE5->(DBSkip())
970 End
971
972 If ChkFile("MHJ") .AND. ChkFile("MHK")
973     //Recebimentos de Titulos
974     DbSelectArea("MHK")
975     cFiltro := "MHK_FILIAL = '" + xFilial("MHK") + "' .AND. DToS(MHK_DATMOV) >= '" + aChave[2] + "' .AND. DToS(MHK_DATMOV) <= '" + DToS(dDataBase) + "' " + ";
976     " .AND. MHK_BANCO = '" + aChave[3] + "' .AND. MHK_NUMMOV = '" + cNumMov + "'"
977
978     MHK->(DbSetFilter({|&cFiltro},cFiltro))
979     MHK->(dbGoTop())
980
981     While !MHK->(EOF())
982

```

```

983      cForma      := ""
984      cDesForma    := ""
985      nValFP       := MHK->MHK_VALOR
986
987      If nOpcConf == 2
988          cForma      := AllTrim(MHK->MHK_TIPOPG)
989          cDesForma    := STDDescFP(cForma)
990          cCodAdmins   := MHK->MHK_CODADM
991          cDescAdmins  := MHK->MHK_DESADM
992      Else
993          cForma      := "REC"
994      EndIf
995
996      If !Empty(cForma)
997          If !DetADMF .AND. (AllTrim(cForma) == "CC" .OR. AllTrim(cForma) == "CD" )
998              nPos := aScan(aRet,{|x| AllTrim(x[POS_CODADM]) == AllTrim(cCodAdmins)})
999          Else
1000              nPos := aScan(aRet,{|x| AllTrim(x[POS_FP]) == AllTrim(cForma)})
1001          Endif
1002
1003          If nPos > 0
1004              //Quando dinheiro e for sangria nao deve somar nem subtrair a quantidade
1005              aRet[nPos][POS_QTDE] += IIf(nValFP < 0, IIf(AllTrim(cMoedaSimb) == AllTrim(cForma),0,-1),1)
1006              aRet[nPos][POS_VALAPU] += nValFP
1007          Else
1008              If !DetADMF
1009                  aAdd(aRet,{PadR(cForma,6),PadR(cDesForma,TamSX3("X5_DESCRI")[1]),0,0,0,0,"",""})
1010              Else
1011                  aAdd(aRet,{PadR(cForma,6),PadR(cDescAdmins,TamSX3("X5_DESCRI")[1]),cCodAdmins,PadR(cDescAdmins,TamSX3("X5_DESCRI")[1]),0,0,0,0})
1012              Endif
1013              aRet[Len(aRet)][POS_QTDE] += 1
1014              aRet[Len(aRet)][POS_VALAPU] := nValFP
1015          Endif
1016      EndIf
1017
1018      MHK->(DBSkip())
1019  End
1020
1021      SE5->(dbClearFilter())
1022      MHK->(dbClearFilter())
1023
1024  Else
1025      LjGrvLog("STDConfMov","Tabelas MHJ e/ou MHK não encontrada, a ausencia destas tabelas implicara na conferência de caixa não listando os recebimentos realizados.")
1026  Endif
1027
1028  Return aRet
1029
1030  //-----
1031  /*/{Protheus.doc} STDAjustNum
1032  Remove os registros (formas) que não possui valores a serem exibidos na conferencia.
1033
1034  @param      aNumerario - Array com as formas e valores para o fechamento

```

```
1035 @return      aRet - Array com as formas e valores para o fechamento sem as formas em branco.
1036 @obs
1037 @sample
1038 /*/
1039 //-----
1040 Static Function STDAjustNum(aNumerario)
1041 Local aRet      := {}
1042 Local aCopyNum  := aClone(aNumerario)
1043 Local nI       := 1
1044 Local lConfCega := SuperGetMV("MV_LJEXAPU", Nil, .F.) //Utiliza a conferencia cega?
1045
1046 If !lConfCega
1047     //Remove os registros sem valores
1048     For nI := 1 To Len(aCopyNum)
1049         If aCopyNum[nI][POS_VALAPU] <> 0
1050             aAdd(aRet, aCopyNum[nI])
1051         EndIf
1052     Next
1053 EndIf
1054
1055 //Caso não tenha nenhuma movimentação será exibido todos os registros em branco
1056 If Len(aRet) == 0
1057     aRet := aClone(aNumerario)
1058 EndIf
1059
1060 Return aRet
```

Componentes/Cashier/STDOpenCash.prw

554 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4
5 Static lConfCaixa := SuperGetMV( "MV_LJCONFF",,.F. ) //Parametro da conferencia de caixa
6
7 //-----
8 /*{Protheus.doc} STDStStation
9 Verifica status da estacao e usuario
10
11 @param cStation      Estacao
12 @param cOperName     Nome do Operador
13 @return aRet[1] CAIXA_FECHADO      1 // Caixa foi fechado pelo operador
14 @return aRet[2] TROCOU_OPERADOR    2 // Houve troca de operador
15 @return aRet[3] ULTIMO_OPERADOR     3 // Ultimo operador que usou o sistema e nao fechou caixa
16 @obs
17 @sample
18 /*/
19 //-----
20 Function STDStStation( cStation , cOperName )
21
22 Local          aRet          := Array(3)                // Retorno
23
24 Default        cStation      := 0                      // Numero do item na venda
25 Default        cOperName     := ""                     // Codigo do Item
26
27 ParamType 0 Var cStation AS Character Default ""
28 ParamType 1 Var cOperName AS Character Default ""
29
30 /*/
31 Verifica status da estacao e usuario
32 /*/
33 DbSelectArea("SLI")
34 DbSetOrder(1) //LI_FILIAL+LI_ESTACAO+LI_TIPO
35 If DbSeek ( xFilial("SLI")+PadR(cStation,4)+"OPE")
36
37 If Empty(SLI->LI_MSG)
38 aRet[CAIXA_FECHADO] := .T.
39 Else
40 aRet[CAIXA_FECHADO] := .F.
41 EndIf
42
43 If AllTrim(SLI->LI_USUARIO) == AllTrim(cOperName)
44 aRet[TROCOU_OPERADOR] := .F.
45 Else
46 aRet[TROCOU_OPERADOR] := .T.
47 EndIf
48
49 // Ultimo operador que usou o sistema e nao fechou caixa
50 aRet[ULTIMO_OPERADOR] := SLI->LI_USUARIO
51
52 EndIf

```

```

53
54
55 Return aRet
56
57
58 //-----
59 /*{Protheus.doc} STDNumMov
60 Retorna o ultimo numero de movimento
61
62 @param  aFieldsRet  Campos que devem ser retornados (caracter passar apenas 1 | Array ilimitado)
63 @return  uRetorno   Caixa foi fechado pelo operador / Campos solicitados
64
65 @obs
66 @sample
67 */
68 //-----
69
70 Function STDNumMov( aFieldsRet )
71
72 Local aStation := STBInfoEst(1,.T.,.F.) // Retorna Informacoes da estacao
73 Local aArea    := GetArea()           // Guarda Area
74 Local nI       := 0                   // Variavel de controle
75 Local lRetCmp  := .F.                 // Retornara campos
76 Local aLstCmp  := {}                 // Guarda Lista de campos validados
77 Local uRetorno := Nil                 // Retorno Indefinido
78 Local aLstCmpPad := {"LW_NUMMOV","LW_DTFECHA"} // Lista de Campos padrao para consulta
79 Local cArq      := ""                 // Arquivo
80 Local cIndice    := ""                 // Indice
81 Local cFiltro    := ""                 // Filtro
82 Local nInd       := 0                 // Guarda Index
83 Local cQry       := ""                 // Query
84 Local cAlias     := ""                 // Alias
85 Local cSGBD      := ""                 // guarda Gerenciador de banco de dados
86 Local nPos       := 0                 // Posicao
87 Local aEstru     := {}                 // Estrutura
88 Local CNumMov    := ""                 // Movimento
89
90 Default aFieldsRet := {} // aFieldsRet Campos que devem ser retornados
91
92 ParamType 0 Var aFieldsRet As Array Default {}
93
94 //Caso existam, validar campos de retorno solicitados
95 Do Case
96   Case ValType(aFieldsRet) == "C"
97     If !Empty(aFieldsRet)
98       aAdd(aLstCmp,AllTrim(Upper(aFieldsRet)))
99     Endif
100   Case ValType(aFieldsRet) == "A"
101     If Len(aFieldsRet) > 0
102       For nI := 1 to Len(aFieldsRet)
103         If ValType(aFieldsRet[nI]) == "C" .AND. !Empty(aFieldsRet[nI])
104           aAdd(aLstCmp,AllTrim(Upper(aFieldsRet[nI])))

```

```

105         Endif
106     Next nI
107 Endif
108 EndCase
109
110 //Verificar os campos solicitados
111 If Len(aLstCmp) > 0
112     aFieldsRet := {}
113     dbSelectArea("SX3")
114     SX3->(dbSetOrder(2))
115     For nI := 1 to Len(aLstCmp)
116         If Substr(aLstCmp[nI],1,3) # "LW_"
117             aLstCmp[nI] := "LW_" + aLstCmp[nI]
118         Endif
119         //Caso o campo nao tenha sido utilizado ainda, adicionar a lista
120         If aScan(aFieldsRet,{|x| x == aLstCmp[nI]}) == 0
121             If SX3->(dbSeek(aLstCmp[nI]))
122                 aAdd(aFieldsRet,aLstCmp[nI])
123             Endif
124         Endif
125     Next nI
126     If Len(aFieldsRet) > 0
127         lRetCmp := .T.
128     Endif
129 Endif
130
131 If (AllTrim(CNumMov) == "" .OR. lRetCmp)
132
133     #IFDEF TOP
134
135         cAlias      := GetNextAlias()
136
137         aEstru := SLW->(dbStruct())
138         cSGBD := Upper(AllTrim(TcGetDB()))
139
140         cQry := "SELECT DISTINCT LW_NUMMOV, LW_DTFECHA "
141
142         If lRetCmp
143             For nI := 1 to Len(aFieldsRet)
144                 If aScan(aLstCmpPad,{|x| x == aFieldsRet[nI]}) == 0
145                     cQry += "," + aFieldsRet[nI] + Space(1)
146                 Endif
147             Next nI
148         Endif
149
150         cQry += "FROM " + RetSQLName("SLW") + " "
151         cQry += "WHERE LW_FILIAL = '" + xFilial("SLW") + "' AND LW_PDV = '" + aStation[4] + "' AND LW_OPERADO = '" + aStation[1] + "' "
152
153         Do Case
154             Case cSGBD $ "MSSQL|SYBASE"
155                 cQry += "AND (LEN(LW_DTFECHA) = 0) "
156             Case cSGBD $ "MYSQL|POSTGRES|INFORMIX"

```

```

157         cQry += "AND (LENGTH(LW_DTFECHA) = 0) "
158     Case cSGBD $ "DB2|DB2/400|SQLITE"
159         cQry += "AND (LENGTH(TRIM(LW_DTFECHA)) = 0) "
160     Case cSGBD $ "ORACLE"
161         cQry += "AND NVL2(TRIM(LW_DTFECHA),8,0) = '0' "
162     Otherwise
163         cQry += "AND (LEN(LW_DTFECHA) = 0) "
164     EndCase
165
166     cQry += "ORDER BY LW_DTFECHA DESC, LW_NUMMOV ASC"
167     cQry := ChangeQuery(cQry)
168
169     // Importante: Por utilizar funcao build in de SGBD, nao aplicar o PARSER.
170     dbUseArea(.T., __cRDD, TcGenQry(, , cQry), cAlias, .T., .F.)
171     // Se retorna Campos
172     If lRetCmp
173         For nI := 1 to (cAlias)->(FCount())
174             If (nPos := aScan(aEstru, {|x| AllTrim(x[1]) == AllTrim((cAlias)->(FieldName(nI)))}) > 0
175                 If aEstru[nPos][2] # "C"
176                     TcSetField(cAlias, aEstru[nPos][1], aEstru[nPos][2], aEstru[nPos][3], aEstru[nPos][4])
177                 Endif
178             Endif
179         Next nI
180     Endif
181
182     (cAlias)->(dbGoTop())
183
184     If !(cAlias)->(Eof())
185         CNumMov := (cAlias)->LW_NUMMOV
186         If lRetCmp
187             uRetorno := Array(Len(aFieldsRet))
188             For nI := 1 to Len(aFieldsRet)
189                 uRetorno[nI] := (cAlias)->&(aFieldsRet[nI])
190             Next nI
191         Else
192             uRetorno := CNumMov
193         Endif
194     Else
195         uRetorno := ""
196     Endif
197
198     FechaArqT(cAlias)
199
200 #ELSE
201
202     //Buscar o movimento em aberto
203     dbSelectArea("SLW")
204     cArq := CriaTrab(, .F.)
205     cIndice := "DTOS(LW_DTABERT)"
206     cFiltro := "Empty(LW_DTFECHA) .AND. LW_FILIAL = '" + xFilial("SLW") + "' .AND. LW_PDV = '" + aStation[4] + "' .AND. LW_OPERADO = '" + aStation[1] + "'"
207     IndRegua("SLW", cArq, cIndice, "D", cFiltro, "", .F.)
208     dbSelectArea("SLW")

```

```

209     nInd := RetIndex("SLW")
210     dbSetIndex(cArq + OrdBagExt())
211     SLW->(dbSetOrder(nInd + 1))
212     SLW->(dbGoTop())
213     If !SLW->(Eof())
214         CNumMov := SLW->LW_NUMMOV
215         If lRetCmp
216             uRetorno := Array(Len(aFieldsRet))
217             For nI := 1 to Len(aFieldsRet)
218                 uRetorno[nI] := SLW->&(aFieldsRet[nI])
219             Next nI
220         Else
221             uRetorno := CNumMov
222         Endif
223     Else
224         uRetorno := ""
225     Endif
226
227     //Retornar ao estado anterior
228     dbSelectArea("SLW")
229     dbClearFilter()
230     RetIndex("SLW")
231     If File(cArq + OrdBagExt())
232         fErase(cArq + OrdBagExt())
233     Endif
234
235     #ENDIF
236
237     RestArea(aArea)
238
239 Else
240     uRetorno := CNumMov
241 Endif
242
243 Return uRetorno
244
245
246 //-----
247 /*/{Protheus.doc} STDOpenCash
248 Abre o caixa
249 @param cCash          Caixa
250 @param dDtOpen        Data de abertura
251 @param cHrOpen        Hora de abertura
252 @return lRet Retorna se abriu o caixa
253 @obs
254 @sample
255 */
256 //-----
257 Function STDOpenCash( cCash , dDtOpen , cHrOpen )
258
259 Local aArea          := GetArea()          // Guarda area
260 Local lRet           := .F.                // Retorno

```

```

261
262 Default cCash          := ""                // Caixa
263 Default dDtOpen        := CtoD(" / / ")    // Data de abertura
264 Default cHrOpen        := " : "           // Hora de abertura
265
266
267 ParamType 0 Var         cCash              As Character      Default ""
268 ParamType 1 Var         dDtOpen            As Date           Default CtoD(" / / ")
269 ParamType 2 var         cHrOpen            As Character      Default " : "
270
271 DbSelectArea("SA6")
272 DbSetOrder(2)//A6_FILIAL+A6_NOME
273 DbGotop()
274 If MsSeek( xFilial("SA6") + Upper(cUserName) )
275
276     RecLock("SA6",.F.)
277
278     SA6->A6_DATAABR       := dDtOpen
279     SA6->A6_HORAABR       := cHrOpen
280     SA6->A6_DATAFCH       := CtoD(" / / ")
281     SA6->A6_HORAFCH       := " : "
282
283     MsUnLock()
284
285     lRet := .T.
286
287 EndIf
288
289 RestArea(aArea) // Restaura area
290
291 Return lRet
292
293 //-----
294 /*{Protheus.doc} STDOpenMult
295 Abre caixas associados (tratamento multi-moeda)
296 @param cCash          Caixa
297 @param dDtOpen        Data de abertura
298 @param cHrOpen        Hora de abertura
299 @return lRet Retorna se abriu os caixas
300 @obs
301 @sample
302 */
303 //-----
304 Function STDOpenMult( cCash , dDtOpen , cHrOpen )
305
306 Local aArea          := GetArea()          // Guarda area
307 Local lRet            := .T.                // Retorno
308 Local cCodigo         := ""                // Codigo do caixa principal
309 Local cAgencia        := ""                // Agencia do caixa principal
310 Local cConta          := ""                // Conta do caixa principal
311
312 Default cCash         := ""                // Caixa

```

```

313 Default dDtOpen      := CtoD(" / / ")      // Data de abertura
314 Default cHrOpen      := " : "              // Hora de abertura
315
316
317 ParamType 0 Var        cCash                As Character      Default ""
318 ParamType 1 Var        dDtOpen              As Date           Default CtoD(" / / ")
319 ParamType 2 var        cHrOpen              As Character      Default " : "
320
321 // Abre caixas associados(tratamento multi-moeda)
322 xNumCaixa()
323 cCodigo      := SA6->A6_COD
324 cAgencia     := SA6->A6_AGENCIA
325 cConta       := SA6->A6_NUMCON
326
327 DbSelectArea("SA6")
328 DbSetOrder(1) //A6_FILIAL+A6_COD+A6_AGENCIA+A6_NUMCON
329 If DbSeek( xFilial("SA6") + Upper(cCash) )
330
331     While !Eof() .And. ( xFilial()+cCodigo ) == ( SA6->A6_FILIAL+SA6->A6_COD )
332
333         If AllTrim(cCodigo + cAgencia + cConta) == AllTrim(SA6->A6_COD + SA6->A6_AGENCIA + SA6->A6_NUMCON)
334             SA6->(dbSkip())
335             Loop
336         EndIf
337
338         RecLock("SA6",.F.)
339
340         SA6->A6_DATAABR      := dDtOpen
341         SA6->A6_HORAABR      := cHrOpen
342         SA6->A6_DATAFCH      := CtoD(" / / ")
343         SA6->A6_HORAFCH      := " : "
344
345         MsUnLock()
346
347         SA6->(dbSkip())
348     EndDo
349 EndIf
350
351 EndIf
352
353 RestArea(aArea) // Restaura area
354
355 Return lRet
356
357
358 //-----
359 /*{Protheus.doc} STDSaveMov
360 Grava Movimento da Abertura de Caixa sendo que:
361 A cada troca de "PDV , Operador ou , Data"
362 o Numero Passa a Ser 1 (Um)
363 @param aDtHr      Array de data e hora
364 @param cCupomIni  Numero do Cupom inicial

```

```

365 @return lRet    Retorna se realizou a gravacao
366 @obs
367 @sample
368 /*/
369 //-----
370 Function STD0SaveMov( aDtHr , cCupomIni)
371
372 Local aArea      := GetArea()                // Guarda area
373 Local aStation   := {}                      // Informacoes da estacao // [1]-CAIXA [2]-ESTACAO [3]-SERIE [4]-PDV
374 Local lRet       := .T.                     // Retorno
375 Local cChave      := ""                     // Chave de Pesquisa
376 Local cNumMov     := ""                     // Numero do Movimento
377
378 Local lConfCega   := SuperGetMV("MV_LJEXAPU",.T.,.F.) == .F. //Conferencia cega?
379 Local lDetADMF    := SuperGetMV("MV_LJDESM",.T.,.F.)         //Conferencia detalhada por adm. financeira?
380 Local cOpcExib    := ""                                     //Grava informacao sobre exibicao da Administradora Financeira
381
382 Default aDtHr      := STBDtCash()
383 Default cCupomIni  := ""
384
385 ParamType 0 Var    aDtHr           As Array      Default STBDtCash()
386 ParamType 1 Var    cCupomIni       As Character  Default ""
387
388 //Tratamento especifico para conferencia de caixa
389 If lConfCaixa
390     aStation := STBInfoEst( 1, .T. )
391
392     If !lConfCega .AND. lDetADMF
393         cOpcExib := "3" //Detalha a conferencia por administradoras financeiras e não mostra a coluna de valores apurados
394     ElseIf !lConfCega
395         cOpcExib := "1" //Não exibe a coluna de valores apurados e não detalha a conferencia por administradoras financeiras
396     ElseIf lDetADMF
397         cOpcExib := "2" //Exibe a coluna de valores apurados e detalha a conferencia por administradoras financeiras
398     Else
399         cOpcExib := "0" //Exibe a coluna de valores apurados e não detalha a conferencia por administradoras financeiras
400     EndIf
401 Else
402     aStation := STBInfoEst( 1, .T. )// , .T. )
403 EndIf
404
405 //Numero da movimentação da SLW
406 cNumMov := STDGerMov(aStation, aDtHr)
407
408 // Grava SLW
409 RecLock("SLW", .T.)
410
411 Replace SLW->LW_FILIAL      With xFilial("SLW")           // Filial
412 Replace SLW->LW_PDV         With aStation[4]              // PDV
413 Replace SLW->LW_OPERADO     With aStation[1]              // Caixa
414 Replace SLW->LW_DTABERT     With aDtHr[1]                // Data
415 Replace SLW->LW_HRABERT     With aDtHr[2]                // Hora
416 Replace SLW->LW_NUMMOV      With cNumMov                 // Numero do Movimento

```

```

417
418 If Empty(SerieNfId("SLW",1,"LW_SERIE",dDataBase,LjEspecieNF(),aStation[3]))           // Serie Movimento de Processo de Vendas
419     //Se nao achou campo na funcao fiscal faz a gravação manual na tabela SLW
420     Replace SLW->LW_SERIE           With aStation[3]                               // Numero da serie
421 EndIf
422
423 Replace SLW->LW_NUMINI           With cCupomIni                               // Cupom inicial
424 Replace SLW->LW_ESTACAO           With aStation[2]                               // Estacao
425 Replace SLW->LW_SITUA           With Replicate("0",TamSX3("LW_SITUA")[1]) // Determina situacao como pendente de transmissao "00"
426 Replace SLW->LW_ORIGEM           With cModulo                               // Modulo
427 Replace SLW->LW_CONFERE           With "1"                               // Tipo de conferencia
428 Replace SLW->LW_OPCEXIB           With cOpcExib                               // Grava informacao sobre exibicao da Administradora Financeira
429
430 MsUnlock()
431
432 If lConfCaixa
433
434     cChave := SLW->(LW_FILIAL + LW_PDV + LW_OPERADO + DTOS(LW_DTABERT) + LW_ESTACAO + LW_NUMMOV)
435
436     If !Empty(cChave) .AND. Len(aDtHr) > 0
437
438         STFSLICreate(AllTrim(aStation[1]),"ABR",StrZero(3,2) + cChave,"ABANDONA",.T.,AllTrim(UsrRetName(__cUserID)),aDtHr[1],aDtHr[2])
439
440     EndIf
441
442 EndIf
443
444
445 RestArea(aArea)
446
447 Return lRet
448
449 //-----
450 /*{Protheus.doc} STDNumCash
451 Devolve o Numero do Caixa Ativo
452
453 @return cNumCash   Retorna numero do caixa Ativo
454 @obs
455 @sample
456 */
457 //-----
458 Function STDNumCash()
459
460 Local cNumCash      := ""           // Numero do caixa
461 Local aArea         := GetArea()    // Guarda area
462
463 SA6->(dbSetOrder(2))
464 cNumCash := Iif(SA6->(dbSeek(xFilial( "SA6" )+Upper(cUsername))),SA6->A6_COD," ")
465 SA6->(dbSetOrder(1))
466
467
468 RestArea(aArea)

```

```

469
470 Return cNumCash
471
472
473 //-----
474 /*{Protheus.doc} STDGerMov
475 Gera numero de movimento para SLW
476
477 @return  cNumMov    Retorna novo numero de movimento
478 @obs
479 @sample
480 */
481 //-----
482 Function STDGerMov(aStation, aDthHr)
483
484 Local cChave      := ""
485 Local cNumMov     := ""
486 Local nTamNumMov  := TamSX3("LW_NUMMOV")[1]
487
488 Default aStation  := {}
489 Default aDthHr    := {}
490
491 ParamType 0 Var aStation  As Array  Default {}
492 ParamType 1 Var aDthHr    As Array  Default {}
493
494 If lConfCaixa
495
496     cChave := xFilial("SLW") + aStation[4] + aStation[1] + aStation[2] + DtoS(aDthHr[1])
497
498     DbSelectArea("SLW")
499     DbSetOrder(5) //LW_FILIAL + LW_PDV + LW_OPERADO + LW_ESTACAO + DTOS(LW_DTABERT) + LW_NUMMOV
500     If DbSeek( cChave )
501
502         While !SLW->(Eof()) .AND. SLW->(LW_FILIAL + LW_PDV + LW_OPERADO + LW_ESTACAO + DtoS(LW_DTABERT)) == cChave
503             cNumMov := SLW->LW_NUMMOV
504             SLW->(DbSkip())
505         EndDo
506
507         cNumMov := Soma1(AllTrim(cNumMov),nTamNumMov)
508
509     Else
510         cNumMov := StrZero( 1, nTamNumMov )
511     EndIf
512
513 Else
514
515     cChave := xFilial("SLW") + aStation[4] + aStation[1] + DtoS(aDthHr[1])
516
517     DbSelectArea("SLW")
518     DbSetOrder(1) //LW_FILIAL+LW_PDV+LW_OPERADO+DTOS(LW_DTABERT)+LW_NUMMOV
519     If DbSeek( cChave )
520

```

```

521      While !SLW->(Eof()) .AND. ( SLW->(LW_FILIAL + LW_PDV + LW_OPERADO + DtoS(LW_DTABERT)) == cChave )
522          cNumMov := SLW->LW_NUMMOV
523          SLW->(DbSkip())
524      EndDo
525
526      cNumMov := Soma1(AllTrim(cNumMov),TamSX3("LW_NUMMOV")[1])
527
528      Else
529          cNumMov := StrZero( 1, nTamNumMov )
530      EndIf
531
532 EndIf
533
534 Return cNumMov
535
536
537 //-----
538 /*{Protheus.doc} STDGtLstMov
539 Pega data do ultimo fechamento do caixa
540
541 @return dDtLastMov      Retorna data do fechamento do caixa
542 @obs
543 @sample
544 */
545 //-----
546 Function STDGtLstMov()
547
548 Local dDtLastMov := CtoD("  /  / ")      //Pega data do ultimo fechamento antes de abrir
549
550 xNumCaixa()
551
552 dDtLastMov      := SA6->A6_DATAFCH
553
554 Return dDtLastMov

```